

PRISEC IV: Context-Aware Cryptographic Selection for IoT Environments

Guilherme de Brito Vasques

Master in Computer Engineering

Supervisor:

PhD, Valderi Reis Quietinho Leithardt, Assistant Professor with Aggregation,
ISCTE-IUL

September, 2026

Department of Information Science and Technology

PRISEC IV: Context-Aware Cryptographic Selection for IoT Environments

Guilherme de Brito Vasques

Master in Computer Engineering

Supervisor:

PhD, Valderi Reis Quietinho Leithardt, Assistant Professor with Aggregation,
ISCTE-IUL

September, 2026

Write your dedication here

Acknowledgements

Write your acknowledgements here, and any funding sources, if applicable.

Resumo

A Internet das Coisas (IoT) liga um número crescente de dispositivos heterogêneos e com recursos limitados, cujos requisitos de segurança variam consoante as capacidades do hardware, a sensibilidade dos dados e as exigências da aplicação. As configurações criptográficas estáticas frequentemente não conseguem equilibrar a segurança e a eficiência de recursos sob condições operacionais variáveis. As versões anteriores do PRISEC resolviam este problema ao associar diretamente um nível de segurança fixo (Guest, Basic, Advanced ou Admin) a um conjunto predefinido de cifras, aplicado de forma uniforme, independentemente do dispositivo ou do contexto. Esta dissertação apresenta o PRISEC IV, uma framework criptográfica sensível ao contexto que seleciona dinamicamente os mecanismos criptográficos para ambientes IoT ao longo de três eixos independentes: Device Fit, Security Fit e Application Fit. Estas pontuações combinam-se numa recomendação de uma cifra ou cascata de cifras, estendendo a estrutura de níveis já existente no PRISEC ao tratar esses níveis como âncoras de referência, em vez de conjuntos fixos de cifras, transformando-os de restrições rígidas em elementos facilitadores de decisões orientadas pelo contexto.

Os algoritmos AES, ChaCha20, SPECK, RECTANGLE, HIGHT e ECC foram avaliados empiricamente por meio de múltiplas métricas (tempo de encriptação e desencriptação, velocidade de transmissão, latência, sobrecarga de memória, custo de configuração e consumo energético), tanto isoladamente quanto em cascatas de cifras, fornecendo os dados medidos nos quais o modelo se baseia para tomar decisões. Esta evidência sustenta o desenvolvimento de um protótipo funcional que seleciona configurações criptográficas adequadas com base no contexto observado. O PRISEC IV estende o PRISEC ao oferecer uma abordagem adaptável e experimentalmente fundamentada para equilibrar a confidencialidade, a disponibilidade operacional e a eficiência de recursos em ambientes IoT heterogêneos.

Palavras-chave: Avaliação de Desempenho, Criptografia Leve, Internet das Coisas, Segurança Sensível ao Contexto, Seleção Criptográfica

Abstract

The Internet of Things (IoT) connects an increasing number of resource-constrained, heterogeneous devices whose security requirements vary with hardware capabilities, data sensitivity, and application demands. Static cryptographic configurations often fail to balance security and resource efficiency across changing operational conditions. Earlier PRISEC versions addressed this by mapping a fixed security tier (Guest, Basic, Advanced, or Admin) directly onto a predetermined cipher suite, applied uniformly regardless of device or context. This dissertation presents PRISEC IV, a context-aware cryptographic framework that dynamically selects cryptographic mechanisms for IoT environments along three independent axes: Device Fit, Security Fit, and Application Fit. These scores combine into a recommendation of a cipher or cascade, extending PRISEC’s existing tier structure by treating these tiers as reference anchors rather than fixed cipher suites, transforming them from rigid constraints into context-driven decision enablers.

AES, ChaCha20, SPECK, RECTANGLE, HIGHT, and ECC were empirically benchmarked across multiple metrics (encryption and decryption time, throughput, latency, memory overhead, setup cost, and energy consumption) as single ciphers and as cipher cascades, providing the measured data the model relies on to make decisions. This evidence informs the design of a functional prototype that selects appropriate cryptographic configurations based on the observed context. PRISEC IV thereby extends PRISEC by offering an adaptable, experimentally grounded approach to balancing confidentiality, operational availability, and resource efficiency in heterogeneous IoT environments.

Keywords: Context-aware security, Cryptographic selection, Internet of Things, Lightweight cryptography, Performance evaluation

Contents

Acknowledgements	i
Resumo	iii
Abstract	v
List of Figures	xi
List of Tables	xiii
List of Acronyms	xv
Chapter 1. Introduction	1
1.1. Motivations	1
1.2. Framework for PRISEC	3
1.3. Project Objectives	4
1.4. Research Objectives	4
1.5. Contributions	5
1.6. Document Structure	6
Chapter 2. Literature Review	7
2.1. Research Question 1 - Cryptographic Algorithms	7
2.1.1. Cryptography	7
2.1.2. Cryptographic Algorithms in IoT Environments	9
2.1.3. Selected Algorithms	12
2.2. Research Question 2 - Context-Aware Adaptation Mechanisms	13
2.2.1. Overview of Adaptive Security Approaches in IoT	14
2.2.2. Design Principles and Architectural Patterns	14
2.2.3. Implementation Strategies, Contextual Parameters and Mechanisms	15
2.2.4. Comparative Trade-offs Among Adaptation Mechanisms	17
2.2.5. Evaluation Metrics and Performance Trade-offs	17
2.2.6. Research Gaps and Synthesis: Positioning PRISEC IV	18
Chapter 3. Research Methodology	21
3.1. Design Science Research Methodology	21
3.2. Systematic Literature Review	23
3.2.1. Planning the Review	23
3.2.2. Conducting the Review	24

3.2.3. Reporting the Review	26
Chapter 4. Cryptographic Evaluation	27
4.1. Cryptographic Evaluation	27
4.1.1. Evaluation Methodology	27
4.1.2. Single-Cipher Performance	31
4.1.3. Cascade Performance	32
4.1.4. Impact of Hardware Acceleration	33
4.1.5. Setup and Key-Exchange Cost	34
Chapter 5. Prototype Design and Implementation	37
5.1. System Architecture	37
5.1.1. Technology Stack	37
5.2. Diagrams	38
5.2.1. Use Case Diagram	38
5.2.2. Activity Diagram	39
5.2.3. Sequence Diagram	40
5.3. Context Parameters	40
5.3.1. Device	41
5.3.2. Security	42
5.3.3. Application	43
5.4. Decision Model	44
5.4.1. Device Fit	44
5.4.2. Security Fit	46
5.4.3. Application Fit	48
5.4.4. Final Score	49
5.5. Backend Implementation	49
5.5.1. API Endpoints	49
5.5.2. Database Schema	50
5.6. Frontend Implementation	50
5.6.1. Profile	51
5.6.2. Context, Decision, and Execution	52
Chapter 6. Conclusion and Future Work	57
6.1. Conclusion	57
6.2. Key Scientific Contributions	59
6.3. Future Work	60
Bibliography	61
Appendix A. Single-Cipher Benchmark Results	65
A.1. Encryption and Decryption Time	65
A.2. Throughput	65

Appendix B. Cascade Benchmark Results	67
B.1. Encryption and Decryption Time	67
B.2. Throughput	68
Appendix C. Impact of Hardware Acceleration	71
Appendix D. Setup and ECC Key-Exchange Cost	77
D.1. Single Ciphers	77
D.2. Two-Cipher Cascades	78
D.3. Three-Cipher Cascade	79
D.4. Hardware-Acceleration Configurations	80
Appendix E. UML Diagrams	83
Appendix F. Device Fit Weighting Scheme	85
Appendix G. Security Fit Value Scales and Cipher Ranking	87
G.1. Value Scales	87
G.2. Security Strength Ranking	87
Appendix H. Application Fit Value Scales and Weights	89
H.1. Value Scales	89
H.2. Duty Cycle Base Weights	89
Appendix I. Deeper Analysis Table	91

List of Figures

2.1	Symmetric encryption process.	8
2.2	Asymmetric encryption process.	9
2.3	Selected cryptographic algorithms.	13
3.1	Design Science Research methodology process model.	22
3.2	Phases of the Systematic Literature Review process.	23
5.1	System architecture.	38
5.2	Profile creation page of the PRISEC IV prototype.	51
5.3	Profile list page of the PRISEC IV prototype.	52
5.4	Context page of the PRISEC IV prototype.	53
5.5	Recommendation result page of the PRISEC IV prototype.	53
5.6	Deeper Analysis view of the PRISEC IV prototype.	54
5.7	Live encryption execution result in the PRISEC IV prototype.	55
A.1	Encryption and decryption time by algorithm and payload size.	65
A.2	Encryption and decryption throughput by algorithm and payload size.	66
B.1	Encryption and decryption time by two-cipher cascade combination and payload size.	67
B.2	Encryption and decryption time for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.	68
B.3	Encryption and decryption throughput by two-cipher cascade combination and payload size.	69
B.4	Encryption and decryption throughput for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.	70
C.1	Baseline throughput and hardware-acceleration gain for solo ciphers, by payload size.	72
C.2	Baseline throughput and hardware-acceleration gain for lightweight-cipher pairings, by payload size.	73
C.3	Baseline throughput and per-state hardware-acceleration gain for AES-256+ChaCha20, by payload size.	74
C.4	Baseline throughput and per-state hardware-acceleration gain for AES-256+ChaCha20+AES-128, by payload size.	75

D.1	ECC handshake share of total time for single ciphers by payload size.	78
D.2	ECC handshake share of total time for two-cipher cascades by payload size.	79
D.3	ECC handshake share of total time for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.	80
D.4	ECC handshake share of total time for selected ciphers and cascades under different hardware-acceleration configurations, by payload size.	81
E.1	Use case diagram of the PRISEC IV prototype.	83
E.2	Activity diagram of the PRISEC IV prototype.	84
E.3	Sequence diagram of the PRISEC IV prototype.	84
I.1	Device fit, security fit, and application fit score breakdown for every candidate cipher and cascade.	92

List of Tables

2.1	Cryptographic algorithms considered in selected studies and in PRISEC IV.	13
2.2	Contextual dimensions considered by selected adaptive IoT security frameworks and PRISEC IV.	19
3.1	Inclusion and exclusion criteria.	25
3.2	Stages of the selection process.	26
4.1	Overview of the cryptographic algorithms selected for PRISEC IV.	27
F.1	Device fit weights by duty cycle.	85
G.1	Security level value scale.	87
G.2	Data confidentiality and data lifetime value scale.	87
G.3	Security strength ranking, by cipher and cascade.	87
H.1	Latency tolerance value scale.	89
H.2	Throughput required value scale.	89
H.3	Application fit base weights by duty cycle.	89

List of Acronyms

AES:	Advanced Encryption Standard
AES-NI:	Advanced Encryption Standard New Instructions
API:	Application Programming Interface
ARX:	Add-Rotate-XOR
ASCON:	Lightweight authenticated-encryption algorithm
AVX2:	Advanced Vector Extensions 2
AVX-512:	Advanced Vector Extensions 512
CPU:	Central Processing Unit
DDPG:	Deep Deterministic Policy Gradient
DES:	Data Encryption Standard
DoS:	Denial of Service
DSR:	Design Science Research
DTLS:	Datagram Transport Layer Security
ECC:	Elliptic Curve Cryptography
ECDH:	Elliptic Curve Diffie–Hellman
EVP:	OpenSSL high-level cryptographic interface
HKDF:	HMAC-based Key Derivation Function
IEEE:	Institute of Electrical and Electronics Engineers
IoT:	Internet of Things
KVM:	Kernel-based Virtual Machine
LTS:	Long-Term Support
ML:	Machine Learning
NIST:	National Institute of Standards and Technology
PQC:	Post-Quantum Cryptography
PRISEC:	Privacy Security
RAM:	Random-Access Memory
RFID:	Radio-Frequency Identification
RSA:	Rivest–Shamir–Adleman
SIMD:	Single Instruction, Multiple Data
SLR:	Systematic Literature Review
XOR:	Exclusive OR

CHAPTER 1

Introduction

1.1. Motivations

The Internet of Things (IoT) has become a fundamental component of modern computing infrastructures, experiencing rapid and sustained growth over recent years and enabling the interconnection of billions of devices across diverse domains such as healthcare, agriculture, home automation, smart cities, and Industry 4.0 [1]. The number of connected IoT devices is estimated to have exceeded 20 billion in 2025, with projections indicating that this number will surpass 30 billion by 2030 [2]. This continuous expansion has led to the generation and exchange of large volumes of sensitive data, which must be protected against unauthorized access and manipulation [3], [4]. IoT systems increasingly integrate sensing, communication, and computation across devices that vary widely in processing capability, available memory, energy supply, and hardware support. This heterogeneity makes secure communication a design challenge rather than a simple matter of selecting the strongest available cryptographic algorithm.

The importance of IoT does not stem solely from the number of connected objects. IoT systems increasingly mediate interactions between the physical and digital worlds. Devices may observe environmental conditions, monitor human activity, support medical care, manage domestic environments, control industrial processes, or participate in infrastructure services [1]. The information generated in these settings can therefore vary widely in sensitivity and consequence. A measurement may represent routine system telemetry, while another may reveal a person's location, health condition, behavior, or presence in a particular environment. Similarly, an instruction sent to an actuator may have consequences that extend beyond the communication channel itself. The security of an IoT system must therefore be considered in relation to the meaning and use of the data, not only to the existence of a network connection.

The distributed nature of IoT also changes where computation and security operations take place. Data may be collected at a constrained endpoint, transmitted through a wireless network, processed at an edge node, and stored or further analysed in a cloud service. Each stage introduces a potential exposure point and may involve different assumptions about trust, availability, and resource capacity. The endpoint may not have the same protection mechanisms as the gateway or cloud layer, and it may remain physically accessible to an attacker for long periods. As a result, security cannot be treated solely as a property of a central server. It must also be supported by the devices that originate, receive, and initially process the data.

Despite the widespread adoption of IoT across various fields, many IoT devices are designed with strict constraints, primarily driven by cost, size, and deployment needs. Size limitations require IoT devices to accommodate low-power microprocessors, small-capacity power supply units, and other small electronic components. Consequently, such devices typically rely on lightweight or stripped-down operating systems that provide only the bare minimum essential functions and features, minimizing memory and processing overhead [5]. As a result, these devices are fundamentally limited in terms of computational power, memory capacity, and energy availability [6], and the difficulty is therefore not that cryptographic algorithms are unavailable, but that their suitability depends on the context in which they are used.

Securing IoT environments presents significant challenges, as it requires protecting sensitive information while ensuring timely, reliable access to services. In this context, confidentiality and availability are key security objectives for IoT systems. Due to hardware limitations, IoT systems are often deployed with reduced security configurations, making them particularly vulnerable to a range of malicious attacks that can lead to unauthorized data access, data theft, or service disruptions. The security of an IoT deployment must therefore account for protection against unauthorized disclosure while preserving the operation of the services that depend on the devices.

Confidentiality is vital in IoT systems because devices often handle personal and sensitive information, such as health data from wearable devices or data generated in smart home environments. Attacks such as eavesdropping and spoofing directly threaten confidentiality by enabling unauthorized parties to intercept and access transmitted data [7]. The security of this sensitive data primarily relies on cryptography, which encrypts data so that its content remains inaccessible to any party lacking the authorization to access it. IoT communications may also require mechanisms that support integrity, authentication, and secure key management. A message must not only be concealed, but also protected against undetected modification and associated with an appropriate communicating entity. These requirements influence the choice of cryptographic mechanisms and the way in which they are combined within a system.

On the other hand, ensuring availability means that IoT systems remain accessible and operational when authorized users need them. Attacks targeting availability, such as denial-of-service (DoS) or flooding attacks, aim to exhaust device or network resources [7]. This makes services inaccessible or significantly degrades their performance: an attacker who floods the device with requests it cannot process in time causes those requests to be delayed or dropped entirely, denying the service to legitimate users [8]. To address these challenges, edge computing has emerged as an essential paradigm in IoT, enabling data processing closer to the data source and thereby reducing latency, network congestion, and reliance on centralized cloud infrastructure. Because most processing happens near the device, less data has to travel across the network before it can be acted on, allowing

edge computing to improve service availability while also limiting how much sensitive data is exposed on the internet.

Availability can also be affected without a deliberate denial-of-service attack. Excessive cryptographic computation may delay sensor data processing, increase response times, drain battery power, or compete with other essential tasks. An operation that is individually inexpensive may become significant when repeated thousands of times during the device's lifetime. For this reason, cryptographic overhead should be considered part of an IoT system's operational behavior. The security mechanism must protect communication without creating a new condition in which the device cannot fulfill its intended function.

A context-aware perspective does not imply that security should be reduced whenever resources are limited. Instead, it requires that the protection requirement and the available resources be considered together. A constrained device should not automatically receive the weakest mechanism, nor should it be forced to use the most expensive mechanism regardless of the data and workload. The decision should reflect the relationship among what the request requires, what the device can support, and what the application can tolerate.

The challenge is therefore not simply to find a cryptographic algorithm that is fast, lightweight, or strong in isolation. It is to reason about the interaction between cryptographic protection, device limitations, data requirements, workload characteristics, and service availability. These factors influence one another, but none can be completely represented by a single value. Security selection must consequently balance several forms of cost and several forms of protection without assuming that one dimension always dominates the others.

Considering these challenges, this dissertation's motivation lies in the clear need for security mechanisms that can adapt to the dynamic, heterogeneous nature of IoT environments. The main goal is to design and develop a context-aware security approach for IoT systems, capable of dynamically selecting and configuring cryptographic settings based on contextual factors, including the type and sensitivity of the information being processed, the computational and energy capabilities of the device, target throughput needs, and acceptable encryption and decryption latencies. By adapting the security configurations to these context parameters, the proposed approach aims to effectively balance strong data protection with efficient resource use, ensuring that confidentiality and availability requirements are met without sacrificing system performance.

1.2. Framework for PRISEC

The PRISEC (Privacy Security) framework is a cryptographic security framework designed to protect data communications in Internet of Things (IoT) environments. The framework adopts a flexible, multi-layered security model that enables selecting and configuring cryptographic mechanisms based on the environment, data sensitivity, and device capabilities.

PRISEC IV builds upon the principles established in earlier versions of the framework, namely PRISEC I and PRISEC II, which focused on the evaluation and benchmarking of cryptographic algorithms suitable for IoT environments, and PRISEC III, which introduced a structured, access-level-based approach to cryptographic selection [9], [10], [11]. In contrast to its predecessors, the framework proposed in this dissertation extends PRISEC by introducing a context-aware security model. Rather than relying solely on static access levels to directly determine cryptographic suites, the proposed framework adapts cryptographic configurations based on runtime-observed contextual parameters.

The PRISEC framework maintains the multi-level security structure inherited from PRISEC III, preserving the Guest, Basic, Advanced, and Admin levels as conceptual abstractions for different security needs. However, in the proposed approach, these levels are no longer associated with fixed cryptographic configurations. Instead, they serve as reference profiles that guide the dynamic selection and composition of cryptographic mechanisms. This enables the framework to adjust encryption strength, algorithm choice, and cryptographic layering in real time, ensuring that confidentiality and availability requirements are met under varying operational conditions.

1.3. Project Objectives

The main goal of this dissertation is to develop a context-aware security framework for Internet of Things environments that dynamically adjusts cryptographic methods based on device characteristics, data sensitivity, and system performance needs.

To achieve this objective, the following specific goals are outlined:

- Identify and assess cryptographic algorithms suitable for IoT systems, including symmetric and asymmetric, considering metrics such as security strength, encryption and decryption speed, memory usage, and energy consumption.
- Design and develop a context-aware model capable of dynamically selecting cryptographic mechanisms based on contextual factors.
- Extend the PRISEC framework to enable adaptive cryptographic configurations across different security levels while maintaining the structure from previous PRISEC versions.
- Experimentally evaluate the proposed solution.

1.4. Research Objectives

To ensure the proposed context-aware security approach for IoT environments is grounded in a strong theoretical basis, this study is supported by a Systematic Literature Review (SLR). The SLR helps identify limitations and gaps in current IoT security solutions, examines relevant frameworks, and provides the conceptual basis for the research presented in this dissertation.

The following research questions have been formulated:

- RQ1 - Which cryptographic algorithms are used in IoT systems to handle resource limitations while ensuring sufficient security?

- RQ2 - How can IoT security mechanisms be designed to dynamically adapt cryptographic configurations based on contextual information to balance security and efficient resource use in resource-constrained environments?
- RQ3 - What are the benefits of implementing a Context-Aware Cipher Selection model?

RQ1 investigates existing cryptographic algorithms and aims to establish the current landscape of cryptographic solutions by identifying which algorithms are most commonly used to address resource constraints while maintaining security. This question provides essential information to understand current practices, limitations, and trade-offs involved in choosing cryptographic mechanisms for resource-constrained environments.

RQ2 aims to explore the theoretical and practical foundations of IoT security, focusing on how adaptive cryptographic configurations can be designed to respond dynamically to operational conditions. It addresses the need to transition from static security setups to flexible, runtime-adaptive mechanisms that balance security strength with resource efficiency.

RQ3 is different in kind from the two preceding ones. Rather than being investigated through the literature review, it can only be answered after the framework is implemented and evaluated. It asks what the gains are of implementing a context-aware cipher selection model presented in this dissertation rather than using a static cryptographic configuration.

1.5. Contributions

This dissertation aims to advance the state of the art in IoT security by addressing the limitations of static cryptographic configurations in resource-constrained environments and proposing a context-aware approach to cryptographic adaptation.

From a theoretical perspective, this research contributes to the identification and analysis of cryptographic algorithms suitable for IoT systems, with particular emphasis on how existing mechanisms address the trade-off between security robustness and computational efficiency. This study also identifies key contextual parameters, such as device capabilities, data sensitivity, performance requirements, and environmental conditions that influence security decisions in adaptive IoT environments. These findings provide a structured foundation for understanding how context-aware security strategies can be designed for heterogeneous and resource-constrained systems.

From a practical and technical standpoint, the primary contribution of this work is the design and development of a context-aware security method that can dynamically choose and configure cryptographic mechanisms based on contextual information. The proposed solution extends the PRISEC framework by introducing adaptive cryptographic configurations that respond to variations in device capabilities, operational, and security requirements.

Finally, to validate the proposed approach, a functional prototype will be implemented and evaluated through experimental scenarios representative of real IoT deployments.

1.6. Document Structure

The remainder of this dissertation is organized into five chapters.

Chapter 2 presents the literature review, structured around the first two research questions. It examines cryptographic algorithms suitable for resource-constrained IoT environments and reviews context-aware and adaptive security approaches, identifying the research gaps that motivate PRISEC IV.

Chapter 3 describes the research methodology. It presents the Design Science Research process used to develop and evaluate the proposed artifact and details the Systematic Literature Review protocol, study-selection process, data extraction, and synthesis procedures.

Chapter 4 presents the benchmark methodology, before reporting results for individual ciphers, cascades, hardware acceleration, and setup and key-exchange costs.

Chapter 5 describes the design and implementation of the PRISEC IV prototype. It describes the system architecture and user interactions, defines the context parameters and decision model, and explains the backend and frontend implementation.

Finally, Chapter 6 concludes the dissertation by summarizing the contributions of PRISEC IV, discussing the benefits of its context-aware selection approach, and identifying directions for future work.

CHAPTER 2

Literature Review

This chapter provides the theoretical foundation for the dissertation by presenting a structured review of the relevant literature on security in Internet of Things environments. The Systematic Literature Review (SLR) was carried out to systematically identify and critically analyze existing approaches, concepts, and research contributions related to secure communication within constrained environments, as well as to examine the primary challenges impacting their practical implementation. The review is structured around the first two research questions outlined in Chapter 1. In this way, the chapter provides the conceptual support required for the dissertation's development.

2.1. Research Question 1 - Cryptographic Algorithms

This section investigates the first research question outlined in 1.4. Research Objectives (Which cryptographic algorithms are used in IoT systems to handle resource limitations while ensuring sufficient security?), which examines the security requirements of IoT systems, the main cryptographic approaches proposed for constrained devices, and the trade-offs among security, computational efficiency, memory usage, and energy consumption between them. In this context, conventional cryptographic algorithms are not always suitable for direct use in IoT environments because these limitations require lightweight or adapted solutions to balance security and performance.

Consequently, recent research has shifted towards lightweight and adapted conventional cryptographic models aimed at maintaining confidentiality, integrity, authenticity, and secure key management, while remaining feasible for constrained devices.

2.1.1. Cryptography

Cryptography is a fundamental technique for protecting information by taking data that otherwise be freely readable (plaintext) and converts it into an unintelligible form (ciphertext) with no meaning to anyone who lacks the corresponding key, limiting access to those the sender actually intended [12]. It supports essential security properties by securing communication over potentially insecure channels. Cryptography is broadly divided into two main categories: symmetric and asymmetric cryptography, each with distinct characteristics, advantages, and limitations [13].

2.1.1.1. Symmetric Cryptography

In symmetric cryptography, the same key that encrypts a message is also used to decrypt it, rather than using separate keys for each direction. (Figure 2.1). This approach is computationally efficient and suitable for applications that require fast processing and low

latency [14]. However, a significant limitation of this methodology lies in the requirement for secure key distribution. The secret key must be exchanged between communicating parties prior to communication, and if this key is exposed or intercepted, the confidentiality of the communication is compromised.

Two distinct designs exist within symmetric cryptography: one that works on data in fixed-size chunks, a block cipher, making it well-suited for bulk data encryption, and one that works through data continuously, one bit or byte at a time, a stream cipher, offering lower latency and making it more suitable for real-time or resource-constrained applications [15].

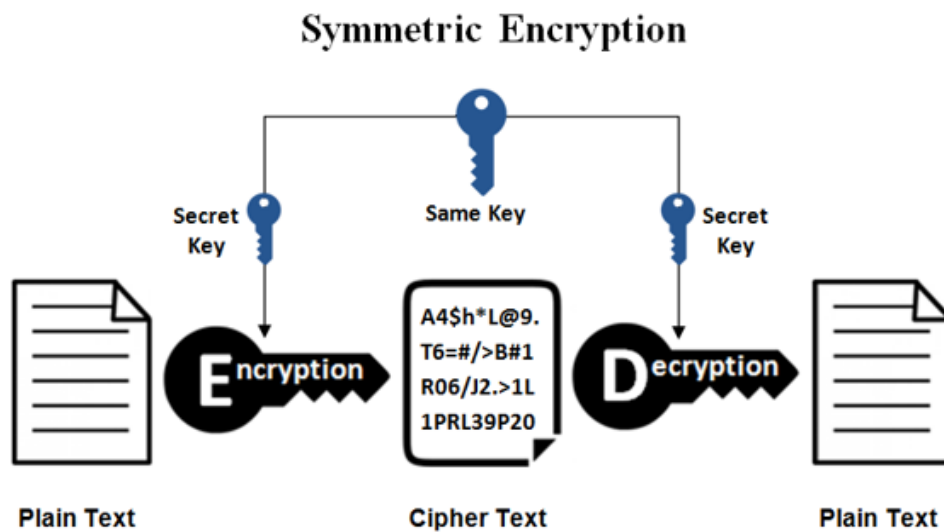


FIGURE 2.1. Symmetric encryption process.

2.1.1.2. Asymmetric Cryptography

Asymmetric cryptography, also known as public-key cryptography, involves two mathematically related keys: a public key and a private key (Figure 2.2). The public key is designated for encryption purposes, while the private key is employed for decryption, thereby obviating the need for the secure exchange of keys between communicating parties [16].

This methodology offers a resilient framework for secure communication, authentication, and key establishment, making it a fundamental component in modern security protocols. Nonetheless, asymmetric cryptography is computationally more expensive than symmetric cryptography, resulting in higher processing time and energy consumption [17].

Asymmetric Encryption

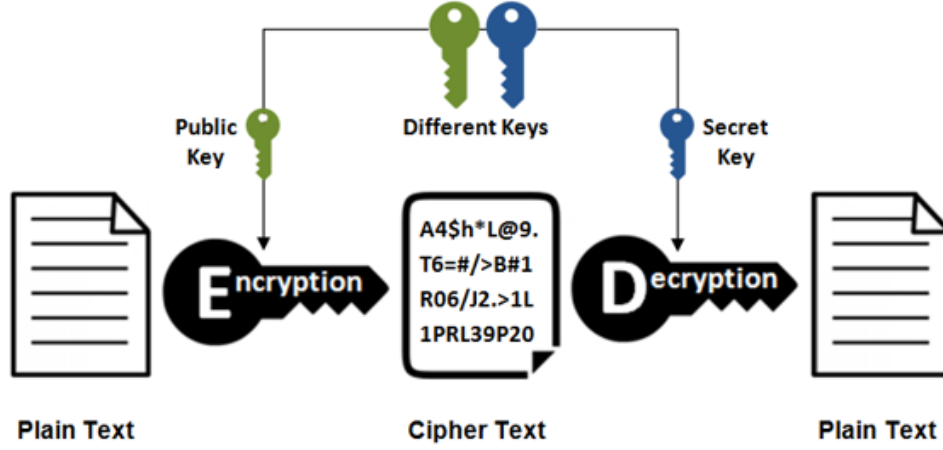


FIGURE 2.2. Asymmetric encryption process.

2.1.1.3. Lightweight Cryptography

With the rapid growth of IoT devices, traditional cryptographic techniques face significant challenges, mainly because of limitations in processing power, memory, and energy. These limitations make the direct implementation of conventional cryptographic algorithms inefficient or impractical in many IoT scenarios.

Lightweight cryptographic algorithms emerged as a solution to these challenges, designed to require fewer computational resources while maintaining essential security properties [18]. An algorithm is considered lightweight if it is optimized for software and/or hardware. This includes minimizing execution time and power consumption while maintaining high throughput and low latency, which are essential for the majority IoT devices [19], [20].

2.1.2. Cryptographic Algorithms in IoT Environments

The analysis of the selected studies delineates a diverse and heterogeneous landscape of cryptographic approaches for IoT environments. The reviewed literature indicates that no single algorithm can be regarded as universally optimal; instead, the selection of cryptographic mechanisms is consistently characterized by a context-dependent trade-off that balances security robustness against performance constraints inherent to IoT settings. The subsequent subsections present a synthesis of the principal findings, organized according to the primary cryptographic categories identified earlier.

2.1.2.1. Symmetric Cryptographic Algorithms

Symmetric cryptography remains a cornerstone of secure IoT communication because it uses a single shared key for both encryption and decryption, which generally makes it faster and less computationally demanding than asymmetric approaches, as discussed in [20].

Symmetric algorithms dominate the literature as the preferred category for data encryption in resource-constrained IoT deployments, primarily because they have lower computational overhead than asymmetric alternatives [19], [20]. Within symmetric cryptography, block ciphers are predominantly favored over stream ciphers in Internet of Things contexts due to their perceived greater adaptability and relative ease of implementation within constrained hardware and software environments, as indicated by [21].

The Advanced Encryption Standard (AES) is the most frequently cited symmetric algorithm in the reviewed studies. Its widespread adoption is attributed to its well-established security guarantees, maturity [22], flexibility in key size, standardization by the National Institute of Standards and Technology (NIST), and resistance to known cryptographic attacks [23].

Beyond AES, the literature documents a range of symmetric cryptographic algorithms. In [22], [24], the authors present a structured comparison of major symmetric-key algorithms, including DES, 3DES, AES, Blowfish, Twofish, Serpent, and ChaCha20, evaluating them on security, efficiency, implementation suitability, and limitations. In these comparative analyses, DES and 3DES are typically classified as legacy or obsolete encryption schemes due to their relatively weaker security frameworks and limited suitability for contemporary deployment environments. By contrast, Twofish and Serpent are regarded as robust and secure alternatives, although their stronger security margins are associated with slower execution and greater implementation complexity. Blowfish is presented as a practical alternative due to its straightforward implementation and satisfactory operational performance, although it is not regarded as resilient or as widely applicable as AES in current contexts [22]. Even though ChaCha20, as a stream cipher, has not achieved the same level of adoption as AES, being described as less standardized in comparative evaluations [24], its efficiency profile is nonetheless considered competitive in scenarios where computational speed is a priority.

To reinforce the argument within the context of IoT communication, R. David and P. Deepalakshmi in [25] evaluate AES and ChaCha20 against DES and two asymmetric cryptographic algorithms, RSA and ECC, using criteria such as encryption and decryption times, energy consumption, CPU usage, and communication latency. The authors assign a security score of 9 to both AES and ChaCha20, noting that they consume less energy, have lower computational overhead, and exhibit lower latency, providing a more favorable balance between security and efficiency than the asymmetric methods considered. By contrast, DES, despite lower resource use in some cases, is considered outdated due to its weaker security profile.

2.1.2.2. Asymmetric Cryptographic Algorithms

In the literature, asymmetric cryptography is predominantly regarded not as the primary mechanism for continuous data encryption but rather as an essential foundational security layer utilized for authentication, digital signatures, and the secure establishment of cryptographic keys, particularly within resource-constrained environments [24], [25].

This position reflects a recurring argument in the literature: although public-key techniques improve secure key distribution, they also incur higher computational, memory, and energy costs than lightweight symmetric alternatives typically used for frequent data protection on resource-constrained devices.

Among the asymmetric approaches most frequently discussed in the selected literature, RSA and Elliptic Curve Cryptography (ECC) are the most prominent. The scholarly works consistently regard ECC as the preferable option for resource-constrained Internet of Things (IoT) devices due to its ability to deliver robust security with significantly smaller key sizes, typically around 256 bits, compared to RSA, which often relies on substantially larger keys, such as 2048 bits, as reported in comparative IoT security evaluations [26], [12]. This distinction is substantial in the IoT context, as smaller key sizes are associated with reduced storage, communication, and computational demands, thereby enhancing the practicality and deployment efficiency of IoT devices in low-power environments.

The article by [25] enhances this interpretation by conducting a comparative analysis of ECC and RSA, focusing on parameters such as energy consumption, computational overhead, and key management costs. Within this study, ECC is consistently characterized as more efficient than RSA, exhibiting lower energy requirements, reduced encryption and decryption overhead, decreased memory usage, and diminished costs associated with key generation and exchange. Conversely, RSA is depicted as a more resource-intensive asymmetric cryptosystem and is therefore generally regarded as less suitable for constrained devices, despite remaining relevant for specific public-key functions. Nonetheless, it is important to interpret these findings as context-specific comparative evidence rather than definitive performance benchmarks, given that such metrics are influenced by implementation strategies, hardware configurations, and evaluation conditions.

Overall, asymmetric cryptography functions primarily as a supporting layer for key establishment rather than continuous data encryption, with ECC standing out for resource-limited IoT devices due to its smaller key sizes and lower computational, memory, and energy demands.

2.1.2.3. Lightweight Cryptographic Algorithms

Unlike asymmetric cryptographic methods, which are frequently characterized as more computationally demanding and therefore less suitable for low-power devices, research on lightweight IoT security predominantly concentrates on symmetric encryption techniques [20], [27]. Within this scope, block ciphers receive particular emphasis being described as more adaptable and simpler to implement in constrained environments, although stream ciphers are also recognized as relevant alternatives [20].

In [27], the authors conducted an evaluation of ten lightweight symmetric cryptographic algorithms implemented on low-power microcontroller platforms. These algorithms were categorized into two groups: five block ciphers, PRESENT, SPECK, Piccolo, CRAFT, and Hummingbird-2, and five stream ciphers, ASCON, ACORN, Lizard,

Fruit-80, and TRIVIUM. The study systematically compared these algorithms based on criteria such as RAM and ROM usage, latency, throughput, current, power, and energy consumption across multiple development boards. This comprehensive analysis provides valuable experimental insights into the suitability of these algorithms for deployment in resource-constrained IoT systems.

The findings indicate that SPECK emerges as a prominent efficiency-oriented candidate among those evaluated. It consistently demonstrated the lowest latency and highest throughput across the tested hardware platforms and was among the most energy-efficient algorithms. Nevertheless, these performance advantages were accompanied by higher RAM and ROM requirements relative to certain alternative methods. Conversely, TRIVIUM exhibited a competitive performance profile, combining high throughput with lower resource consumption. Consequently, the study suggests that a sole focus on SPECK may overlook other viable options, and that both SPECK and TRIVIUM constitute effective candidates for resource-constrained cryptographic applications.

In contrast, PRESENT demonstrates a more limited profile within the same study. Although it is recognized as highly memory-efficient, it is also characterized by elevated latency, reduced throughput, and increased energy consumption, thereby positioning it unfavorably and making it less appropriate for time-sensitive Internet of Things (IoT) applications. Similar limitations are observed for Lizard and Fruit-80, which the study categorizes among methods that face greater challenges concerning efficiency and processing power.

The study conducted in [21] provides a comparative analysis of lightweight and conventional cryptographic algorithms, including AES, RSA, RECTANGLE, SIMON, SPECK, and PRESENT. The results indicate that RECTANGLE exhibits the shortest encryption and decryption times, while SPECK also demonstrates strong performance. SIMON displays intermediate processing times, whereas PRESENT again records the slowest execution among the evaluated algorithms. This analysis underscores the variability in performance among lightweight cryptographic methods and highlights that certain ciphers, particularly SPECK and RECTANGLE, offer higher efficiency in resource-constrained IoT environments.

In [20], the scope is broader and emphasizes descriptive analysis over direct experimental implementation. The article reviews several lightweight block ciphers, including HIGHT, PRESENT, SIMON, SPECK, CLEFIA, and TWINE, and examines their structural designs, implementation orientations, and reported throughput metrics. It is noted that HIGHT is particularly suitable for deployment in severely constrained devices, while SPECK demonstrates superior software performance.

2.1.3. Selected Algorithms

The algorithms selected for analysis in this study (Table 2.3) were determined based on a comprehensive review of existing literature, which highlighted key criteria such as security strength, computational efficiency, memory utilization, energy consumption, and

suitability for resource-constrained IoT environments. Informed by these criteria, a representative set of algorithms encompassing both traditional and lightweight approaches is selected to compare their relevance, performance, and practical applicability across diverse IoT scenarios.

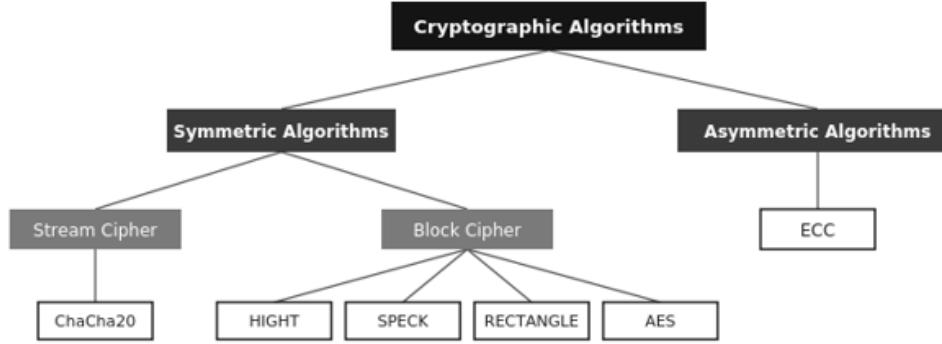


FIGURE 2.3. Selected cryptographic algorithms.

TABLE 2.1. Cryptographic algorithms considered in selected studies and in PRISEC IV.

Author/Study	AES	ChaCha20	SPECK	RECTANGLE	HIGHT	ECC	RSA
Sasikumar & Nagarajan (2024) [12]	✓	—	✓	✓	✓	✓	✓
Amrita et al. (2024) [20]	✓	—	✓	—	✓	—	—
Usha & Amuthaguka (2024) [21]	✓	—	✓	✓	—	—	✓
Banik, Maity & Chakraborty (2026) [22]	✓	—	—	—	—	—	—
Muhamad Abdullah (2017) [23]	✓	—	—	—	—	—	—
Ozaif, Mustajab & Alam (2024) [24]	✓	✓	—	—	—	—	—
David & Deepalakshmi (2025) [25]	✓	✓	—	—	—	✓	✓
Nabeel et al. (2025) [26]	✓	—	—	—	—	✓	—
Soto-Cruz et al. (2025) [27]	—	—	✓	—	—	—	—
This dissertation (PRISEC IV)	✓	✓	✓	✓	✓	✓	—

Table 2.1 summarizes the cryptographic algorithms covered, evaluated, implemented, or recommended in the studies selected for this review, alongside the catalogue adopted by PRISEC IV.

2.2. Research Question 2 - Context-Aware Adaptation Mechanisms

This section addresses the second research question outlined in 1.4. Research Objectives (How can IoT security mechanisms be designed to dynamically adapt cryptographic configurations based on contextual information to balance security and efficient resource use in resource-constrained environments). The analysis examines the engineering of adaptive mechanisms across heterogeneous devices and various usage scenarios. Emphasis is placed on how existing approaches capture and interpret contextual parameters, how these parameters drive runtime cryptographic decisions, and the subsequent impact of these adaptations on the security-resource trade-off.

2.2.1. Overview of Adaptive Security Approaches in IoT

Across the selected studies, adaptive security usually involves three main functions: ongoing context monitoring, a decision engine that links the context to a security profile, and enforcement that applies the selected cryptographic settings at runtime. In most cases, these mechanisms choose among different algorithms, key sizes, or security levels rather than selecting a single "best" cipher.

From a design perspective, three main classes of adaptive mechanisms emerge: Rule-based schemes set explicit thresholds and policies that directly connect resource states or risk levels to specific security modes. A second group of approaches uses fuzzy inference and semantic models to handle uncertainty in context measures. Finally, there are frameworks driven by machine learning, in which the training and application of machine-learning models are utilized to inform the selection of cryptographic methodologies.

Overall, this panorama highlights that adaptive security in IoT is being explored from multiple angles. And rather than proposing entirely new ciphers, most works focus on orchestrating existing primitives in a context-aware manner to reconcile resource constraints with application-level security requirements.

2.2.2. Design Principles and Architectural Patterns

As mentioned, the surveyed frameworks converge on a core set of design principles and architectural patterns. The first principle is separation of concerns: context sensing, decision-making, and cryptographic operations are implemented as separate components, sometimes with clear interfaces between them [28], [29], [30], [31]. This separation allows the adaptation logic to develop independently from the underlying ciphers and protocol stacks, which is essential in diverse IoT environments where cryptographic libraries and hardware accelerators can differ.

At the device and edge layers, most architectures include a context monitor that gathers local metrics such as memory usage, battery level, CPU utilization, current throughput, and, in some cases, message type or criticality. The framework proposed in [30] profiles devices to compute a capability score that aggregates CPU, memory, energy, and latency into a single scalar. In contrast, in [28], the battery, CPU, and memory are continuously monitored to inform the algorithm selection process. Similarly, [31] tracks CPU usage, memory footprint, battery level, and communication type to support its adaptive cipher and key selection. The approach proposed in [32] not only profiles device resources but also incorporates a security context of device location, connection type, and historical patterns, thereby extending the scope of context monitoring.

Above this layer, a decision or adaptation layer defines the policy that translates context into a specific configuration. Rule-based systems achieve this through decision trees or handcrafted tables. In [30], thresholds on the capability score define three power modes that directly map to the chosen ECC curve. While [28] uses a three-tier decision strategy that relies on fixed battery thresholds and message criticality to choose between AES-128 and RC4, effectively creating an energy-aware switching policy. Sinaga et al. [33] take

a similar threshold-based approach, switching between lighter and heavier configurations based on fixed system-load thresholds, and Bhandarkar et al. [31] extend this further, choosing among AES, SPECK, and SIMON based on thresholds of CPU utilization, available memory, and remaining battery power. Fuzzy-logic-based systems move away from strict thresholds by using fuzzy rules that integrate multiple imprecise measurements to generate more refined recommendations. Zeshan et al.’s fuzzy ontology-based approach [34] annotates device properties and confidentiality needs within an ontology, then applies fuzzy rules to suggest an encryption protocol balancing resource availability against confidentiality level. Building on this idea for fog-enabled IoT settings, Deb, Mukherjee and Misra [29] implement a two-layer fuzzy inference system on fog nodes, where the first layer identifies the device type and the second, considering device class, data size, and link status, chooses an encryption algorithm and key length. ML-based frameworks such as [35] or [36] implement the adaptation layer as a trained model that directly determines the algorithm choice or security level. In [35], a context-aware cryptography selection framework trains a Random Forest classifier to select an algorithm for each session, aiming to reduce latency and energy use within security constraints. [36] advances this approach by framing adaptation as a control problem: a DDPG (Deep Deterministic Policy Gradient) reinforcement learning agent selects security levels and algorithm combinations to minimize an energy–latency cost function while satisfying security requirements.

The third component, the enforcement layer, performs the actual cryptographic operations through a small set of abstractions. Often, this layer is built as a library that exposes a handful of profiles, which define algorithm choice, key size, and mode of operation. In cloud and fog environments, this layer is deployed as a service, so constrained devices only need to send context information and data, while fog nodes or cloud controllers handle key management and encryption [29], [37]. In device-centric frameworks, the enforcement logic is integrated into the firmware or software stack of the node, directly calling different selected algorithms based on the selected profile [28], [30], [31], [33].

2.2.3. Implementation Strategies, Contextual Parameters and Mechanisms

The implementation strategies used by the surveyed works are mainly influenced by the selection of contextual parameters and the complexity of the adaptation mechanisms. Four main categories of context are identified: device capabilities and energy status, data features, network conditions, and threat or risk indicators.

2.2.3.1. Device Capabilities and Energy State

Across the surveyed frameworks, device capabilities and energy availability are consistently treated as primary inputs to the adaptation logic rather than as static constraints. Most proposals monitor metrics such as CPU load, available memory, and battery state, and use them to guide algorithm selection, security level, or parameter sets, ensuring that cryptographic strength matches what each device can realistically support at a given time.

Some works illustrate this in more nuanced ways. The fuzzy ontology-based, context-aware encryption approach by Zeshan et al. [34] combines a device grader that assesses battery status and available RAM, then integrates these with other parameters, feeding them into a fuzzy inference system that then selects the suitable encryption protocol without overloading constrained nodes. Similarly, [29] uses device CPU cycles and RAM in a two-level fuzzy inference process at fog nodes to classify devices and assign encryption algorithms that match their resource profiles, thereby reducing delay and energy per transmission. In these cases, device resources and energy become key control variables in the decision-making process, which is crucial for enabling adaptive security in diverse, resource-limited IoT deployments.

2.2.3.2. Data Sensitivity, Type, and Volume

Several frameworks are explicitly data aware. The multi-cloud adaptive encryption framework in [37] classifies data into sensitivity tiers and considers data size, producing one of three configuration profiles: a High Sensitivity profile, a Large Data profile, or a Standard Requirements profile. The fuzzy and ontology-based protocol for [34] defines three confidentiality levels (high, medium, or low) and assigns them based on the semantic nature of the data: information about users is classified as high confidentiality, measurements related to the environment are treated as medium confidentiality, and common routing information is categorized as low confidentiality. These confidentiality levels are then combined with a device grader to decide when more costly cryptographic algorithms are justified, given resource constraints. Data size is included among the inputs to [29]’s second-level fuzzy inference system as well, while Rafat et al. [28] differentiate critical from non-critical messages, requiring more expensive algorithms for critical traffic. Routine data routing uses less computationally expensive algorithms.

2.2.3.3. Network Conditions and Topology

Deb et al. [29] use measured bandwidth and link quality alongside device configuration to guide encryption choices at the fog layer, recognizing that more robust schemes may be acceptable when bandwidth is ample but problematic on constrained links. The selection framework [35] incorporates latency and packet loss rate as features for its Random Forest classifier, effectively training the model to understand how different algorithms perform under various network conditions. [38] defines a context vector, where factors such as location, proximity to critical assets or nearby readers, and system load at a given time are used to calculate a risk score, which determines both key exchange and cipher strength.

2.2.3.4. Threat and Risk Indicators

A smaller subset of works explicitly models threat levels. In [38], risk level belongs to $[0,1]$ and is compared against two thresholds to determine whether to use symmetric key exchange or ECDH, and whether to upgrade from smaller to larger key lengths. [31] introduces a threat variable and defines cipher selection and key length as functions of

both resource variables and the threat variable, reflecting the intuition that resource-constrained devices should still escalate security when a high threat is detected.

2.2.4. Comparative Trade-offs Among Adaptation Mechanisms

Building upon the three adaptation families outlined in Section 2.2.1, the reviewed works reveal clear trade-offs among rule-based, fuzzy, and machine-learning-based mechanisms. Rule and threshold-based schemes remain appealing for constrained devices due to their low computational cost and easy implementation, but they rely on carefully crafted policies and may not generalize beyond the conditions they were tuned for. Fuzzy and ontology-driven approaches provide more expressive decision models, capturing complex situations where simple thresholds fall short, though they require more effort to design membership functions and rule bases. Data-driven mechanisms can learn non-linear relationships between context and performance, but they also bring challenges related to training, model management, and explainability, which are often not thoroughly addressed in current IoT prototypes.

These mechanisms also differ in how often and where adaptation is applied in the protocol flow: some frameworks adjust configurations only at initial setup or when major capability changes are detected [30], while others support more frequent, even per-message, adjustments [28], [38], [39] or per-session decisions in fog/cloud services [29], [37]. In all cases, reconfiguration is constrained to well-defined points, such as session establishment, key exchange, or message boundaries, to limit protocol instability.

2.2.5. Evaluation Metrics and Performance Trade-offs

The evaluation approaches used by the surveyed works reflect the key trade-off identified in RQ2: enhancing security while maintaining the usability of IoT devices. Most studies measure computational and resource-related metrics of their adaptive context-aware mechanisms, such as encryption and decryption times, latency, CPU and memory usage, and energy consumption, comparing these results to static cryptographic configurations to determine whether the performance and efficiency improvements justify selectively adjusting security parameters in some operations while still satisfying the overall security requirements of the application [33], [40].

For device-centric frameworks, energy and latency dominate the analysis. Purohit and Joshi’s [30] adaptive curve selection is reported to reduce CPU utilization, memory footprint, and energy cost per cryptographic operation by 78% relative to a standard ECC setup. A 23.8% reduction in average power consumption compared to static AES-128 is achieved in [28] by dynamically switching between AES-128 and RC4, alongside a 40.3% improvement in energy efficiency over static RC4, while keeping encryption latency below 3 ms throughout. Spina and De Rango’s dynamic security mechanism [39] takes a similar approach further: adapting security levels over time is shown to modestly reduce client-side energy use and boost server battery life compared to a fixed high-security setup and to DTLS.

Fog- and cloud-focused solutions emphasize system-level overhead and scalability rather than individual device costs. [29] measures CPU and memory usage on fog nodes, along with delay and energy per protected transmission, demonstrating that its two-level fuzzy inference can run without overloading fog resources. The multi-cloud adaptive encryption framework [37] compares its hybrid ECC and symmetric design against RSA-based baselines, showing better computational efficiency and scalability by selecting curves and ciphers based on data sensitivity, volume, and resource availability, rather than always defaulting to maximum security. In the post-quantum cryptography (PQC) domain, the [33] framework and the ML-based PQC selection scheme [35] both evaluate the cost of algorithms in constrained environments, illustrating that adaptive selection of parameter sets or algorithms can improve encryption time, memory usage, CPU load, and even reduce cryptographic latency and energy consumption compared to static PQC choices.

Finally, some studies go beyond resource metrics to include clearer views of security effectiveness and resilience, such as the chosen algorithms, key sizes, or attack-detection accuracy. Kushwaha et al. [32] report an attack detection accuracy of 94.7%, along with 37% lower computational overhead, 74% lower latency, and 91.2% better energy efficiency compared to traditional stacks, showing that adaptive, ML-assisted security can enhance both protection and performance. A 98.67% accuracy in traffic classification on the UNSW-NB15 intrusion detection dataset is likewise achieved by the framework in [31], accurately distinguishing normal from malicious network traffic while maintaining low encryption time, modest memory usage, and about 23% less energy per encryption compared to ECC-only baselines.

Overall, evidence from these studies indicates that context-aware adaptation can significantly improve energy efficiency and reduce latency while maintaining or even enhancing security, especially when compared to static, one-size-fits-all setups. Therefore, choosing the right metrics is more than just a reporting step: how performance and security are measured affects whether the benefits of adaptation are apparent and if trade-offs are honestly represented. This highlights the importance of establishing a consistent and comprehensive evaluation methodology as a key step in advancing the field.

2.2.6. Research Gaps and Synthesis: Positioning PRISEC IV

The review of the chosen studies highlights various research gaps that guide the development of PRISEC IV. First, most existing frameworks focus on one dominant axis of adaptation rather than integrating device resources, data sensitivity, performance requirements, and network conditions into a unified decision model. Second, many solutions are tightly coupled to specific technologies or protocol stacks, including CoAP [39], ESP-32 platforms [28], RFID infrastructures [38], or multi-cloud architectures [37], which constrains their applicability across heterogeneous IoT deployments.

Another gap, especially relevant for this dissertation, is that none of the surveyed frameworks use an explicit multi-level security structure like the one employed by the

PRISEC framework. While existing proposals may distinguish between devices, data sensitivity levels, or traffic types, they do so without establishing persistent access levels that connect authentication, authorization, and cryptographic strength across the entire system. PRISEC IV is designed specifically to address this gap by reinterpreting these access levels as high-level security profiles that serve as guidelines rather than rigid constraints tied to specific algorithms. Building on the multi-level structure (Guest, Basic, Advanced, Admin), PRISEC IV maintains these levels as abstract policy anchors but separates them from fixed cryptographic suites; instead, the context-aware adaptation layer adjusts each level’s baseline requirements using runtime information to dynamically select appropriate symmetric, asymmetric, and lightweight configurations.

Table 2.2 summarizes the contextual dimensions considered by the frameworks selected for Research Question 2, alongside the model proposed in this dissertation.

The purpose of this comparison is not to rank the reviewed frameworks or to imply that a missing dimension is necessarily a weakness in a specific deployment. Each proposal addresses a particular problem, platform, or operational setting, and therefore selects context inputs that are appropriate to its own scope. Instead, the table documents how contextual information is distributed across the reviewed literature. It also positions PRISEC IV: the proposed model combines device and energy constraints with data-security requirements while retaining the Guest, Basic, Advanced, and Admin multi-level structure inherited from PRISEC III. Network conditions and explicit threat or risk indicators are deliberately outside the current prototype’s scope and are identified as directions for future work.

TABLE 2.2. Contextual dimensions considered by selected adaptive IoT security frameworks and PRISEC IV.

Study	Device / Energy	Data Sensitivity	Network Conditions	Threat / Risk	Application	Multi-Level Structure
Rafat et al. [28]	✓	✓	–	–	–	–
Deb, Mukherjee & Misra [29]	✓	✓	✓	–	–	–
Purohit & Joshi [30]	✓	–	–	–	–	–
Bhandarkar et al. [31]	✓	–	–	✓	–	–
Kushwaha et al. [32]	✓	✓	✓	✓	–	–
Sinaga et al. [33]	✓	–	–	–	–	–
Zeshan et al. [34]	✓	✓	–	–	–	–
Khara et al. [35]	✓	–	✓	–	✓	–
Alanazi et al. [36]	✓	–	–	–	–	–
Multi-cloud framework [37]	✓	✓	–	–	–	–
Amoah et al. [38]	–	–	✓	✓	–	–
Spina & De Rango [39]	✓	–	–	–	–	–
This dissertation (PRISEC IV)	✓	✓	–	–	✓	✓

CHAPTER 3

Research Methodology

As articulated in Section 1.4, this dissertation adopts the Design Science Research (DSR) methodology, given its suitability for investigating real-world issues through the design, development, and evaluation of innovative artifacts, moving beyond descriptive analysis of IoT security challenges to address the shortcomings of existing static approaches in resource-constrained environments.

Moreover, to ensure that the artifact’s design is based on solid scientific evidence, the research is informed by a Systematic Literature Review (SLR). The SLR provides the theoretical and analytical foundation for identifying limitations in the literature, examining current cryptographic approaches, and determining the contextual factors that should guide the proposed solution.

This chapter outlines the adopted methodological framework, presenting the overall structure of the study and describing the phases of the Design Science Research process in relation to the research objectives.

3.1. Design Science Research Methodology

This dissertation follows the Design Science Research Methodology proposed by [41], which provides a structured process for developing and evaluating research artifacts intended to address identified problems. Figure 3.1 presents the Design Science Methodology process model, which structures DSR into six key activities: problem identification and motivation, defining solution objectives, design and development, demonstration, evaluation, and communication. As illustrated in the figure, these activities are organized in a nominal sequence, with each step building upon the previous, although the methodology also allows iteration between stages and recognizes multiple possible entry points into the research process. In the present dissertation, the study follows primarily a problem-centered initiation, since the research originates from the identified limitations of static cryptographic configurations in resource-constrained and heterogeneous IoT environments.

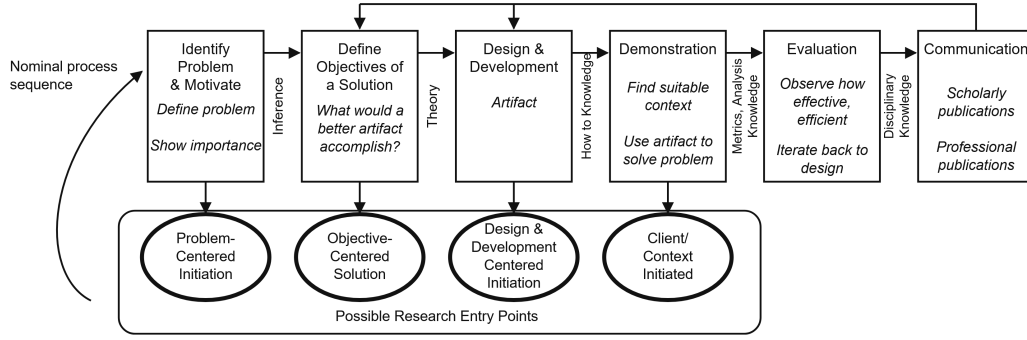


FIGURE 3.1. Design Science Research methodology process model.

Building on this process model, the six activities are detailed below in relation to the specific objectives of this dissertation.

Problem Identification and Motivation: This dissertation addresses the inadequacy of static cryptographic configurations in computationally constrained IoT environments, motivating adaptive security mechanisms that balance confidentiality and availability without unnecessary overhead.

Define the Objectives of a Solution: The main objective is to develop a context-aware security framework for IoT environments that dynamically adapts cryptographic methods to device characteristics, data sensitivity, and performance requirements. Specifically, the goals are to identify suitable cryptographic algorithms, develop a context-aware selection model, and extend the PRISEC framework with adaptive configurations.

Design and Development: The artifact is a context-aware security tool that extends the PRISEC framework, enabling dynamic selection and configuration of cryptographic algorithms. The design phase specifies the contextual parameters, adaptation logic, and decision criteria for cryptographic selection in heterogeneous IoT environments, later implemented as a functional prototype.

Demonstration: Demonstration is conducted through deployment of the prototype across representative IoT scenarios, establishing the framework’s practical applicability and its capacity for adaptive cryptographic decision-making under diverse contextual conditions.

Evaluation: This dissertation evaluates the proposed solution through experimental analysis, assessing whether the selected cryptographic method suits the specific IoT context and whether the artifact can adapt its configuration as contextual conditions change.

Communication: The results are systematically documented within this dissertation, through a rigorous presentation of the research problem, theoretical framework, artifact development process, and evaluation results, making the dissertation itself the primary vehicle for articulating the artifact’s scientific and practical contribution.

3.2. Systematic Literature Review

This dissertation is supported by a Systematic Literature Review (SLR) conducted in accordance with the guidelines proposed by Kitchenham’s and Charters’ for software engineering research [42], which represents a revised version of Kitchenham’s 2004 work, “Procedures for Performing Systematic Reviews” [43].

Figure 3.2 presents the three phases that structure the SLR process: planning the review, conducting the review, and reporting the review. Each phase groups a set of activities that guide the systematic identification, selection, and synthesis of relevant literature.

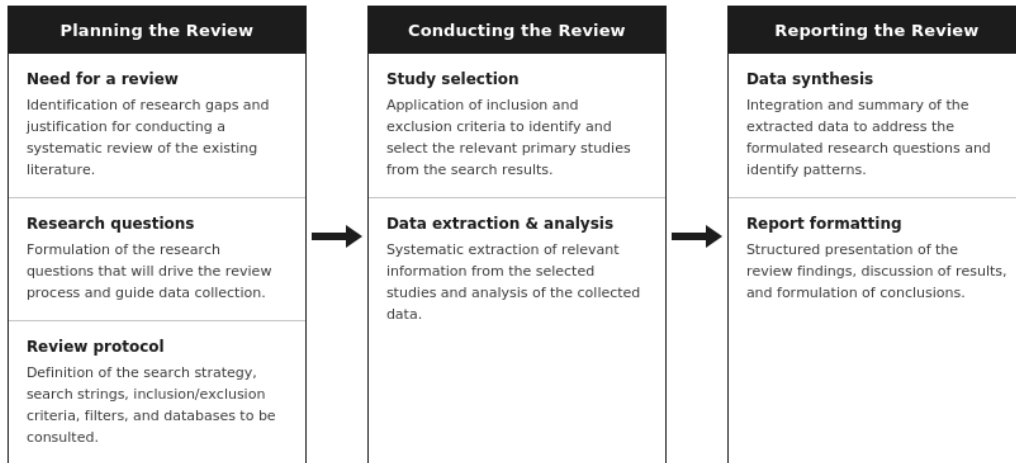


FIGURE 3.2. Phases of the Systematic Literature Review process.

3.2.1. Planning the Review

Following the proposed structure, the review process was organized into three main phases: planning the review, conducting the review, and reporting the review.

The planning phase includes identifying the need for the review, specifying the research questions, and developing the review protocol.

3.2.1.1. Need for the Review

In this dissertation, the need for the review emerged from the necessity to obtain a clearer understanding of which cryptographic algorithms are most suitable for use in resource-constrained IoT environments, as well as from the still-limited understanding of how context-aware security frameworks are designed and implemented in practice. Despite the presence of several preliminary studies on IoT cryptography and adaptive security mechanisms, a comprehensive and systematic synthesis that concurrently addresses both dimensions within resource-constrained environments has yet to be established. Conducting a systematic review is therefore essential to identify existing approaches, evaluate their suitability, and highlight key research gaps that inform the design of the proposed context-aware security framework.

3.2.1.2. Research Questions

The review’s objective is to answer the following research questions that were presented in Section 1.4, Research Objectives:

- **RQ1:** Which cryptographic algorithms are used in IoT systems to handle resource limitations while ensuring sufficient security?
- **RQ2:** How can IoT security mechanisms be designed to dynamically adapt cryptographic configurations based on contextual information to balance security and efficient resource use in resource-constrained environments?

3.2.1.3. The Review Protocol

The review protocol was established before starting the search process to ensure a structured, transparent, and reproducible methodology. It establishes the search strategy, the academic databases consulted, and the inclusion and exclusion criteria used during study selection.

The literature search was conducted across two major academic databases: IEEE Xplore and Scopus. These databases were selected for their extensive and complementary coverage of computer science and engineering publications, as well as their relevance to research on IoT security and cryptographic mechanisms.

The search strings were constructed from the key concepts derived from the research questions, namely IoT environments, cryptographic algorithms, lightweight cryptography, and context-aware adaptive security. The following primary search strings were applied:

- For RQ1: (“Internet of Things” OR “IoT”) AND (“cryptography” OR “cryptography algorithm” OR “encryption” OR “encryption algorithm” OR “cipher”) AND (“security” OR “secure communication” OR “confidentiality” OR “data protection”)
- For RQ2: (“IoT” OR “Internet of Things”) AND (“context-aware” OR “adaptive” OR “resource-aware”) AND (“encryption” OR “cryptography”)

The search strings presented were then adapted to the syntax requirements of each database prior to execution, thereby ensuring compatibility with the respective query languages while preserving the intended search logic, and were then applied to each database. For RQ1, after the first execution, an additional clause was added to the string to limit the results to review-oriented publications: AND (“review” OR “survey” OR “systematic review”). This addition aimed to prioritize studies that offer a broader comparison of cryptographic algorithms used in constrained IoT environments, providing a more comprehensive basis for answering RQ1.

A set of inclusion and exclusion criteria was systematically established and consistently applied during the study selection process, as detailed in Table 3.1.

3.2.2. Conducting the Review

The conducting phase implements the review protocol established during the planning stage. It involves the systematic execution of the search strategy, the orderly application

TABLE 3.1. Inclusion and exclusion criteria

Criteria	Definition
Inclusion	
IC1	The study title or abstract must contain terms from the search string
IC2	Published between 2024 and 2026
IC3	Articles written in English
IC4	Studies published in peer-reviewed journals or conferences
Exclusion	
EC1	Duplicate articles
EC2	Articles that do not address IoT environments
EC3 (RQ1)	Manual filter (articles that do not provide a comparison of cryptographic algorithms)
EC3 (RQ2)	Manual filter (articles that do not address IoT security mechanisms designed to dynamically adapt cryptographic configurations)

of the selection criteria, and the extraction and analysis of data from the studies that are included. Each of these steps was performed in a structured and traceable way, ensuring that the final collection of selected literature is both relevant and methodologically sound.

3.2.2.1. Study Selection

The study selection process began with the execution of the search strings across the selected databases. The initial search produced a large and diverse set of results, which was expected considering the wide range of query terms. This initial pool served as the starting point for a sequential filtering process, in which the inclusion and exclusion criteria were applied gradually to narrow down the corpus to the most relevant studies.

Before the initial filter, the additional clause mentioned in the Review Protocol was added to RQ1’s search string, and then IC1 was applied. This required that each record’s title or abstract contain terms from the search string. Next, a publication date filter based on IC2 was implemented, where studies published between 2024 and 2026 were included. IC3 was then used to restrict the corpus to studies written in English. Following this, IC4 was applied to retain only journal articles and conference papers.

After the inclusion criteria were established, the exclusion criteria were enforced. EC1 involved removing duplicate records. EC2 followed, eliminating studies that, despite passing previous filters, did not focus on IoT environments in a meaningful way. Finally, a manual screening was conducted. For RQ1, EC3 excluded studies lacking a comparative analysis of cryptographic algorithms. For RQ2, EC3 excluded studies that did not address the dynamic adaptation of cryptographic configurations within IoT security mechanisms.

Following the sequential application of all criteria, a total of 28 studies were selected for inclusion in the review, 15 for RQ1 and 13 for RQ2. The distribution of results across databases and research questions is shown in Table 3.2.

TABLE 3.2. Stages of the selection process.

Databases	Search Strings	Initial	RQ1 Changes	IC1	IC2	IC3	IC4	EC1	EC2	EC3
IEEE Xplore	RQ1	15088	948	345	147	146	142	135	119	9
	RQ2	969	–	352	263	261	250	183	97	8
Scopus	RQ1	18197	1109	550	399	390	85	75	68	6
	RQ2	1174	–	903	626	622	280	198	76	5
Total		35 428	4 200	2 150	1 422	1 419	757	591	360	28

3.2.2.2. Data Extraction and Analysis

After completing the study selection process, data extraction was performed for each included study. Each study was read thoroughly, and relevant information was recorded. The extracted data was then analyzed and organized thematically around the two research questions. For RQ1, the analysis focused on characterizing the most evaluated cryptographic algorithms and their reported trade-offs in resource-limited IoT environments. For RQ2, the analysis explored how dynamic and context-aware security frameworks have been conceptualized and implemented in the literature.

3.2.3. Reporting the Review

The reporting phase is the final stage of the systematic literature review, covering the synthesis of the collected data and the clear presentation of the findings. Its goal is to combine the evidence gathered during the conducting phase into a unified and understandable account that directly answers the formulated research questions.

3.2.3.1. Data Synthesis

The findings from the data extraction and analysis were combined and summarized for each research question. For RQ1, the synthesis uses evidence from selected studies to identify the most widely evaluated cryptographic algorithms for IoT environments, highlighting their strengths, limitations, and suitability for resource-constrained deployments. For RQ2, the synthesis reviews the approaches proposed in the literature for dynamically adapting cryptographic configurations, focusing on common design principles, contextual factors, and implementation strategies. When patterns or differing viewpoints emerge across studies, they are highlighted and discussed to offer a clear, well-rounded view of the current state of the art.

3.2.3.2. Report Formatting

The results of the systematic literature review are presented in Chapter 2 of this dissertation.

CHAPTER 4

Cryptographic Evaluation

The cryptographic algorithms selected for the PRISEC IV framework, summarized in Figure 4.1 and introduced in Section 2.1.3 on the basis of their security strength, computational efficiency, memory footprint, and prevalence in the reviewed literature, are described in mathematical detail in this section. PRISEC IV relies on a small set of algorithms consisting of AES, ChaCha20, SPECK, RECTANGLE, and HIGHT for symmetric encryption, and ECC for key establishment. These algorithms were deliberately chosen to span both conventional and lightweight design philosophies. The remainder of this chapter presents the cryptographic evaluation of each algorithm.

TABLE 4.1. Overview of the cryptographic algorithms selected for PRISEC IV.

Algorithm	Type	Key Size	Block Size	Number of Rounds
AES	Symmetric Block Cipher	128/192/256 bits	128 bits	10/12/14
ChaCha20	Symmetric Stream Cipher	256 bits	N/A ^a	20
ECC	Asymmetric	256 bits (most common)	N/A	N/A
SPECK	Symmetric Block Cipher	128 bits	128 bits	32
RECTANGLE	Symmetric Block Cipher	128 bits	64 bits	25
HIGHT	Symmetric Block Cipher	128 bits	64 bits	32

^a As a stream cipher, ChaCha20 has no fixed block size; internally, it processes data in 64-byte (512-bit) blocks.

4.1. Cryptographic Evaluation

In this section, the previously described algorithms undergo empirical evaluation and comparison. These algorithms were implemented and benchmarked, both individually and combined into cryptographic cascades, to quantify their performance characteristics under realistic operating conditions, rather than relying solely on theoretical complexity or published figures from unrelated hardware. Section 4.3.1 describes the methodology underlying this evaluation, including its scope, the metrics collected, and the testing procedure followed, before the results themselves are presented and discussed.

4.1.1. Evaluation Methodology

4.1.1.1. Evaluation Scope

The evaluation covers the five symmetric algorithms listed in Table 4.1 (AES, ChaCha20, SPECK, HIGHT, and RECTANGLE), with AES available in three key sizes (128, 192, 256), along with eight cascades combining two or three of these algorithms: seven two-layer cascades (AES-256+AES-128, AES-128+HIGHT, AES-128+SPECK, AES-256+ChaCha20, ChaCha20+SPECK, SPECK+HIGHT, RECTANGLE+HIGHT) and one three-layer cascade (AES-256+ChaCha20+AES-128). ECC is evaluated separately, as its role in this

dissertation is confined to key establishment rather than bulk encryption. For a subset of these algorithms and cascades, ECC is used beforehand to perform a key exchange, and the resulting shared secret supplies the key material subsequently used by the symmetric cipher(s) to encrypt the data.

The evaluation is organized into five phases:

- **Phase 1** - performance of the seven single algorithms
- **Phase 2** - performance of the seven two-layer cascades
- **Phase 3** - performance of the three-layer cascade
- **Phase 4** - the effect of disabling hardware acceleration (AES-NI and/or ChaCha20's AVX-512, RECTANGLE's AVX2) on the subset of single algorithms and cascades where these ciphers are present
- **Phase 5** - the setup cost of every algorithm and cascade, together with the ECC key-exchange cost combined with each algorithm/cascade setup cost

Phases 1 through 3 measure every algorithm and cascade with hardware acceleration enabled by default. Phase 4 isolates the effect of hardware acceleration by disabling it for AES, ChaCha20, and RECTANGLE on the subset of single algorithms and cascades where these ciphers are present. SPECK and HIGHT are always benchmarked using only software implementations, unlike AES, ChaCha20, and RECTANGLE. Both ciphers were deliberately designed for minimal, register-constrained microcontrollers where no vector unit is assumed to exist, relying only on the simplest scalar addition, XOR, and rotation instructions present on virtually any processor. Therefore, no hardware-accelerated implementation of either cipher is used in this benchmark suite, since doing so would contradict the design intent of both ciphers and the role they occupy in PRISEC IV as its always-software, lowest-footprint options. This contrasts with RECTANGLE, which is also a lightweight cipher but benefits from bit-slicing techniques and SIMD (Single Instruction, Multiple Data) instructions to parallelize this technique across multiple data lanes.

To capture how each algorithm and cascade behaves across payloads of markedly different scale, a common set of nine data sizes is used across all benchmarks and evaluations, spanning three orders of magnitude from 1KB to 50MB: 1KB, 5KB, 10KB, 50KB, 100KB, 1MB, 5MB, 10MB, and 50MB.

In addition to these five phases, energy consumption was measured separately using a companion Android application because this metric cannot be captured by a desktop benchmark suite. This mobile evaluation is organized into three scripts that run across the same nine data sizes:

- **Script 1** - energy cost of the seven single algorithms and eight cascades, with hardware acceleration enabled where applicable, mirroring Phases 1 through 3
- **Script 2** - energy cost of the same algorithms and cascades with hardware acceleration disabled, mirroring Phase 4

- **Script 3** - energy cost of a single ECC key-exchange handshake that derives 1, 2, or 3 keys, measured in isolation and then combined with the energy cost of a plain algorithm or cascade, mirroring Phase 5's setup-cost reconstruction for ECC-prefixed combinations

4.1.1.2. Evaluation Metrics

Each phase of the benchmarking suite quantifies a subset of the subsequent metrics, which are subsequently integrated to develop a comprehensive performance profile for each individual cipher and cascade configuration:

Encryption/decryption time (ms): the duration required to encrypt or decrypt a specified payload, serving as the most direct indicator of an algorithm's computational burden.

Throughput (Mbps): expressed as a data transfer rate, facilitating comparison of the relative efficiency across various payload sizes.

Latency (μ s): the interval for processing a single data block. Given that ChaCha20 operates as a stream cipher lacking a fixed block size, its internal 64-byte keystream block is employed as the equivalent measurement unit. This standardization enables latency reporting on a per-block basis across different cipher types, including those without traditional block structures.

Memory usage (KB): recorded as peak resident memory (`memory_peak`) during the execution of encryption or decryption. This represents the maximum live memory footprint. Memory overhead is calculated as `overhead = memory_peak - data_size`, where `data_size` signifies the size of the original plaintext, subtracted once to isolate the memory cost attributable solely to cryptographic processing. For cascades, this single subtraction is deliberate: only the first buffer required to hold the input data is treated as baseline, while every subsequent buffer, including intermediate cascade outputs and each additional layer's working memory, is attributed to overhead, since it exists specifically because more than one cipher is applied in sequence.

Setup cost (μ s): the time required to initialize a cipher or cascade prior to commencing encryption or decryption. For symmetric ciphers, this corresponds to key scheduling and cipher-context initialization, whereas for elliptic-curve cryptography (ECC), it pertains to the key-exchange process (ECDH handshake) required to establish a shared secret.

Energy consumption (mAh): the battery cost of encrypting a single data packet with a given algorithm or cascade of algorithms.

These metrics collectively facilitate a detailed evaluation of cryptographic performance, supporting informed comparisons across diverse configurations.

4.1.1.3. Testing Environment

The desktop C benchmark suite (Phases 1 through 5) was executed on an AWS EC2 t3.micro instance, running Ubuntu 26.04 LTS under a KVM hypervisor, with the following specifications:

- **CPU:** Intel® Xeon® Platinum 8259CL @ 2.50 GHz (Cascade Lake), exposed as 2 vCPUs. All encryption and decryption operations were run single-threaded and pinned to a single CPU core, even when the second vCPU was available.
- **Memory:** 1GB RAM

The energy-consumption trials were run on a physical Samsung Galaxy A53 5G smartphone, with the following specifications:

- **Chipset:** Exynos 1280 (5nm), octa-core - 2× ARM Cortex-A78 @ 2.4 GHz + 6× ARM Cortex-A55 @ 2.0 GHz
- **Memory:** 6GB RAM
- **Battery:** 5,000mAh
- **Operating system:** Android 16

It is important to note that the performance outcomes depend on these particular hardware configurations. Switching to different hardware could result in either improved or reduced performance.

4.1.1.4. Testing Procedure

AES, ChaCha20, and ECC implementations utilize OpenSSL’s EVP API, ensuring that the obtained results reflect a realistic, optimized deployment rather than an idealized reference model. On the other hand, SPECK, HIGHT, and RECTANGLE are manually implemented in C, following their respective published specifications. The energy consumption experiments combine Kotlin for orchestrating trials with C++, compiled via Android NDK, to perform cryptographic operations, thus ensuring both environments execute identical cipher code instead of two independently developed implementations.

The desktop benchmark suite (comprising Phases 1 through 5) is fully implemented in C. This choice leverages C’s ability to compile directly into native machine code, thereby enabling instruction execution with minimal overhead. This property is particularly crucial for SPECK, HIGHT, and RECTANGLE, as each encryption round involves numerous bitwise and arithmetic operations executed within tight loops over thousands of iterations. Languages such as Python, being interpreted, would incur significant per-call overhead, undermining measurement accuracy. Even with OpenSSL-backed ciphers (AES, ChaCha20, ECC), where a Python wrapper would invoke the underlying C library, direct C implementation circumvents foreign function interface calls and interpreter dispatch, thereby avoiding microsecond-scale timing distortions pivotal to this analysis. In essence, C is the only language capable of precisely capturing the authentic computational costs of these algorithms, independent of runtime overheads.

Each (algorithm/cascade, data size) pairing is evaluated within an isolated child process created via `fork()`, preventing interference from previous processes’ memory states. Within each process, measurements follow a two-tiered repetition scheme: an inner loop of encryption/decryption operations (with iteration counts decreasing as data size increases, from 10,000 at 1KB to a single iteration at 50MB), and an outer loop of repeated trials (ranging from 300 at the smallest data sizes to 50 at the largest). The reported metric for each configuration is the median of the outer loop means, reducing the influence of outliers while accurately capturing genuine variability across data sizes.

Timing measurements utilize `CLOCK_MONOTONIC`, which is resistant to system clock adjustments during execution. Memory usage is monitored via a wrapper around the standard memory allocation functions (`malloc`, `free`, `realloc`, `calloc`) that tracks peak memory usage throughout the benchmarking process.

Energy consumption assessments on an Android device employ iteration schemes comparable to those used in the desktop setup. To ensure measurement accuracy, the device remains on battery power with the screen off, airplane mode enabled, and background applications suppressed, minimizing extraneous power draw. Energy metrics are obtained directly from Android’s `dumpsys batterystats` utility, which provides per-application power usage estimates.

4.1.2. Single-Cipher Performance

Appendix A.1 and A.2 present the encryption/decryption time and throughput of all seven single algorithms across the nine evaluated data sizes. ChaCha20 achieves the highest throughput of any algorithm at every size beyond the smallest, reaching approximately 15,700 Mbps at steady state. Among the AES variants, throughput decreases as key size increases (AES-128 > AES-192 > AES-256), reflecting the larger number of rounds required for larger key sizes. SPECK, RECTANGLE, and HIGHT trail well behind AES and ChaCha20, consistent with their role as lightweight ciphers designed for constrained hardware rather than raw throughput. Notably, SPECK outperforms RECTANGLE and HIGHT at every size in this phase ($\approx 2,900$ Mbps versus ≈ 800 and ≈ 430 Mbps). Despite RECTANGLE’s bit-sliced design being built specifically to parallelize across many blocks at once, SPECK’s simpler two-word ARX structure evidently carries less overhead per byte.

Latency shows a similar pattern across most algorithms, decreasing sharply at the smallest sizes before leveling off. SPECK and HIGHT are the exception: both show nearly flat per-block latency from the smallest size onward ($\approx 0.044 \mu\text{s}/\text{block}$ and $\approx 0.148 \mu\text{s}/\text{block}$, respectively, barely changing between 1KB and 50MB). This indicates that, unlike the other five algorithms, SPECK and HIGHT have very little fixed per-call cost to begin with, and their latency is dominated almost entirely by genuine per-block computation from the very first block processed, rather than by a setup cost that only amortizes away at larger sizes. AES variants range from about $0.045 \mu\text{s}/\text{block}$ down to 0.0139 – $0.0190 \mu\text{s}/\text{block}$, each following the expected pattern of a fixed setup cost amortizing as payload

size grows. ChaCha20 shows the same pattern but at a larger scale, from $0.1798 \mu\text{s}/\text{block}$ at 1KB down to $0.0332 \mu\text{s}/\text{block}$ at 50MB. RECTANGLE follows a similar curve, from 0.1581 down to $0.0797 \mu\text{s}/\text{block}$.

Memory overhead remains constant across all nine data sizes for each algorithm, indicating that this cost is a fixed property of each implementation rather than scaling with payload. AES consistently incurs 1.89KB overhead, ChaCha20 2.31KB, and SPECK 0.47KB. HIGHT has the lowest overhead at 0.38KB, closely followed by SPECK, both designed for memory-constrained devices. ChaCha20 has the highest fixed overhead among conventional algorithms. RECTANGLE is the exception, with 1.65KB overhead up to 1KB payloads, then jumping to 5.31KB for larger sizes, due to its AVX2 bit-sliced implementation that processes data in fixed 256-block batches. Payloads below this size use a smaller scalar path, while larger payloads allocate a full batch-sized workspace, a fixed cost that does not increase with additional payload.

4.1.3. Cascade Performance

Appendices B.1 and B.2 present the encryption/decryption time and throughput for all eight cascades across the nine evaluated data sizes. At steady state, AES-256+ChaCha20 is the fastest cascade ($\approx 4,730$ Mbps), followed by AES-256+AES-128 ($\approx 3,900$ Mbps), then the two cascades pairing a conventional cipher with SPECK, ChaCha20+SPECK and AES-128+SPECK ($\approx 2,420$ and $\approx 2,170$ Mbps, respectively). The three cascades involving HIGHT or RECTANGLE trail well behind (280–410 Mbps), consistent with these being the two slowest single algorithms in Phase 1. The three-layer cascade, AES-256+ChaCha20+AES-128, reaches $\approx 3,130$ Mbps at steady state, slower than either of its two constituent two-layer pairings (AES-256+ChaCha20 or AES-256+AES-128) taken alone, as expected from the additional sequential stage, but still faster than any cascade involving SPECK, RECTANGLE, or HIGHT.

Latency follows the same overall pattern established in Section 4.3.2. Cascades involving only conventional algorithms show latency decreasing sharply from the smallest sizes before leveling off: AES-256+AES-128 and AES-256+ChaCha20 both follow this curve, with per-block cost falling substantially between 1KB and the larger sizes before flattening out at steady state. Cascades involving SPECK or HIGHT, by contrast, remain comparatively flat across the full-size range, since neither cipher’s stage has meaningful setup cost left to amortize; this holds whether SPECK or HIGHT is paired with a conventional algorithm (AES-128+SPECK, ChaCha20+SPECK) or with each other (SPECK+HIGHT, RECTANGLE+HIGHT). The three-layer cascade shows the same amortizing pattern as its two-layer, all-conventional counterparts, with latency falling from its highest value at 1KB down to a steady-state minimum by the largest sizes tested.

Memory overhead departs sharply from the pattern observed in individual algorithms. This divergence can be directly attributed to the fundamental distinction underlying the memory-overhead methodology outlined in Section 4.3.1.2: the retention of the plaintext is unavoidable, regardless of subsequent processing; at least one buffer of the same size as

the input must be maintained. The decision to apply additional encryption layers is discretionary rather than mandatory. Each supplementary buffer introduced by this choice thereby constitutes additional overhead. Specifically, a cascade of multiple encryption stages results in precisely one such buffer per additional stage: the intermediate ciphertext generated by one layer, which must be stored in memory during processing by the subsequent layer. While Phase 1 demonstrated that the overhead of single algorithms remains a small, size-independent constant, since a single cipher need not retain data beyond its operational memory, the overhead associated with cascades scales nearly linearly with payload size. It increases by approximately one data-size unit for each additional layer beyond the first. For example, a two-layer cascade exhibits overhead roughly equal to one full data-size unit plus a minimal residual (e.g., AES-256+ChaCha20 at 50MB payload: 51,202KB overhead against a 51,200KB payload, corresponding to a residual of approximately 2.35KB). Similarly, a three-layer cascade incurs overhead approximately equal to two data-size units at the same payload size (102,402KB overhead against 51,200KB payload), corresponding to one intermediate buffer for each of the two additional stages.

4.1.4. Impact of Hardware Acceleration

Appendix C presents, for each cipher or cascade with at least one component that has an accelerated path, the baseline throughput with all relevant acceleration disabled, along with the throughput gain from enabling it.

AES-NI’s gain is, for all practical purposes, identical across all three key sizes. AES-128 reaches +1,605% at steady state, AES-192 +1,615%, and AES-256 +1,626%, a spread of barely 1% despite AES-256 performing 40% more rounds than AES-128. This is the expected signature of **AESENC**: a single instruction that executes one complete AES round at fixed, constant latency, so accelerating each round by the same factor leaves the overall speedup essentially unaffected by the number of rounds performed.

ChaCha20’s SIMD gain (+337% at steady state) reflects a different, tiered mechanism: OpenSSL selects the widest SIMD instruction set the underlying CPU supports from among several vectorized ChaCha20 implementations. Because the AWS EC2 instance used throughout this evaluation (Section 4.3.1.3) supports AVX-512, the accelerated ChaCha20 measurements reported in Phases 1 through 3 already reflect OpenSSL’s widest and fastest available tier; on hardware without AVX-512 support, the realized gain from enabling ChaCha20’s SIMD path would likely be smaller, since OpenSSL would fall back to a narrower tier such as AVX2 or SSSE3.

RECTANGLE’s gain (+78%) is the smallest among all evaluated algorithms. RECTANGLE’s non-accelerated implementation is a bit-sliced design that packs 64 blocks into a single 64-bit machine word and processes them all with a single bitwise instruction. The AVX2 implementation applies the same bit-slicing technique at 256 bits rather than 64, a fourfold widening of the same underlying approach rather than a transition from a serial algorithm to a parallel one. RECTANGLE’s non-accelerated implementation has a memory overhead of only $\approx 1.78\text{KB}$, compared with the $\approx 5.31\text{KB}$ overhead of

its AVX2 implementation reported in Section 4.3.2. In other words, it trades a comparatively modest throughput gain for a substantially larger memory footprint, while AES and ChaCha20 deliver far larger gains at essentially no memory cost at all.

The same principle of how much a given path helps its specific algorithm, and how large a share of total time that algorithm represents, carries over directly to cascades. AES-256+AES-128, the only cascade where both layers are AES-NI-eligible, shows the largest gain of any configuration at steady state (+1,624%), followed closely by the three-layer cascade AES-256+ChaCha20+AES-128 (+1,377%), which combines two AES-NI-eligible stages with one ChaCha20 stage. The two-layer AES-256+ChaCha20 pairing (+1,252%) sits below both, consistent with it combining one very large per-round gain (AES-NI alone: +561%) with one considerably smaller one (ChaCha20-SIMD alone: +124%), rather than two large ones.

4.1.5. Setup and Key-Exchange Cost

In the fifth phase, the fixed cost of preparing each cipher, cascade, and ECC handshake for use, before any encryption takes place, is extremely small for every symmetric algorithm and cascade evaluated, several orders of magnitude below even the encryption time of the smallest payload. The ECC handshake stands in stark contrast: deriving a single key takes 112.99 μ s, over 60 times the setup cost of the most expensive symmetric cascade. Deriving additional keys from that same handshake via HKDF-Expand is comparatively cheap, confirming that the ECDH exchange itself, not the key derivation that follows it, is responsible for essentially all of ECC’s setup cost.

Appendix D presents the ECC handshake as a share of total time (handshake plus encryption) across increasing payload sizes for each algorithm. For AES-128, the fastest algorithm evaluated, the handshake accounts for 97.6% of total time at 1KB, falling to 55.5% by 100KB, 11.0% by 1MB, and just 0.2% by 50MB. For RECTANGLE and HIGHT, both considerably slower per byte, the same decline occurs at even smaller sizes: RECTANGLE’s ECC share falls below 50% by 10KB and below 1% by 1MB, while HIGHT’s falls below 40% by 10KB and below 1% by 100KB. AES-256+ChaCha20+AES-128 follows the same pattern. Despite requiring a larger, three-key handshake, its ECC share drops from 92.4% at 1KB to 0.1% at 50MB. In every case, the same underlying reason applies: the handshake’s cost is fixed regardless of payload size, while encryption time grows with it. As the size of what is being protected grows, the handshake’s relative weight falls continuously, and it falls faster for algorithms and cascades that were already slower to begin with, since their own encryption time overtakes the fixed handshake cost sooner.

Payload size is only one axis along which this cost is amortized. Because ECC is used here for key establishment, a single handshake’s derived key or keys may be reused across many subsequent encryption operations. The number of messages encrypted under a given handshake therefore matters as much as the size of any one of them: a handshake that looks expensive against a single 1KB packet becomes comparatively irrelevant once

reused across dozens or hundreds of such packets, exactly as it does when amortized across a single, larger one.

This cost, in either case, is not simply overhead to be minimized away; it is a property none of the symmetric ciphers can provide on their own: a secure key exchange conducted over a channel that may itself be insecure, without any key ever needing to be shared out-of-band beforehand. The results above show that this guarantee, once paid for, becomes progressively cheaper to hold onto the larger or more frequent the encryption that follows it, rather than a fixed tax that must be re-justified every time.

CHAPTER 5

Prototype Design and Implementation

Building on the cryptographic algorithms and cascades evaluated in Chapter 4, this chapter turns to the software system built around them: the PRISEC IV prototype. While Chapter 4 established what each algorithm is and how it performs in isolation, this chapter describes how those results are put to use. The prototype is a web application that takes a device’s characteristics and an application’s operational requirements and recommends the cipher or cascade best suited to them. The prototype functions as a controlled environment that mimics how the decision model would behave if embedded within a real IoT or edge device, allowing its recommendations to be tested, inspected, and validated against a wide range of hypothetical and real device profiles before any such deployment. The chapter begins with the prototype’s UML diagrams, then describes its architecture, decision model, and backend and frontend implementation.

5.1. System Architecture

This section describes the technologies underlying the system’s components and the system architecture. First, it presents the technology stack, framing each choice as a decision made in response to a specific requirement of the prototype, and then uses that same stack to describe how the resulting components are arranged and connected.

5.1.1. Technology Stack

For the frontend, a JavaScript framework was needed to manage the prototype’s multi-step interactions as distinct, stateful views rather than a single static page. React was chosen via the Next.js framework for its component-based structure, with styling handled through Tailwind CSS.

For the backend implementation, a framework capable of exposing the decision model via a web-accessible interface with minimal boilerplate was required. FastAPI was selected for this purpose, operating on Uvicorn as the ASGI server, due to its native integration with Pydantic. This integration allows automatic validation of incoming requests against a predefined schema, obviating the need for manual validation at each endpoint. Persistence requirements were addressed through the utilization of SQLAlchemy as an Object-Relational Mapper (ORM), with PyMySQL serving as the database driver to connect to a MySQL database. This configuration was chosen to define Profile and Decision records as plain Python classes, avoiding the need to write SQL queries manually.

The frontend and backend communicate over HTTP, exchanging JSON payloads through RESTful endpoints exposed by the backend.

Figure 5.1 presents the resulting architecture, using the components established in Section 5.1.1. The Frontend and Backend communicate over HTTP in both directions, as described above. Within the Backend, the API layer and the Decision Model are connected but distinct: this connection is not a network call of any kind but a direct function call within the same Python process, in both directions. The API layer supplies a profile and context, and the Decision Model returns a score for each candidate cipher. This distinction is deliberate, since the Decision Model has no independent existence as a service, and nothing is serialized, transmitted, or deserialized between it and the API layer that calls it. By contrast, the API layer’s connection to the Database is a genuine read/write boundary, with profiles and decisions being written to persistent storage when created and read back whenever a request needs them.

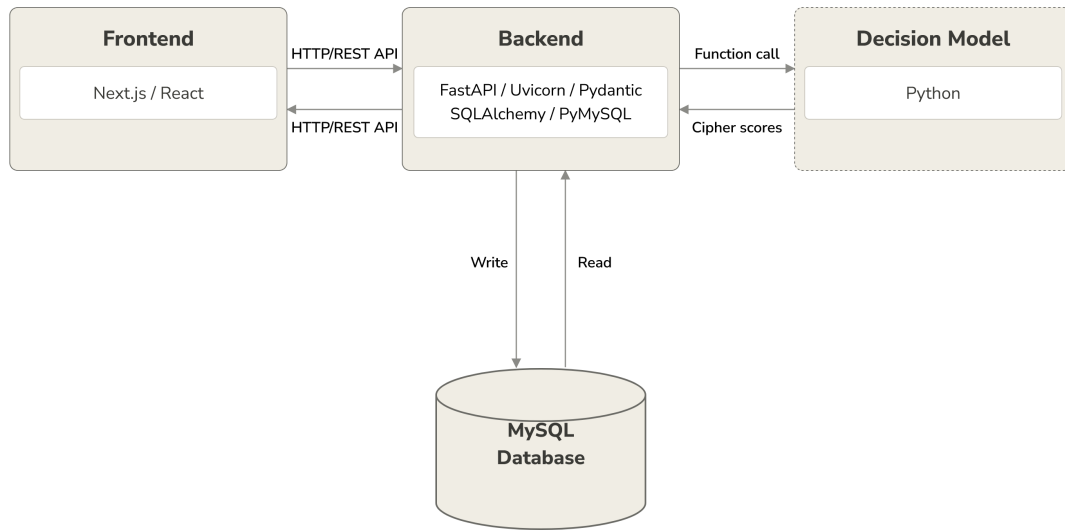


FIGURE 5.1. System architecture

5.2. Diagrams

This section presents the prototype’s Use Case, Activity, and Sequence diagrams, describing the interactions available to the user and the flow of information between the prototype’s components. The corresponding figures are presented in Appendix E.

5.2.1. Use Case Diagram

The PRISEC IV use case diagram (Figure E.1, Appendix E) provides an overview of the interactions available to its single actor, the User, when testing and applying the context-aware decision model.

- (1) **Create device profile:** the user registers a device’s hardware characteristics for later use, either manually or via automatic detection: CPU architecture and word size, clock speed, core count, RAM size, battery power, AES hardware acceleration, and the best available SIMD tier for ChaCha20 and RECTANGLE.

- (2) **Define request context:** the user specifies the operational scenario a cipher is being recommended for: security level (Guest, Basic, Advanced, Admin), data confidentiality (Low, Medium, High), data lifetime (Short-term, Medium-term, Long-term), duty cycle (Sporadic, Periodic, Continuous), latency tolerance (High, Medium, Low), required throughput (Low, Medium, High, Very High), and packet size.
- (3) **Submit test request:** the user requests a recommendation for the defined profile and context, optionally weighting how the decision model balances device suitability, security strength, and application fit.
- (4) **Run decision model** «include»: evaluates each candidate cipher and cascade against the profile and context across the three weighted dimensions (Section 5.3).
- (5) **View recommended cipher** «include»: displays the recommended cipher or cascade, feasibility (and reason, if infeasible), and the score behind the recommendation.
- (6) **Request encryption on device** «extend»: available only when the profile corresponds to the real, physically available device, allowing the recommendation to be validated against genuine on-device measurements rather than the benchmark dataset alone.
- (7) **View execution results** «include»: displays the measured encryption time, decryption time, and throughput from executing the recommended cipher on the device.

5.2.2. Activity Diagram

The activity diagram (Figure E.2, Appendix E) traces the same interactions as a sequential flow, making explicit the two decision points that the use case diagram only implies.

- (1) The flow begins with Start, followed by Create device profile, Define request context, and Submit test request, the same three initial steps detailed in Section 5.1.1.
- (2) Run decision model executes automatically upon request submission, followed by View recommended cipher, which displays the recommendation along with its score.
- (3) At the first decision point, Change context?, a “yes” branch returns to Define request context, allowing the user to adjust the operational scenario for the same device profile and re-run the decision model without recreating the profile.
- (4) A “no” branch proceeds to the second decision point, Real device specifications? A “no” here proceeds directly to End, reflecting that a recommendation is itself a complete and valid outcome of the flow, independent of whether it is subsequently executed.
- (5) A “yes” branch proceeds to Request encryption on device, Execute cipher on device, and View execution results before the flow reaches End.

5.2.3. Sequence Diagram

The sequence diagram (Figure E.3, Appendix E) details the interactions among the prototype’s five components: User, Frontend, Backend, Decision Model, and Execution Layer, across the two phases the activity diagram distinguishes: obtaining a recommendation and, when applicable, executing it.

- (1) The user sends device specifications to the Frontend, which forwards the request to the Backend. The Backend stores the profile in the database and confirms receipt to the Frontend.
- (2) The user then sends the request context and submits a test request to the Frontend.
- (3) The Frontend forwards the test request to the Backend, which sends the device profile and context to the Decision Model.
- (4) The Decision Model returns a recommended cipher to the Backend, which relays it to the Frontend for display to the user.
- (5) The remainder of the sequence is contained within an optional (**opt**) fragment, active only when the submitted profile corresponds to real device specifications. Within it, the user requests encryption on the device.
- (6) The Execution Layer returns the measured encryption/decryption time and throughput to the Backend, which forwards the execution results to the Frontend for display to the user, completing the flow.

5.3. Context Parameters

PRISEC I through III each tied a cipher selection to a fixed security tier: a request classified as Guest, Basic, Advanced, or Admin was mapped to a predetermined algorithm or combination, regardless of anything else about the device running it or the conditions under which it operated.

This is a good starting point, but it lacks adaptability. To create a framework that truly adjusts instead of just categorizing requires identifying which real-world conditions actually cause the best cipher for a request to change, and defining those conditions precisely enough for a model to reason about them.

PRISEC IV identifies three such conditions: the physical device on which a cipher will run, the level of protection the data being handled actually needs, and the operational demands the request itself places on the device. These three factors are independent of each other. Two requests executing on the same device may require entirely different levels of protection; requests demanding the same level of protection may operate on different devices; and requests with identical performance requirements can differ in all other aspects. None of these factors can be inferred from the others, which is why each is considered an independent axis in the decision-making process. The remainder of this section defines the parameters that make up each of these three categories, what each one is, and the conditions under which each possible value applies.

5.3.1. Device

The Device category captures the physical hardware on which a cipher will actually execute. This includes both processing capacity, which encompasses how much computational work the device can perform and how quickly, and a set of specific hardware features whose presence or absence changes a cipher’s real-world cost far more than raw processing power alone does.

5.3.1.1. CPU Architecture and Word Size

CPU architecture and word size record the processor family and its native word size, the number of bits processed as a single unit in one instruction. A cipher’s arithmetic is defined in terms of its own word size; AES, for instance, operates on 32-bit words. A device whose native word is narrower than the cipher it is running cannot perform that arithmetic in a single instruction and instead breaks it into a sequence of smaller ones, at a measurable performance cost.

5.3.1.2. Clock Speed

Clock speed is the number of cycles a device’s processor executes per second, most commonly measured in gigahertz. It is the primary determinant of raw execution speed: the same sequence of instructions takes proportionally longer to complete on a slower clock, all else being equal.

5.3.1.3. Core Count

Core count records the number of physical processing cores a device has available, ranging from a single core on the simplest microcontrollers to several on modern high-end devices.

5.3.1.4. RAM Size

RAM size is the total memory available to the device, typically ranging from a few kilobytes to several gigabytes. It determines both whether a given cipher can run on the device at all and how much of the device’s total memory that cipher’s footprint consumes once it does.

5.3.1.5. Hardware Acceleration

Hardware acceleration records whether a device exposes a dedicated AES instruction set and the most capable vector instruction set it supports for ChaCha20 or RECTANGLE. Both are binary or tiered hardware features found in modern processors but frequently missing from smaller microcontrollers. Their presence changes a cipher’s real-world cost independently of clock speed, word size, or any of the device’s other characteristics.

5.3.1.6. Battery-powered

It is also recorded whether a device runs on a battery or a continuous mains supply. Battery-powered devices operate under an energy budget that mains-powered devices do not.

5.3.2. Security

Security parameters determine the level of protection required for a request's data. The security needs depend on the requester's position within the network's access hierarchy and the sensitivity of the specific data involved. A further, distinct consideration is how long the resulting ciphertext will be stored after encryption and should therefore influence the choice of algorithm and key size.

5.3.2.1. Security Level

Security level reflects the requester's position in the network's access hierarchy, inherited from the four-tier structure already established. Its four values are: **Guest**, the lowest tier, held by unauthenticated or minimally trusted requesters; **Basic**, held by ordinary authenticated users or devices; **Advanced**, held by requesters with elevated operational privileges; and **Admin**, the highest tier, held by requesters with full administrative control.

5.3.2.2. Data Confidentiality

Data confidentiality reflects what is being protected in this specific instance of it. Its three values adapt the data classification logic Zeshan et al. propose for their own framework [34] following concrete rules:

- a) If the data concerns an individual such as identity, location, behaviour, or other personally attributable information, its confidentiality is considered **High**.
- b) If the data concerns the environment or a device's own operating conditions, its confidentiality is considered **Medium**.
- c) If the data is common routing or system metadata with no meaningful content of its own, its confidentiality is considered **Low**.

5.3.2.3. Data Lifetime

Data lifetime indicates how long encrypted data must remain protected. For computationally secure cryptographic schemes, once ciphertext is transmitted or stored, it does not acquire additional cryptographic strength merely through the passage of time and may become more vulnerable as the threat environment evolves. In a harvest-now-decrypt-later scenario, an adversary may intercept and retain ciphertext today for subsequent analysis or decryption, benefiting from advances in cryptanalysis, increases in computing power, newly discovered weaknesses, or access to new computational paradigms that were unavailable at the time of encryption [44], [45]. Consequently, data that requires protection only briefly is exposed to a comparatively shorter period of long-term cryptanalytic risk, whereas data that must remain confidential for years requires security mechanisms capable of meeting that extended protection lifetime.

- a) If the data loses all relevance within seconds to minutes of being generated, its lifetime is considered **Short-term**.

- b) If the data remains relevant for days to weeks, its lifetime is considered **Medium-term**.
- c) If the data must remain confidential for years, its lifetime is considered **Long-term**.

5.3.3. Application

Application parameters capture the performance conditions a request is subject to. These conditions determine what a candidate cipher must deliver in a given instance, rather than what it is capable of in principle: two requests running on identical hardware and protecting equally sensitive data can still demand entirely different behavior from a cipher if one is a single, delay-tolerant transmission and the other is a continuous, latency-critical stream. Application is defined by four parameters: duty cycle, latency tolerance, required throughput, and packet size.

5.3.3.1. Duty Cycle

Duty cycle reflects how often a device needs to perform encryption. This distinction matters because a cipher's fixed, one-time costs, such as setup or key-scheduling time, are paid once per session, not once per encryption within it. The more encryptions a session contains, the more that fixed cost is spread across them. Its three values follow concrete rules:

- a) If a device encrypts only occasionally, with long, irregular intervals between operations, its duty cycle is **Sporadic**.
- b) If a device encrypts at regular, predictable intervals, its duty cycle is **Periodic**.
- c) If a device encrypts continuously or near-continuously, with negligible idle time between operations, its duty cycle is **Continuous**.

5.3.3.2. Latency Tolerance

Latency tolerance reflects how quickly a request needs a result once encryption begins, a purely per-operation measure independent of how many times the operation is repeated or how far apart those repetitions fall. Its three values follow concrete rules:

- a) If a delayed result compromises the request's purpose, its latency tolerance is considered **Low**.
- b) If a request expects a timely but not instantaneous result, its latency tolerance is considered **Medium**.
- c) If a delayed result has no meaningful consequence, its latency tolerance is considered **High**.

5.3.3.3. Throughput Required

Throughput required indicates how much data a request requires a cipher to process per unit of time. Its four values follow concrete rules:

- a) If a request involves a single, small measurement, its throughput required is considered **Low**.

- b) If a request involves a moderate, structured measurement, its throughput required is considered **Medium**.
- c) If a request involves a substantial payload processed as a single operation, its throughput required is considered **High**.
- d) If a request involves sustained, high-volume data movement, its throughput required is considered **Very High**.

5.3.3.4. Packet Size

Packet size is the size, in bytes, of the specific payload to be encrypted for a given request. Unlike the other three Application parameters, packet size is not expressed as one of a small set of categorical values but as a concrete figure, since it is used directly against the benchmark data established in Chapter 4.

5.4. Decision Model

Having established the three independent axes a request can vary along in Section 5.3, this section describes how those parameters are turned into an actual recommendation. Each candidate cipher or cascade in PRISEC IV's catalog is scored against a request along three separate dimensions, device fit, security fit, and application fit, each addressing one of the three axes defined previously and computed independently of the other two. The three scores are then combined into a single final score, from which the highest-scoring candidate is recommended.

5.4.1. Device Fit

Device fit scores how well a candidate cipher fits the physical hardware specifications defined in Section 5.3.1.

5.4.1.1. Memory Fit

Memory is evaluated in two distinct steps. Before any score is computed, a cipher's memory footprint is checked against the device's actual RAM: if its estimated footprint at the request's packet size exceeds the device's total RAM, the cipher is excluded from consideration entirely. For every cipher that passes this check, memory fit is computed as one minus the fraction of the device's RAM the cipher's footprint consumes, with a 25% penalty applied once that fraction exceeds 50%. A cipher that requires more than half the device's total memory for a single encryption operation is penalized since the remaining memory must still hold the operating system, network stack, and the rest of the application, not just the cipher.

5.4.1.2. Word Fit

Word fit captures whether a device's native word size, the number of bits its processor handles in a single instruction, matches the word size that a cipher's arithmetic is defined

in terms of. This is expressed as:

$$\text{word_fit} = \min \left(1.0, \frac{\text{device_word_bits}}{\text{cipher_word_bits}} \right).$$

A device at least as wide as the cipher’s word size produces a ratio of 1.0 or greater, capped at 1.0, and never pays a penalty at all; a device narrower than the cipher’s word size produces a ratio below 1.0, reflecting that its processor cannot perform that cipher’s arithmetic in one instruction and instead breaks it into a sequence of smaller ones, at a cost proportional to how much narrower it is. This base relationship, however, does not hold uniformly across every cipher in PRISEC IV’s catalog, since several are specifically exempted under certain conditions.

RECTANGLE is never penalized on any device, regardless of word size, because its bit-sliced implementation adapts to whatever register width is available rather than assuming a fixed one. HIGHT is defined against an 8-bit word, the narrowest realistic case among any cipher in the catalog, and is consequently never penalized on any device evaluated in this dissertation. ChaCha20 is fully exempted whenever a device exposes a SIMD instruction tier, since SIMD lanes perform ChaCha20’s 32-bit ARX operations independently of the device’s underlying scalar word width. In the absence of SIMD support, ChaCha20 falls back to the base word-size relationship. AES is exempted whenever a device exposes hardware acceleration for AES: `word_fit` is 1.0 for any device at least 32 bits wide, and a flat 0.9 for any device narrower than that, regardless of exactly how narrow, since it executes an entire round as a single fixed-function instruction, leaving the device’s own scalar word width with little bearing on it either way. SPECK, meanwhile, is governed by $\text{word_fit} = \text{device_word_bits}/16$, a 16-bit floor rather than its own 64-bit operand size: any device at least 16 bits wide incurs no penalty at all, since a SPECK variant exists at exactly that word size, leaving no realistically narrower case to penalize against. For a cascade combining more than one cipher, word fit is determined by whichever component scores worst, and the cascade’s overall fit is only ever as good as its weakest link, reflecting that every stage of a cascade must execute on the same device.

5.4.1.3. Energy Fit

Energy fit, computed only for battery-powered devices, is derived from each cipher’s separately measured energy cost per encryption normalized against every cipher and cascade in the catalogue. No absolute battery-capacity ceiling exists in a device’s profile, only whether it is battery-powered at all, so energy fit reflects a cipher’s standing relative to the rest of the catalogue rather than a true utilization ratio, in contrast to memory fit’s ratio against a device’s actual RAM.

5.4.1.4. Combining the Components

For a device without a battery, device fit is the simple average of memory fit and word fit:

$$\text{device_fit} = 0.5 \times \text{memory_fit} + 0.5 \times \text{word_fit}.$$

For a battery-powered device, energy fit is added as a third term, each scaled by its own weight:

$$\text{device_fit} = w_{\text{mem}} \times \text{memory_fit} + w_{\text{word}} \times \text{word_fit} + w_{\text{energy}} \times \text{energy_fit},$$

where the three weights are determined by the request’s duty cycle (Appendix F).

These weights are not fixed because the relative importance of energy consumption is not fixed either, as it depends entirely on how often the device encrypts. A device with a sporadic duty cycle performs so few encryption operations that even a comparatively energy-hungry cipher barely dents its total battery life, so memory and word fit carry most of the weight. As duty cycle becomes more continuous, that assumption breaks down: the same per-operation energy cost is no longer paid occasionally but repeatedly, and it accumulates into a significant drain on the device’s operating lifetime rather than a negligible one. The weighting scheme in Appendix F reflects this directly: memory fit and word fit are always weighted equally to each other, but both are scaled down together as duty cycle becomes more continuous, with energy fit’s weight growing by exactly the amount the other two give up.

5.4.2. Security Fit

5.4.2.1. Requirement

Security fit compares what a request requires against what a candidate cipher or cascade offers, reducing the three parameters defined in Section 5.3.2 to a single requirement value through a weighted sum:

$$\text{requirement} = 0.4 \times \text{security_level} + 0.4 \times \text{data_confidentiality} + 0.2 \times \text{data_lifetime}.$$

Security level and data confidentiality contribute equally and more heavily to this requirement, while data lifetime is the minor factor, making it able to push a requirement up or down without being allowed to dominate it the way who is asking and what is being protected should (exact value scales in Appendix G.1 and G.2). This produces a single number, the requirement, against which every candidate’s own security strength is then compared.

5.4.2.2. Cryptanalytic Strength Dimensions

Every cipher and cascade is evaluated qualitatively across four dimensions grounded in the cryptanalytic literature (Appendix G.3). Key size determines brute-force resistance directly: AES-256 and ChaCha20, both with 256-bit keys, are favored over ciphers with 128-bit keys. Block size matters independently of key size, since a 64-bit block, used in HIGHT and RECTANGLE, becomes susceptible to birthday-bound collision attacks after roughly 2^{32} blocks have been encrypted under the same key, a volume of traffic well within reach for a device in continuous or high-throughput use; SPECK’s and AES’s 128-bit blocks push this same threshold far beyond what any realistic deployment would ever produce, and are favored accordingly. Security margin measures the proportion

of a cipher’s total rounds that remain unbroken against the best published attack. This separates AES and ChaCha20, both with every round still standing after decades of cryptanalysis, from HIGHT, SPECK, and RECTANGLE, each of which has had a substantial fraction of its rounds broken (26 of 32, 20 of 26, and roughly 18 of 25, respectively) even though none has been broken in full. Maturity and scrutiny reflect how long and how widely a cipher has actually been attacked in practice, not just how it performs on paper: AES has stood as a NIST standard since 2001, subject to over two decades of sustained public cryptanalysis, and ChaCha20 has accumulated over fifteen years of the same scrutiny since its standardization, while SPECK, HIGHT, and RECTANGLE remain comparatively academic proposals, with far less standardization and far fewer independent attempts to break them.

5.4.2.3. Cascade and ECC Bonuses

A cascade’s bonus over its strongest single component requires a more careful argument than simply crediting it for combining multiple keys. The naive expectation that chaining two ciphers together squares the effective keyspace an attacker must search, since two independent κ -bit keys should yield $2^{2\kappa}$ security, does not hold against a meet-in-the-middle attack. Given a known plaintext-ciphertext pair, an attacker can compute every possible result of the first cipher forward, every possible result of the second cipher backward, and search for where the two meet, reducing the effective work to roughly $2^{\max(k_1, k_2)+1}$ rather than $2^{k_1+k_2}$. This result is demonstrated historically by Double-DES, whose naively expected 2^{112} security collapses to barely more effort than breaking single DES (2^{56}) once this attack is applied. This bound, however, assumes an attacker who can passively collect known plaintext-ciphertext pairs, which is precisely what PRISEC IV’s edge-encryption threat model denies them: since plaintext is encrypted at the device itself or near the device and is never exposed to internet protocols, an attacker has no equivalent channel from which to harvest such pairs and is instead left to guess plaintext structure with comparatively high entropy. Under this constraint, a cascade’s effective security would sit closer to the full $2^{k_1+k_2}$ than the pessimistic meet-in-the-middle bound alone would suggest ($2^{\max(k_1, k_2)+1}$), and PRISEC IV’s cascade bonuses are set accordingly, less conservatively than a pure meet-in-the-middle analysis would justify.

ECC receives a bonus of its own when included in a cascade. Unlike stacking a second symmetric cipher on top of a first, ECC does not chain another transformation of the same plaintext. Instead, it protects an entirely separate stage of the process: the establishment of the key that the symmetric ciphers subsequently use. An attacker gains no shortcut by attacking the symmetric portion and the key exchange as though they were one continuous transformation; each must be broken on its own terms.

5.4.2.4. Comparison Rule

Security fit follows the same asymmetric comparison rule used throughout the decision model. If a candidate’s security strength meets or exceeds the request’s requirement, its

fit equals the requirement, capped so that a cipher does not gain an advantage purely for being stronger than what was actually asked for; if its strength falls short of the requirement instead, its fit is discounted by a further 25%, meaning that falling short costs more than the raw gap alone would suggest, consistently across every dimension of the model.

5.4.3. Application Fit

Application fit compares what a request demands in throughput and latency with what a candidate cipher or cascade delivers. The benchmark data was measured on a single machine at a single clock speed, so it cannot be compared directly with a request running on a different device without first being rescaled to that device’s terms.

5.4.3.1. Scaled Time

Scaled time performs this rescaling before anything else happens:

$$\text{scaled_time} = \text{benchmark_time} \times \left(\frac{\text{reference_clock_mhz}}{\text{device_clock_mhz}} \right) \times \text{word_penalty}.$$

5.4.3.2. Throughput and Latency Fit

Once scaled, throughput and latency are each converted into a fit score using the same two-step rule. First, every candidate cipher or cascade being compared in that request has its scaled value normalized to a 0–1 scale relative to the others in the same pool. The candidate offering the highest throughput (or lowest latency) among them scores 1.0, the weakest scores 0.0, and the rest fall proportionally in between. Second, this normalized value is compared against the request’s own throughput or latency requirement (value scales in Appendix H.1). If the offer meets or exceeds the requirement, the fit itself is the requirement, capped so that a cipher gains nothing further for exceeding what was asked for. If an offer falls short, the fit is discounted by a further 25%, the same penalty factor already used.

5.4.3.3. Setup Fit

Setup fit is not scored this way. It measures a cost distinct from throughput or latency: the fixed price a cipher pays before any encryption begins, relative to the cost of the encryption that follows it. The amortization factor expresses what share of a cipher’s total cost, at the request’s packet size, is setup rather than encryption:

$$\text{amortization_factor} = \frac{\text{scaled_setup_time}}{\text{scaled_setup_time} + \text{scaled_encryption_time}},$$

$$\text{setup_fit} = 1 - \text{amortization_factor}.$$

5.4.3.4. Combining the Components

This same amortization factor also governs how much setup fit matters in the first place. Duty cycle assigns each of throughput, latency, and setup a base weight (Appendix H.3),

but setup’s base weight is scaled down by the amortization factor itself before use:

$$w_{\text{setup_eff}} = w_{\text{setup}} \times \text{amortization_factor},$$

$$\text{freed} = w_{\text{setup}} - w_{\text{setup_eff}},$$

$$w_{\text{throughput_eff}} = w_{\text{throughput}} + \frac{\text{freed}}{2}, \quad w_{\text{latency_eff}} = w_{\text{latency}} + \frac{\text{freed}}{2}.$$

A cipher whose setup cost is already a negligible share of its total cost has that negligible share reflected in how much weight setup actually gets, rather than counting a fixed amount regardless of how much it matters for that specific cipher and packet size. Whatever weight setup gives up this way is redistributed evenly between throughput and latency. A large payload amortizes a cipher’s setup cost to almost nothing, freeing nearly all of setup’s weight for throughput and latency, while a small payload leaves setup cost still mattering, and its weight correspondingly intact.

The final application score combines all three:

$$\text{application_fit} = w_{\text{throughput_eff}} \times \text{throughput_fit} + w_{\text{latency_eff}} \times \text{latency_fit} + w_{\text{setup_eff}} \times \text{setup_fit}.$$

5.4.4. Final Score

Device fit, security fit, and application fit are combined into a single final score for each candidate cipher or cascade:

$$\text{final_score} = w_{\text{device}} \times \text{device_fit} + w_{\text{security}} \times \text{security_fit} + w_{\text{application}} \times \text{application_fit}.$$

The three top-level weights are user-configurable because how much a device’s constraints, the data’s security needs, and the request’s performance demands should matter relative to one another depends on the priorities of the person making the request. If no preference is submitted, the three are weighted equally, one third each.

The candidate with the highest final score is recommended. On the rare occasion that two or more candidates tie exactly, the tie is broken by whichever has the higher **security_strength** among those tied. Only if **security_strength** also ties does the recommendation return more than one candidate rather than a single winner.

5.5. Backend Implementation

A decision model that exists only as a set of formulas cannot recommend anything to anyone. Having described the decision model’s own logic in Section 5.4, this section turns to how that logic is exposed in a working system: the API endpoints a request passes through and the database tables where a profile and its decisions are persisted.

5.5.1. API Endpoints

The backend exposes nine endpoints. Five endpoints manage device profiles directly: creating a profile (POST /profiles), updating a profile (PUT /profiles/{id}), listing all stored profiles (GET /profiles), retrieving a profile by its identifier (GET /profiles/{id}), and deleting a profile (DELETE /profiles/{id}).

`POST /decision` is where a profile and a context become a recommendation. It retrieves the requested profile from the database, validates any supplied custom weights, and passes the resulting device and context objects to the decision model. Before scoring, the decision model enforces one further constraint not mentioned in any earlier section: a packet size above 100 MB is rejected outright, with an explicit reason returned rather than a silent extrapolation, since the largest size actually benchmarked is 50 MB, and estimating meaningfully beyond that is judged too far from any real measurement to trust. The endpoint's response always includes the recommended cipher or cascade, whether the request was found infeasible and why, which candidates were excluded for exceeding the device's memory, and the full per-cipher score breakdown behind the recommendation. The full score breakdown is present in every response regardless of mode but only displayed to the user in the frontend's deeper analysis mode (Section 5.6), which surfaces every candidate's breakdown rather than only the winner's for a deep analysis for the user. Whether the resulting decision is persisted to the database is a separate, independent choice: the request can set a persist flag to either value in both the normal and deeper analysis flows, rather than one mode always saving and the other never doing so.

The remaining endpoint, `POST /execute`, is what runs a cipher rather than merely recommending one. Given a profile, a cipher name, and a packet size, it compiles and runs a small C program, performing one encryption and decryption on whichever machine the backend process happens to be running on, discarding five warm-up runs beforehand to avoid cold-start skew. Its response includes the measured time, throughput, latency, and memory overhead. The endpoint has no way to verify that the profile it is given describes the machine it is running on, and that check is the frontend's responsibility.

5.5.2. Database Schema

Two tables underpin the backend's persistence: `profiles` and `decisions`. A profile row stores every Device parameter defined in Section 5.3.1, along with a name, and whether it was entered manually or detected automatically. A decision row stores a foreign key back to the profile it was made for, the full context it was evaluated against, the resulting recommendation, and a metadata field that captures any additional information about that decision without requiring a dedicated column for each.

5.6. Frontend Implementation

This section presents the prototype from a user's perspective, following the four steps outlined in the use case diagram in Section 5.2.1: creating a profile, defining a context, making a decision based on both, and testing that decision in practice rather than just recommending it. The first subsection corresponds to the profile, while the other details the relevant pages for the context of the decision and the execution, and their functions for the user.

5.6.1. Profile

Figure 5.2 shows the profile creation page, the entry point to everything that follows: a form covering every defined Device parameter. These include CPU architecture, clock speed, core count, RAM, whether the device is battery-powered, and its support for AES hardware acceleration and SIMD acceleration. These parameters can be entered either by hand or, when the browser’s own machine is being profiled, via a single auto-detect action that queries `GET /detect-specs` and populates the form automatically.

FIGURE 5.2. Profile creation page of the PRISEC IV prototype.

The edit page is not shown separately, since it presents the same fields, in the same layout, as the creation page in Figure 5.2, with the only difference being that it opens pre-filled with a profile’s existing values rather than empty ones and saves changes to that profile in place rather than creating a new one.

Figure 5.3 shows the profile list, from which an existing profile is selected rather than recreated for each subsequent test. Its job is simply to provide an overview of all profiles stored so far and a way back into any one of them, whether to review it, delete it, or carry it forward into a context.

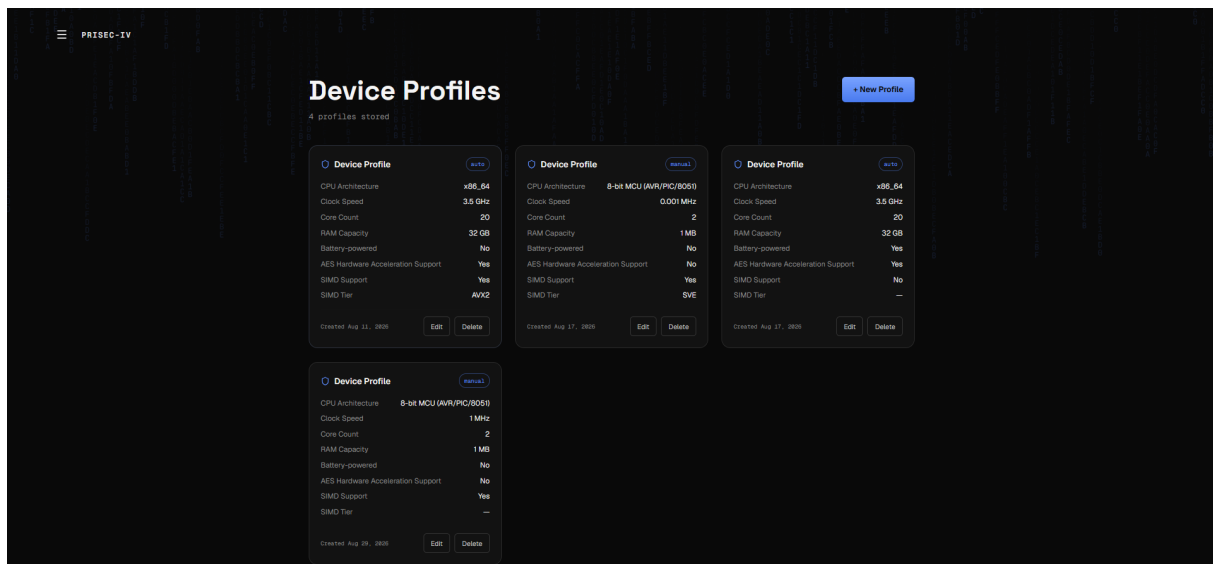


FIGURE 5.3. Profile list page of the PRISEC IV prototype.

5.6.2. Context, Decision, and Execution

Context, the Decision model run, and Execution of the encryption are not three separate views, but three phases of the same two almost-identical pages. From a selected profile, the first does exactly two things: take in a context and return a recommendation, and, where appropriate, let that recommendation be tested against a real encryption. The second, “Deeper Analysis,” exists to lay out, for every candidate cipher and cascade rather than only the winner, the full set of scores each was evaluated on, and not only the final score.

In Figure 5.4 is shown the first state of the context page. Each of the seven defined parameters is mandatory and entered from its own set of possible values, while packet size is free text with a unit selector. Below these, the three top-level weights from the decision model are set through sliders, letting a user weight device suitability, security strength, and application fit against one another however a given request calls for, rather than accepting the equal-thirds default. A button located in the top-right corner of the page allows users to switch to the Deeper Analysis view described and toggle back again. After selecting the context and, if desired, changing the weights, the user can then press “Run Simulation” to run the decision model.

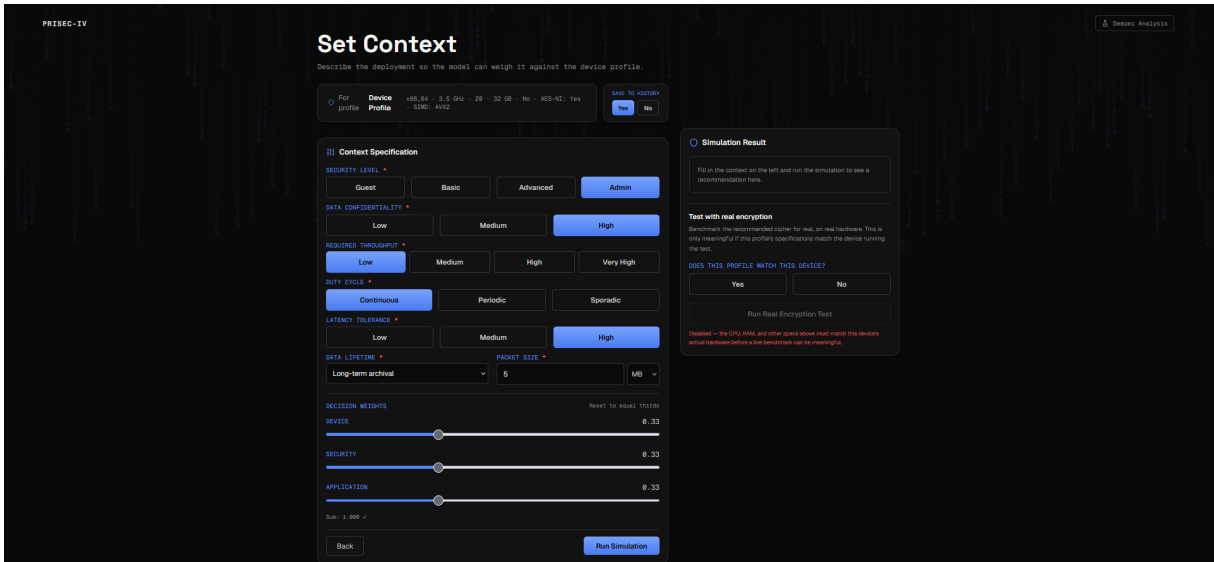


FIGURE 5.4. Context page of the PRISEC IV prototype.

Figure 5.5 shows the recommendation result on the main page. The result is the single cipher or cascade with the highest final score among all evaluated candidates, shown together with that score and labeled for the user as its “Cipher Fit.” When the request’s packet size did not exactly match one of the benchmarked sizes, the result includes one or two small caveat tags alongside it: *Interpolated*, when the figure used was calculated between two benchmarked sizes rather than measured at exactly one of them, or *Below Min Sample*, when the packet size fell below the smallest size benchmarked. This is a transparent signal that this particular number involved some degree of estimation, rather than presenting every recommendation as equally direct and measured.

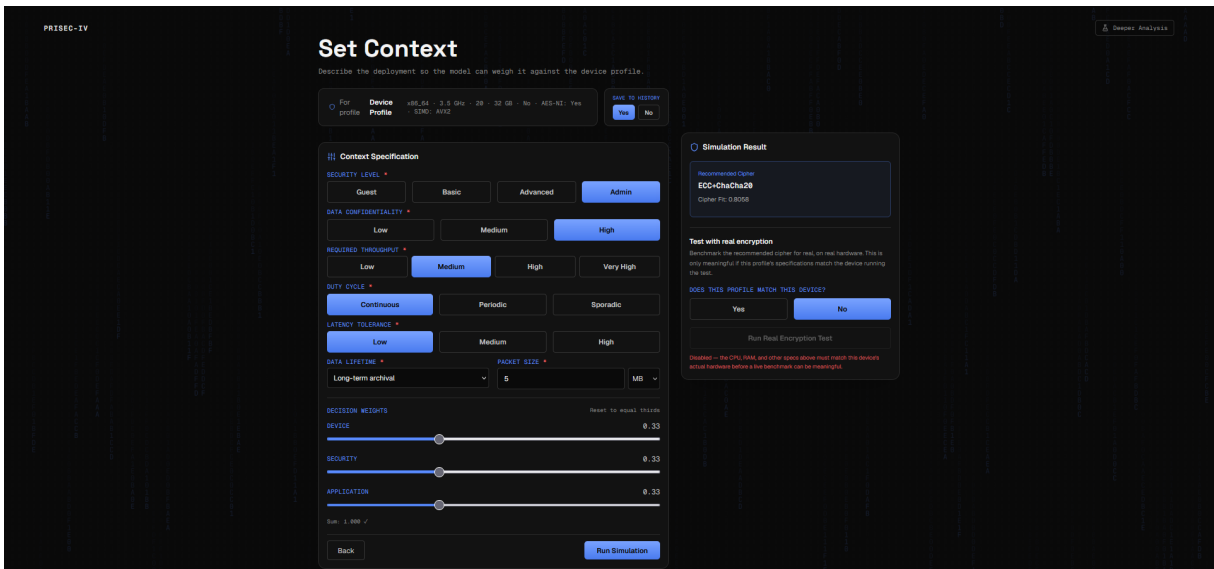


FIGURE 5.5. Recommendation result page of the PRISEC IV prototype.

Deeper Analysis, shown in Figure 5.6 (full example table in Appendix I), derives from the same result, adds to this single answer with a table: every candidate, not only the

winner, with its `device_fit` score and `memory_fit/word_fit/energy_fit` sub-scores, its `security_fit` score and the `security_strength/requirement` pair behind it, and its `application_fit` score and `throughput_fit/latency_fit/setup_fit/amortization_factor` together. The complete arithmetic trail behind every number, laid bare for whoever wants to audit it.

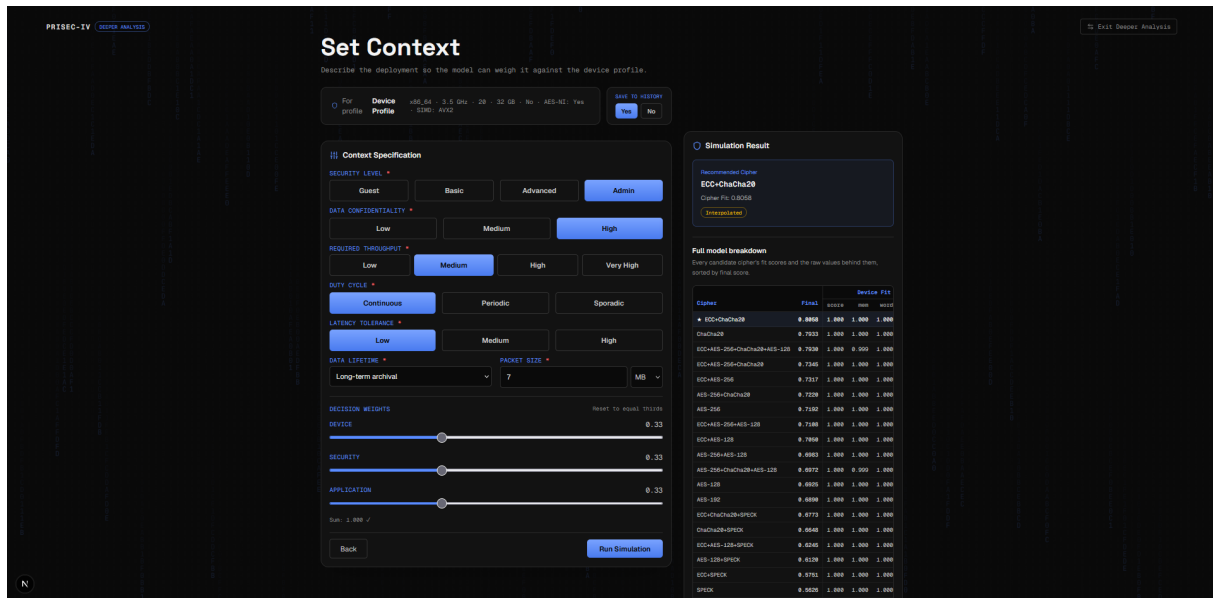


FIGURE 5.6. Deeper Analysis view of the PRISEC IV prototype.

A recommendation is computed from a profile’s stated specifications, not from the machine running the prototype. The two are the same only when the profile being tested describes that machine. Running a live encryption on hardware other than what the profile describes would measure that different hardware’s real performance while still comparing it against a recommendation computed for something else, an inconsistency with no meaningful result. Because of this, the option to run a real encryption is gated behind explicit user confirmation that this specific profile describes the device the prototype is currently running on. Once confirmed, the test sends the recommended cipher and the context’s packet size to the backend, which returns the measured encryption time, throughput, latency, and memory overhead from that single run, shown in Figure 5.7.

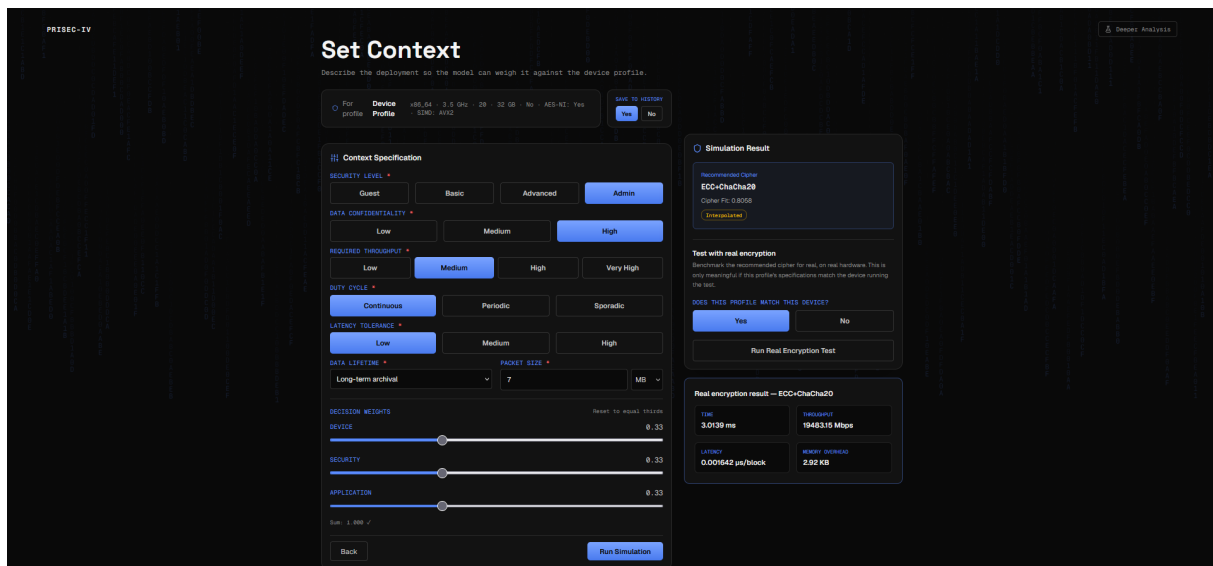


FIGURE 5.7. Live encryption execution result in the PRISEC IV prototype.

Conclusion and Future Work

6.1. Conclusion

Each of PRISEC’s earlier versions [9], [10], [11] treated cryptographic selection as a classification problem: a request was assigned a security tier (Guest, Basic, Advanced, or Admin), and that tier was mapped to a predetermined algorithm or combination, fixed in advance and applied uniformly regardless of the device actually running it or the operational conditions surrounding a given request. This design is not without merit; it is simple to reason about, easy to implement, and gives every request in a security tier a clear, predictable outcome. However, it treats every request from a tier as though it were one and the same. A device’s hardware constraints, the sensitivity of the data it happens to be handling, and the performance demands a specific request places on it can each vary without the security tier changing. A framework that collapses all these into a single tier has no way to tell the difference between a battery-constrained sensor node and a mains-powered gateway sharing the same security level, or between a request where the transmitted data is simple routing metadata and one where it is private personal data.

PRISEC IV was built around the conviction that three properties(Device, Security, and Application) deserve to be treated as what they are: independent axes, each contributing its own evidence to a recommendation rather than being folded prematurely into a single tier-based classification. This was developed into a working decision model that scores every candidate cipher or cascade evaluated within the scope of this dissertation against each axis in turn. `device_fit` is assessed against a device’s actual memory, native word size, and energy budget; `security_fit` against a request’s access hierarchy, data sensitivity, and required protection lifetime; and `application_fit` against its duty cycle, latency tolerance, throughput, and payload size. These three values are then combined into a single, user-weighted final score.

Guest, Basic, Advanced, and Admin were not discarded in this process. They remain exactly where PRISEC III left them: as the reference profiles to which a security level resolves. However, they no longer directly determine a cipher. They are now one input among several that a request’s actual context can adjust, rather than the entire decision on their own.

A broader concluding question follows from the completed framework: what does an environment gain from adopting a hybrid, context-aware approach to cipher selection, beyond the mechanics of the selection itself? The answer is a solution capable of adapting to the current context while optimizing IoT resource use without compromising the

security robustness represented in the decision model. Two gains follow directly from how the model was built, neither of which was a foregone conclusion.

The first gain is resource efficiency, one that is not evenly distributed. On a capable, mains-powered device, there are, practically speaking, no hardware constraints for the model to weigh. As a result, `device_fit` is nearly constant across almost every candidate in the catalog: a few kilobytes of memory footprint against several gigabytes of RAM is a negligible utilization ratio regardless of which cipher is chosen, and a modern 64-bit CPU with AES-NI and a SIMD tier can run any candidate. On such a device, the framework’s recommendation converges toward whichever cipher `security_fit` and `application_fit` would have favoured, and device suitability effectively stops discriminating between candidates.

On a constrained device, this is not the case. The model must maintain the robustness needed to secure information while simultaneously optimizing resource use. This is not a one-size-fits-all allocation, but one that follows the actual shape of the request: because cipher choice is driven jointly by the security needs of the information, the application’s performance requirements, and the device’s real capabilities, a constrained device is neither permanently locked into its most expensive configuration nor into its cheapest. Where a request’s data has little to lose from disclosure and no strict performance demands of its own, the device is free to spend as little of its limited memory, processing time, and energy as the request allows, resources that would otherwise be committed to a stronger cipher whether or not the situation called for one. When the request carries real sensitivity or urgency, the same device is pushed toward whichever candidates its constraints still permit, excluding what it cannot support, so protection is not quietly under-delivered simply because the device is constrained. The device’s resource budget is thus spent where it is needed and conserved everywhere else, rather than fixed at one setting regardless of what a given moment requires. The practical consequence is that PRISEC IV’s marginal value scales inversely with a device’s capability, meaning that the framework’s greatest benefits concentrate on the resource-constrained devices that motivated this dissertation from the outset.

The second gain is less obvious and has nothing to do with resource savings. A deployment that always encrypts with the same fixed algorithm presents an attacker with a single, unchanging target: effort invested in studying and eventually exploiting that algorithm’s implementation can transfer directly to every device in the fleet, since every device is protected identically. Moving target defense (MTD) was developed to counter the advantage that attackers gain from static, predictable configurations, by introducing diversity, unpredictability, and change into the protected system [46]. MTD has also been applied directly to encryption: Tang *et al.* propose a dynamic encryption scheme under the MTD concept, arguing that widely used ciphers such as triple DES and AES cannot meet the requirements of dynamic defense because their key spaces are fixed [47]. Their scheme varies keys within a single construction over time, whereas PRISEC IV produces

a comparable effect at a different level, varying which algorithm protects a given device and request.

PRISEC IV was not designed as a moving-target-defense mechanism, but it produces a comparable effect as a byproduct of its design. Because the cipher protecting a given flow depends on a specific device’s profile, a specific request’s context, and a specific set of top-level weights, none of which is necessarily observable to an outside attacker, knowing a device’s security level alone does not reveal which of up to 27 catalog entries protects it. This contrasts with a static tier-to-cipher table, where knowing the tier reveals the cipher outright. This property is not a substitute for any individual cipher’s cryptographic strength, which is already accounted for directly and is a defense-in-depth benefit sitting on top of that strength. It makes cipher selection less predictable to an adversary and raises the cost of preparing a single exploit that transfers across a heterogeneous fleet.

In PRISEC III [10], the only distinction is the security tier, which determines which combination of ciphers to use to protect information. In the framework developed for PRISEC IV, this decision is based on the device, the sensitivity of the information, and the requirements of the running application. A constrained device is neither burdened with an unnecessarily expensive cipher nor left underprotected by a cheap default, and two devices sharing the same security level are not guaranteed to be defended by the same algorithm in the first place. Resource efficiency and unpredictability are therefore benefits derived from the same design principle: choosing a cipher for a specific device and a specific request, rather than looking it up in a table built to cover every device and every request at once.

6.2. Key Scientific Contributions

The principal contributions of this dissertation are as follows:

- A context-aware cryptographic framework that extends PRISEC’s existing role-based tier structure, whose resource-efficiency benefits concentrate precisely where they are most needed, on constrained devices, and whose device- and context-dependent cipher choice introduces a defense-in-depth unpredictability absent from a static tier-to-cipher configuration.
- A cipher catalog spanning conventional, standardized algorithms and lightweight algorithms, combinable as single ciphers or multi-layer cascades, together with ECC for secure asymmetric key establishment.
- A comprehensive empirical benchmark of the selected single ciphers and cascades across encryption time, decryption time, throughput, latency, memory overhead, setup cost, energy consumption, and the effect of available hardware acceleration.
- A fully quantified, rule-based decision model that establishes a structured evaluation baseline against which future machine-learning-based approaches to cipher selection can be trained and compared, rather than requiring such approaches to be built from first principles.

- A working prototype that makes the security–performance trade-off explicit and directly configurable; evaluates arbitrary combinations of device profiles and request contexts using user-adjustable weights across the three decision axes; and exposes a complete, auditable score breakdown for every candidate rather than only the final recommendation.

6.3. Future Work

PRISEC IV can be enhanced in the following areas:

- **Real-device, real-environment validation:** validate the framework’s recommendations across a heterogeneous fleet of physical IoT hardware, rather than the controlled set of profiles and the single-device live-execution capability demonstrated in this dissertation.
- **Broader coverage:** extend the desktop benchmark suite beyond its single reference machine, evaluate the catalog across a wider range of real-world CPU architectures and hardware-acceleration capabilities, and extend the range of packet sizes benchmarked.
- **Expanded cryptographic catalog :** incorporate parallelizable cipher modes, additional lightweight ciphers beyond the five evaluated in this dissertation, and post-quantum algorithm options that extend the catalog beyond classical cryptography.
- **Threat-informed key rotation:** integrate AI- or machine-learning-based intrusion and threat detection so a device’s active key material can be refreshed in response to a classified threat rather than on a fixed schedule alone.
- **Additional contextual parameters:** extend the model beyond the parameters developed here to include network conditions, regulatory or compliance context, adversary-model assumptions, interoperability requirements, and protocol constraints.

Bibliography

- [1] R. Chataut and A. Phoummalayvane, “Unleashing the Power of IoT: A Comprehensive Review of IoT Applications, Advancements, and Future Prospects in Healthcare, Agriculture, Smart Homes, Smart Cities, and Industry 4.0,” *Preprints*, Jun. 2023, doi: 10.20944/preprints202306.0002.v1.
- [2] L. Abuserrieh and M. H. Alalfi, “A Survey on Verification of Security and Safety in IoT Systems,” *IEEE Access*, vol. 12, pp. 138627–138645, 2024, doi: 10.1109/ACCESS.2024.3413071.
- [3] S. A. Ansari and S. Ali, “A systematic review of lightweight cryptographic schemes for security and privacy in IoT,” *Discover Computing*, vol. 28, art. 266, 2025, doi: 10.1007/s10791-025-09755-3.
- [4] M. Zahid and T. S. Bharati, “Leveraging Machine Learning and Deep Learning in IoT Security: A Review,” *Security and Privacy*, vol. 9, no. 1, art. e70144, 2026, doi: 10.1002/spy2.70144.
- [5] M. Aziz Al Kabir, W. Elmedany, and M. S. Sharif, “Securing IoT Devices Against Emerging Security Threats: Challenges and Mitigation Techniques,” *Journal of Cyber Security Technology*, vol. 7, no. 4, pp. 199–223, 2023, doi: 10.1080/23742917.2023.2228053.
- [6] I. Rozlomii, A. Yarmilko, and S. Naumenko, “Data security of IoT devices with limited resources: challenges and potential solutions,” in *Proceedings of the 4th Edge Computing Workshop (DOORS 2024)*, Zhytomyr, Ukraine, Apr. 2024, pp. 85–96. [Online]. Available: <https://ceur-ws.org/Vol-3666/paper13.pdf>
- [7] M. Litoussi, N. Kannouf, K. El Makkaoui, A. Ezzati, and M. Fartitchou, “IoT security: Challenges and countermeasures,” in *Procedia Computer Science*, Elsevier B.V., 2020, pp. 503–508. doi: 10.1016/j.procs.2020.10.069.
- [8] U. L. M. Rijah, M. I. M. Riyath, and M. A. C. Jiffriya, “A Review of IoT Architecture, Applications, Security Requirements and Mechanisms,” *Journal of Information Systems & Information Technology (JISIT)*, vol. 6, no. 1, pp. 10–27, 2021.
- [9] D. A. F. Saraiva, V. R. Q. Leithardt, D. de Paula, A. S. Mendes, G. V. González, and P. Crocker, “PRISEC: Comparison of symmetric key algorithms for IoT devices,” *Sensors (Switzerland)*, vol. 19, no. 19, Oct. 2019, doi: 10.3390/s19194312.
- [10] P. Costa and V. Leithardt, “Prisec II – A Comprehensive Model for IoT Security: Cryptographic Algorithms and Cloud Integration,” *IEEE Latin America Transactions*, 2024, arXiv:2407.16395, doi: 10.48550/arXiv.2407.16395.
- [11] H. Sohail, V. Leithardt, and A. Trigo, “PRISEC III: Cryptographic Techniques for Enhanced Security,” in *2025 25th International Conference on Control Systems and Computer Science (CSCS)*, Bucharest, Romania, May 2025, pp. 354–361, doi: 10.1109/CSCS66924.2025.00059.
- [12] K. Sasikumar and S. Nagarajan, “Comprehensive Review and Analysis of Cryptography Techniques in Cloud Computing,” *IEEE Access*, vol. 12, pp. 52325–52351, 2024, doi: 10.1109/ACCESS.2024.3385449.
- [13] P. Sharma, Anjali, R. Kumari, and P. Bansal, “Cryptography: A Comparative Analysis of Symmetric and Asymmetric Techniques,” in *2024 IEEE 3rd World Conference on Applied Intelligence and Computing (AIC)*, Gwalior, India, 2024, pp. 1343–1348, doi: 10.1109/AIC61668.2024.10730890.

- [14] A. L. Jeeva, V. Palanisamy, and K. Kanagaram, "Comparative Analysis of Performance Efficiency and Security Measures of Some Encryption Algorithms," *International Journal of Engineering Research and Applications (IJERA)*, vol. 2, pp. 3033–3037, 2012. [Online]. Available: <https://www.ijera.com>
- [15] G. Zhang, "Cryptographic Techniques in Digital Media Security: Current Practices and Future Directions," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 15, no. 8, 2024. [Online]. Available: <https://www.ijacsa.thesai.org>
- [16] S. Chandra, S. Paira, S. S. Alam, and G. Sanyal, "A Comparative Survey of Symmetric and Asymmetric Key Cryptography," in *2014 International Conference on Electronics, Communication and Computational Engineering (ICECCE)*, 2014, pp. 83–88, doi: 10.1109/ICECCE.2014.7086640.
- [17] B. Halak, Y. Yilmaz, and D. Shiu, "Comparative Analysis of Energy Costs of Asymmetric vs Symmetric Encryption-Based Security Applications," *IEEE Access*, vol. 10, pp. 76707–76719, 2022, doi: 10.1109/ACCESS.2022.3192970.
- [18] M. R. Al-Mousa et al., "IoT Security Based On Lightweight Cryptographic (LWC) Algorithms: A Survey," in *2024 25th International Arab Conference on Information Technology (ACIT)*, Institute of Electrical and Electronics Engineers Inc., 2024, doi: 10.1109/ACIT62805.2024.10876942.
- [19] A. Dhingra, V. Sindhu, and A. Sangwan, "Symmetrical Cryptographic Algorithms in the Lightweight Internet of Things," *International Journal of Informatics and Communication Technology*, vol. 14, no. 1, pp. 307–314, Apr. 2025, doi: 10.11591/ijict.v14i1.pp307-314.
- [20] Amrita, C. P. Ekwueme, I. H. Adam, and A. Dwivedi, "Lightweight Cryptography for Internet of Things: A Review," *EAI Endorsed Transactions on Internet of Things*, vol. 10, 2024, doi: 10.4108/ee-tiot.5565.
- [21] B. Usha and D. Amuthaguka, "Powering Up Security as Lightweight Crypto for Efficient IoT," in *2024 3rd International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, Institute of Electrical and Electronics Engineers Inc., 2024, doi: 10.1109/ICEE-ICT61591.2024.10718437.
- [22] S. Banik, S. Maity, and S. Chakraborty, "Creative and Modern Variants of Symmetric-Key Block Cipher Algorithms: A Novel Comprehensive Approach," Institute of Electrical and Electronics Engineers (IEEE), Jan. 2026, pp. 895–902, doi: 10.1109/icaaic64647.2025.11331181.
- [23] A. Muhamad Abdullah, "Advanced Encryption Standard (AES) Algorithm to Encrypt and Decrypt Data," *Cryptography and Network Security*, vol. 16, pp. 1–11, 2017.
- [24] M. Ozaif, S. Mustajab, and M. Alam, "Exploration of Secured Data Transmission in Internet of Things: A Survey," in *Proceedings - International Conference on Computing, Power, and Communication Technologies (IC2PCT)*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 106–112, doi: 10.1109/IC2PCT60090.2024.10486716.
- [25] R. David and P. Deepalakshmi, "Internet of Things (IoT) Devices Encryption Approaches: Enabling Secure Communication Possible in Low-Resource Settings," in *2025 Global Conference in Emerging Technology (GINOTECH)*, Institute of Electrical and Electronics Engineers Inc., 2025, doi: 10.1109/GINOTECH63460.2025.11076617.
- [26] R. Nabeel, M. Burshaid, N. Almahmood, A. Anwar, and W. Elmedany, "Secure and Energy-Efficient Data Encryption Techniques for IoT Devices in Resource-Constrained Environments," in *2025 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT)*, Institute of Electrical and Electronics Engineers Inc., 2025, doi: 10.1109/3ict68299.2025.11442190.
- [27] J. Soto-Cruz, E. Ruiz-Ibarra, J. Vázquez-Castillo, A. Espinoza-Ruiz, A. Castillo-Atoche, and J. Mass-Sanchez, "A Survey of Efficient Lightweight Cryptography for Power-Constrained Microcontrollers," *Technologies*, vol. 13, 2025, doi: 10.3390/technologies13010003.

- [28] S. H. Rafat, M. N. Jarin, T. M. Mahdee, A. S. M. Miah, N. Haque, M. M. R. Tusher, J. Shin, and I. Ahmed, "Lightweight Cryptographic Algorithm Analysis for Secure IoT Communication on ESP-32 Platforms," in *2025 IEEE International Conference on Quantum Photonics, Artificial Intelligence, and Networking (QPAIN)*, 2025, doi: 10.1109/QPAIN66474.2025.11171742.
- [29] P. K. Deb, A. Mukherjee, and S. Misra, "CEaaS: Constrained Encryption as a Service in Fog-Enabled IoT," *IEEE Internet of Things Journal*, vol. 9, no. 20, pp. 19803–19806, Oct. 2022, doi: 10.1109/JIOT.2022.3167832.
- [30] N. Purohit and S. Joshi, "Adaptive Lightweight Elliptic Curve Cryptography Framework for Power-Constrained IoT Devices," in *IC-DECON 2025 - International Conference on Data, Energy and Communication Network*, Institute of Electrical and Electronics Engineers Inc., 2025, doi: 10.1109/DE-CoN67170.2025.11447506.
- [31] V. D. Bhandarkar, S. Patra, N. Malik, A. H. M. Majid, D. J. Preshiya, and B. K. Bala, "Designing Adaptive Lightweight Encryption Protocols for Secure Multi-Device Communication in Resource-Constrained IoT Architectures," in *6th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)*, Institute of Electrical and Electronics Engineers Inc., 2026, doi: 10.1109/IC-NTE66387.2026.11437422.
- [32] A. S. Kushwaha, N. A. N. El-Anqoudy, R. P. Chaturvedi, and M. W. H. Adolfo, "IoT Edge Security: A Lightweight Multi-Layer Protocol Framework for Resource-Constrained Devices," in *Proceedings of the 2025 International Conference on Artificial Intelligence and Emerging Technology (Global AI Summit)*, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 184–189, doi: 10.1109/AISummit66170.2025.11410931.
- [33] R. Sinaga, H. D. Purnomo, I. Sembiring, T. Wellem, and A. Junaidi, "Adaptive Post Quantum Cryptography Tuning for Constrained Devices Using Kyber and Saber," in *Proceeding - 2025 4th International Conference on Creative Communication and Innovative Technology (ICCIT)*, Institute of Electrical and Electronics Engineers Inc., 2025, doi: 10.1109/ICCIT65724.2025.11166918.
- [34] F. Zeshan, Z. Dar, A. Ahmad, and T. Malik, "A Fuzzy Ontology-Based Context-Aware Encryption Approach in IoT Through Device and Information Classification," *Journal of Supercomputing*, vol. 80, no. 16, pp. 23311–23356, Nov. 2024, doi: 10.1007/s11227-024-06317-0.
- [35] S. V. Khara, J. Dave, S. Patel, M. B. Suthar, G. D. Tivari, and M. Kulkarni, "Context-Aware Machine Learning Framework for Adaptive Post-Quantum Cryptography Selection in Resource-Constrained IoT Networks," in *2026 3rd International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*, IEEE, Apr. 2026, pp. 1–6, doi: 10.1109/RMKMATE69073.2026.11518690.
- [36] M. J. Alanazi, R. A. Alhoweiti, G. A. Alhwaity, and A. R. Alharbi, "An Adaptive Hybrid Cryptographic Framework for Resource-Constrained IoT Devices," *Electronics (Switzerland)*, vol. 14, no. 23, Dec. 2025, doi: 10.3390/electronics14234666.
- [37] K. Gowda, "Adaptive Multi-Layer Encryption: A Context-Aware Framework for Enhanced Multi-Cloud Data Security," *Xi'an Shiyou Daxue Xuebao (Ziran Kexue Ban) / Journal of Xi'an Shiyou University*, vol. 68, p. 19, 2025, doi: 10.5281/zenodo.15543642.
- [38] B. Amoah, X. Wang, J. Zhang, S. Mao, S. C. G. Periaswamy, and J. Patton, "DCA-KEAE: A Dynamic Context-Aware Key Exchange and Adaptive Encryption Scheme for Secure RFID Systems," in *IEEE International Conference on Communications*, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 710–715, doi: 10.1109/ICC52391.2025.11161545.
- [39] M. G. Spina and F. De Rango, "Lightweight, Dynamic and Energy Efficient Security Mechanism for Constrained IoT Devices Using CoAP," in *Proceedings - IEEE Consumer Communications and Networking Conference (CCNC)*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 1123–1128, doi: 10.1109/CCNC51644.2023.10059854.

- [40] S. A. Olamaei, M. B. Jezehei, and A. Broumandnia, "Achieving Enhanced Efficiency and Security in IoT: A Hybrid Encryption Approach with ECC and Blowfish," *International Journal of Engineering, Transactions B: Applications*, vol. 39, no. 5, pp. 1238–1253, May 2026, doi: 10.5829/ije.2026.39.05b.16.
- [41] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A Design Science Research Methodology for Information Systems Research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/MIS0742-1222240302.
- [42] B. A. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," Keele University and University of Durham, EBSE Technical Report EBSE-2007-01, 2007.
- [43] B. Kitchenham, "Procedures for Performing Systematic Reviews," Keele University, Keele, UK, Technical Report TR/SE-0401, 2004.
- [44] J. Hughes and J. N. Roge, "Long-Term Security Vulnerabilities of Encrypted Data," *Issues in Information Systems*, vol. 8, no. 2, pp. 522–528, 2007, doi: 10.48009/2_iis_2007_522-528.
- [45] V. Mothukuri and R. M. Parizi, "Securing Cryptography in the Age of Quantum Computing and AI: Threats, Implementations, and Strategic Response," arXiv:2603.06969, Mar. 2026.
- [46] J. Zheng and A. S. Namin, "A survey on the moving target defense strategies: An architectural perspective," *Journal of Computer Science and Technology*, vol. 34, no. 1, pp. 207–233, Jan. 2019, doi: 10.1007/s11390-019-1906-z.
- [47] H. Tang, Q. T. Sun, X. Yang, and K. Long, "A network coding and DES based dynamic encryption scheme for moving target defense," *IEEE Access*, vol. 6, pp. 26059–26068, 2018, doi: 10.1109/ACCESS.2018.2832854.

APPENDIX A

Single-Cipher Benchmark Results

A.1. Encryption and Decryption Time

Figure A.1 presents the encryption and decryption times, measured in milliseconds on a logarithmic scale, for AES-128, AES-192, AES-256, ChaCha20, HIGHT, RECTANGLE, and SPECK across payload sizes ranging from 1 KB to 50 MB.

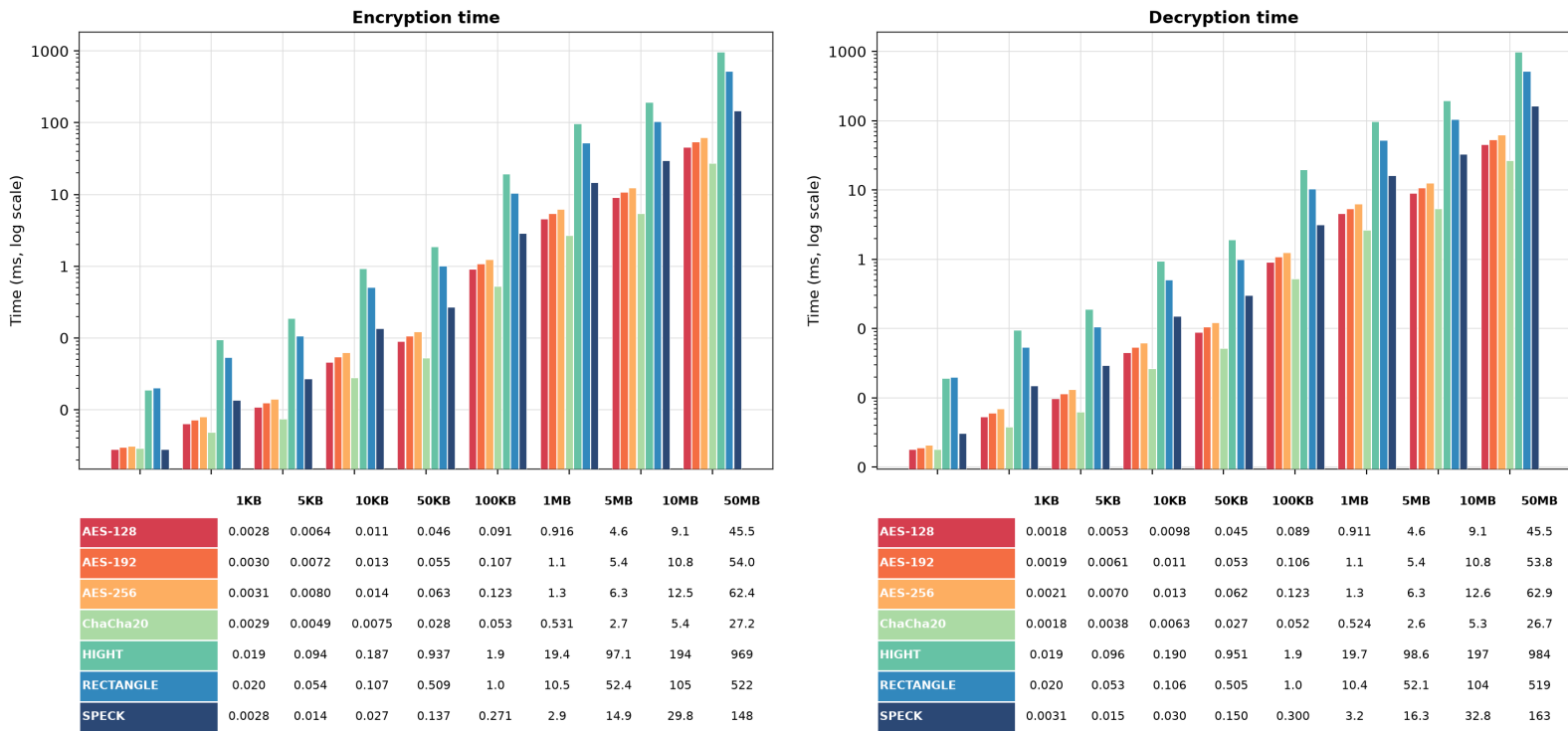


FIGURE A.1. Encryption and decryption time (ms, log scale) by algorithm and payload size.

A.2. Throughput

Figure A.2 presents the corresponding encryption and decryption throughput, measured in Mbps on a logarithmic scale, for the same set of algorithms and payload sizes.

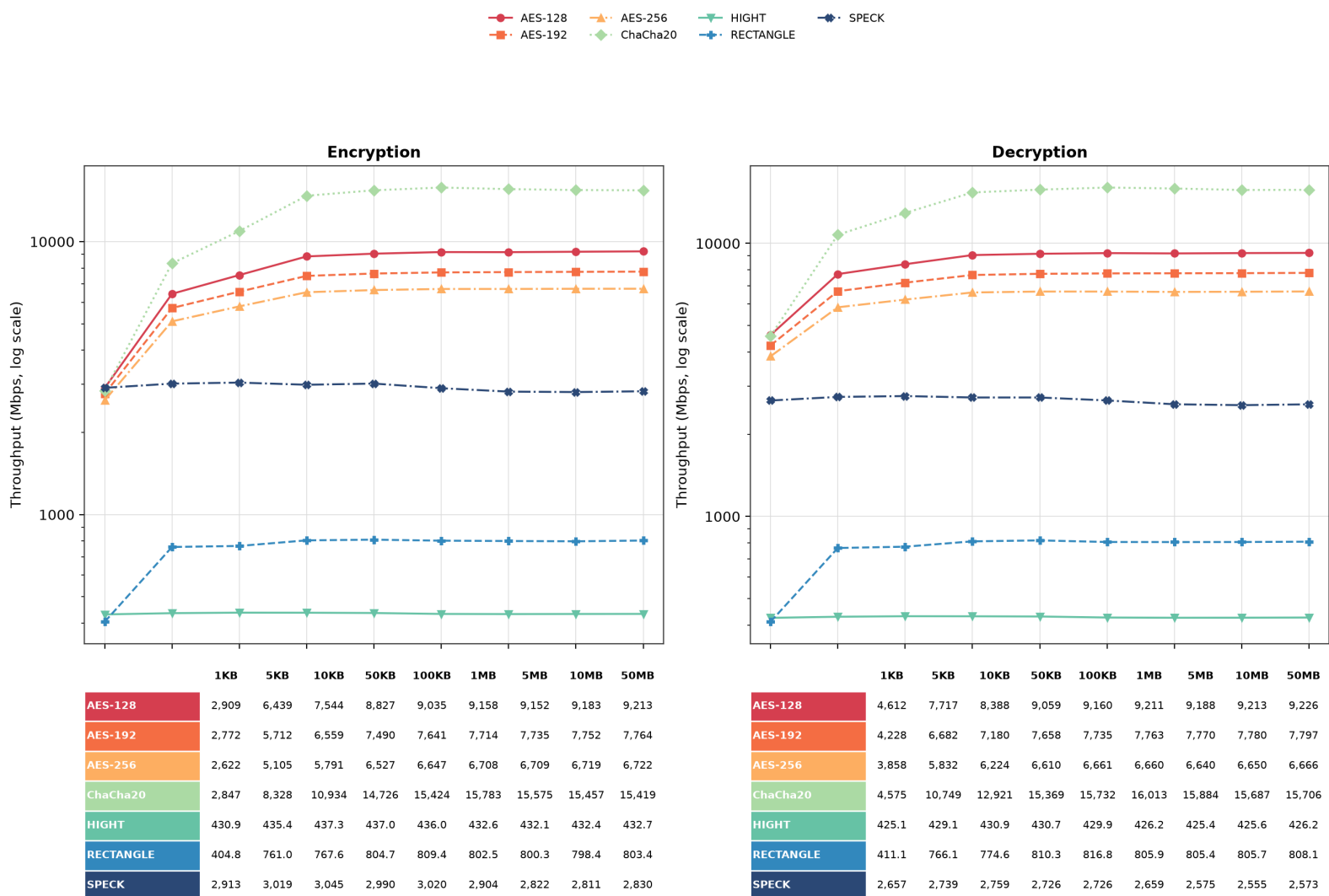


FIGURE A.2. Encryption and decryption throughput (Mbps, log scale) by algorithm and payload size.

APPENDIX B

Cascade Benchmark Results

B.1. Encryption and Decryption Time

Figure B.1 presents the encryption and decryption times, measured in milliseconds on a logarithmic scale, for the seven two-cipher cascade combinations (AES-128+HIGHT, AES-128+SPECK, AES-256+AES-128, AES-256+ChaCha20, ChaCha20+SPECK, RECTANGLE+HIGHT, and SPECK+HIGHT) across payload sizes ranging from 1 KB to 50 MB.

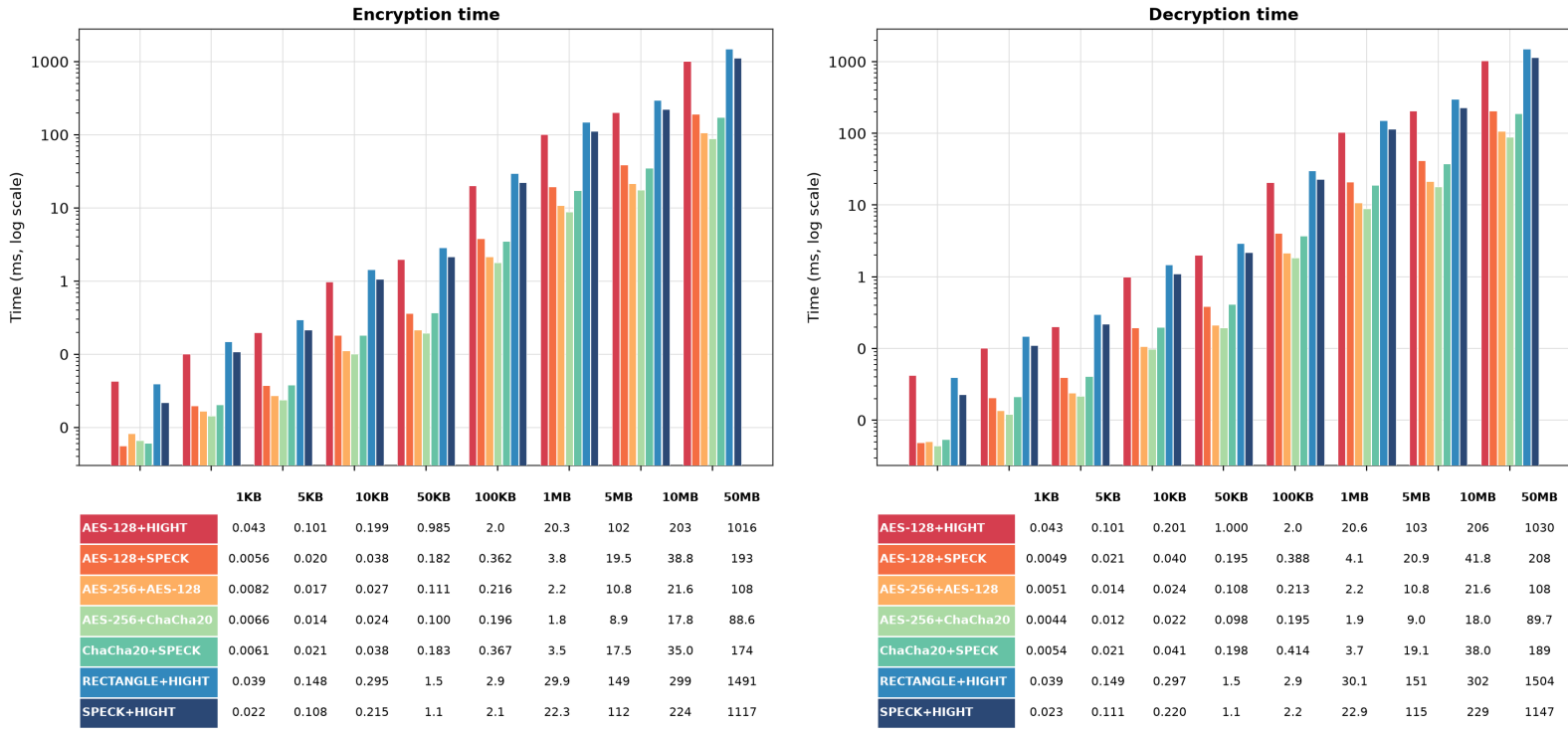


FIGURE B.1. Encryption and decryption time (ms, log scale) by two-cipher cascade combination and payload size.

Figure B.2 presents the equivalent measurements for the three-cipher cascade AES-256+ChaCha20+AES-128, evaluated across the same range of payload sizes.

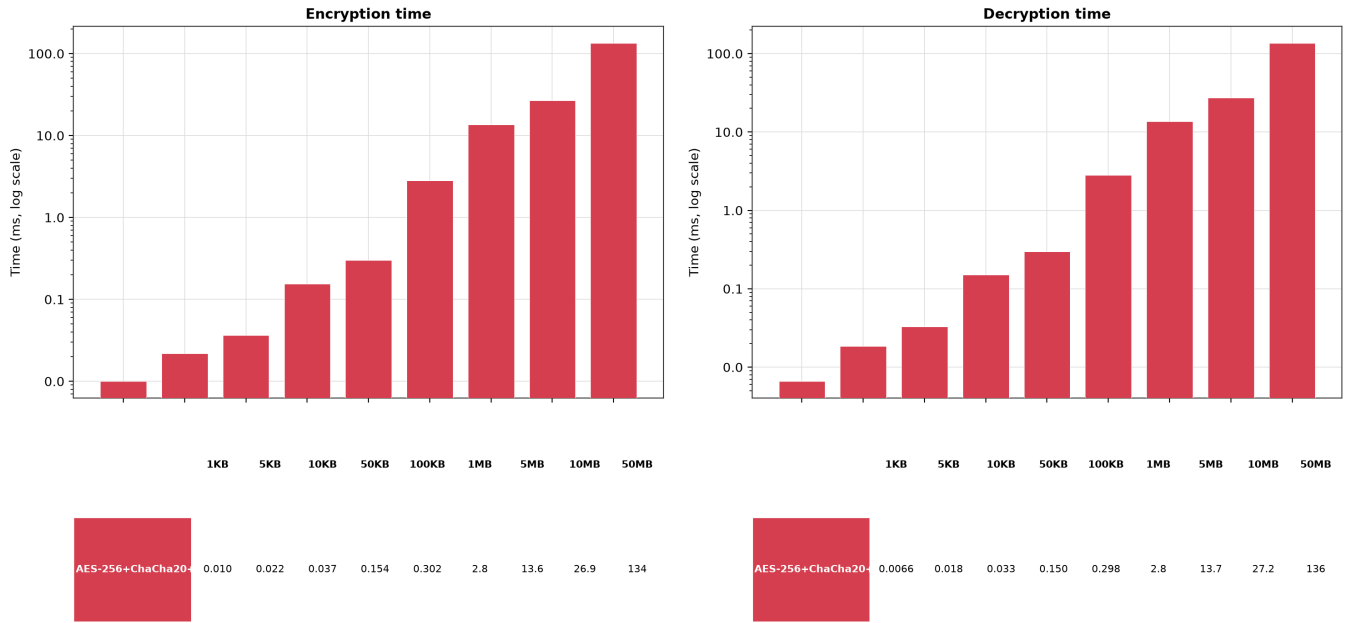


FIGURE B.2. Encryption and decryption time (ms, log scale) for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.

B.2. Throughput

Figure B.3 presents the corresponding encryption and decryption throughput, measured in Mbps on a logarithmic scale, for the same two-cipher cascade combinations and payload sizes.

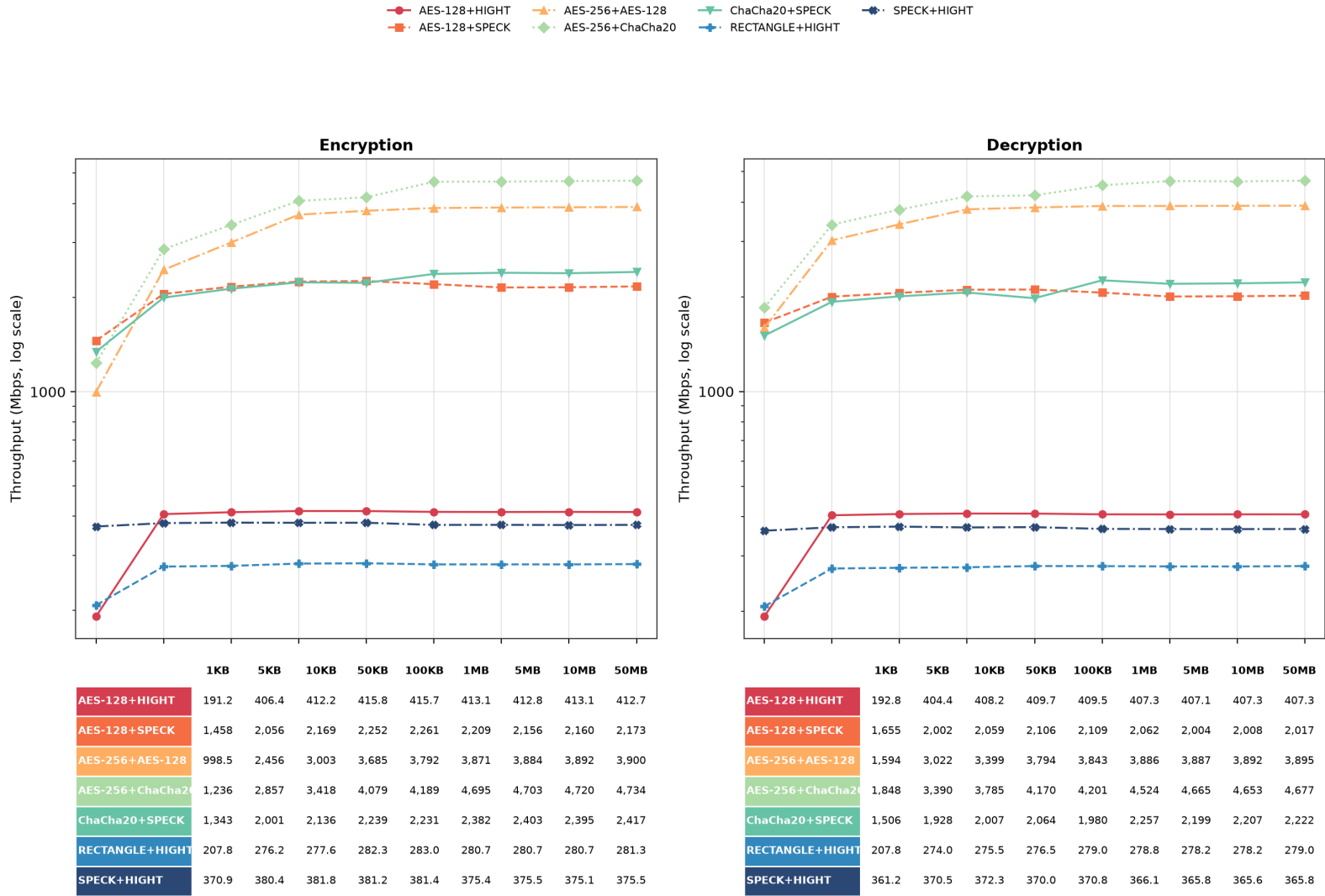


FIGURE B.3. Encryption and decryption throughput (Mbps, log scale) by two-cipher cascade combination and payload size.

Figure B.4 presents the corresponding throughput for the three-cipher cascade AES-256+ChaCha20+AES-128.

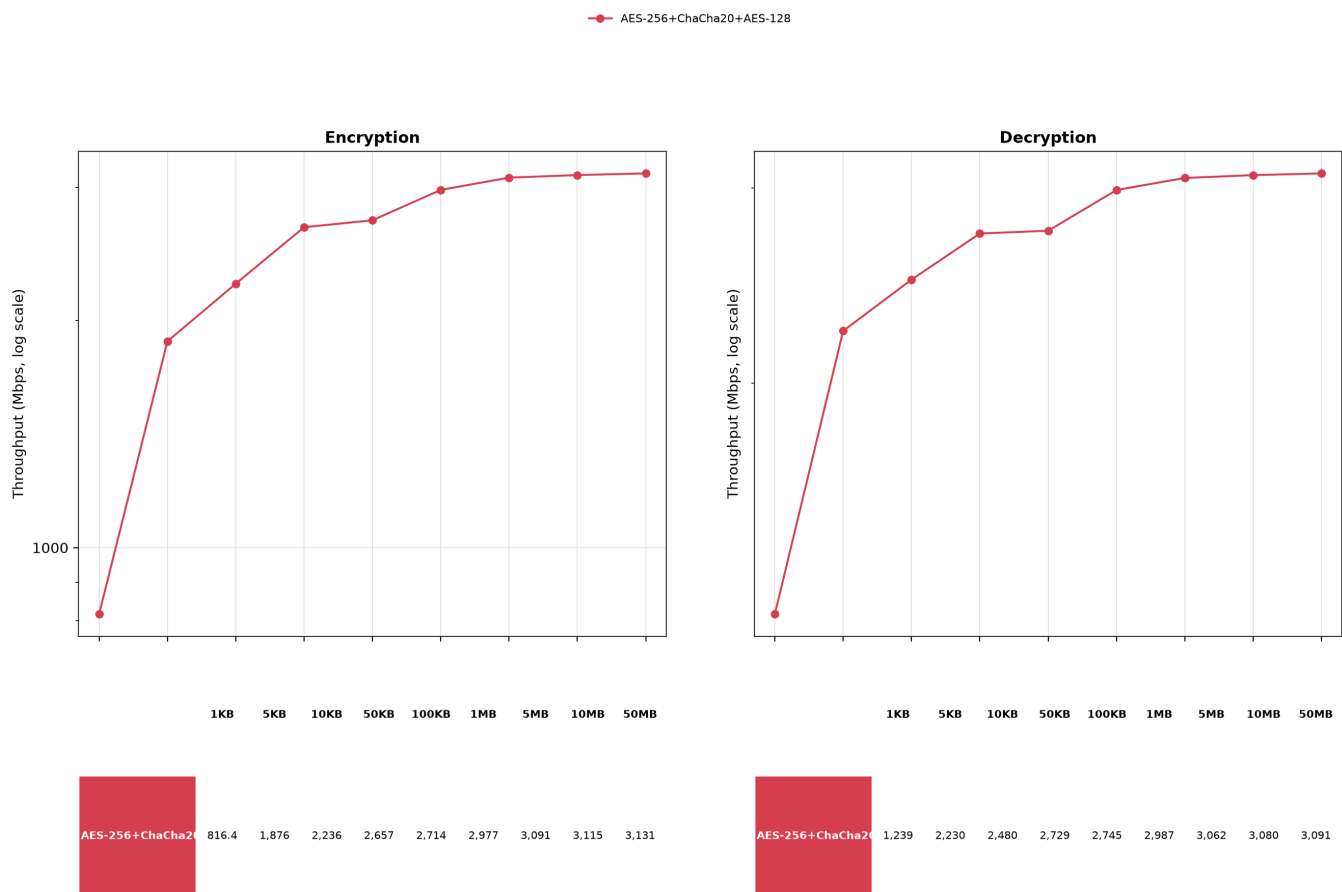


FIGURE B.4. Encryption and decryption throughput (Mbps, log scale) for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.

APPENDIX C

Impact of Hardware Acceleration

This appendix presents the baseline throughput of each cipher and cipher combination with hardware acceleration disabled, together with the percentage throughput gain obtained when AES-NI, SIMD-accelerated ChaCha20, or both are enabled, across payload sizes ranging from 1 KB to 50 MB.

Figure C.1 presents the baseline throughput and acceleration gain for the solo ciphers AES-128, AES-192, AES-256, ChaCha20, and RECTANGLE.

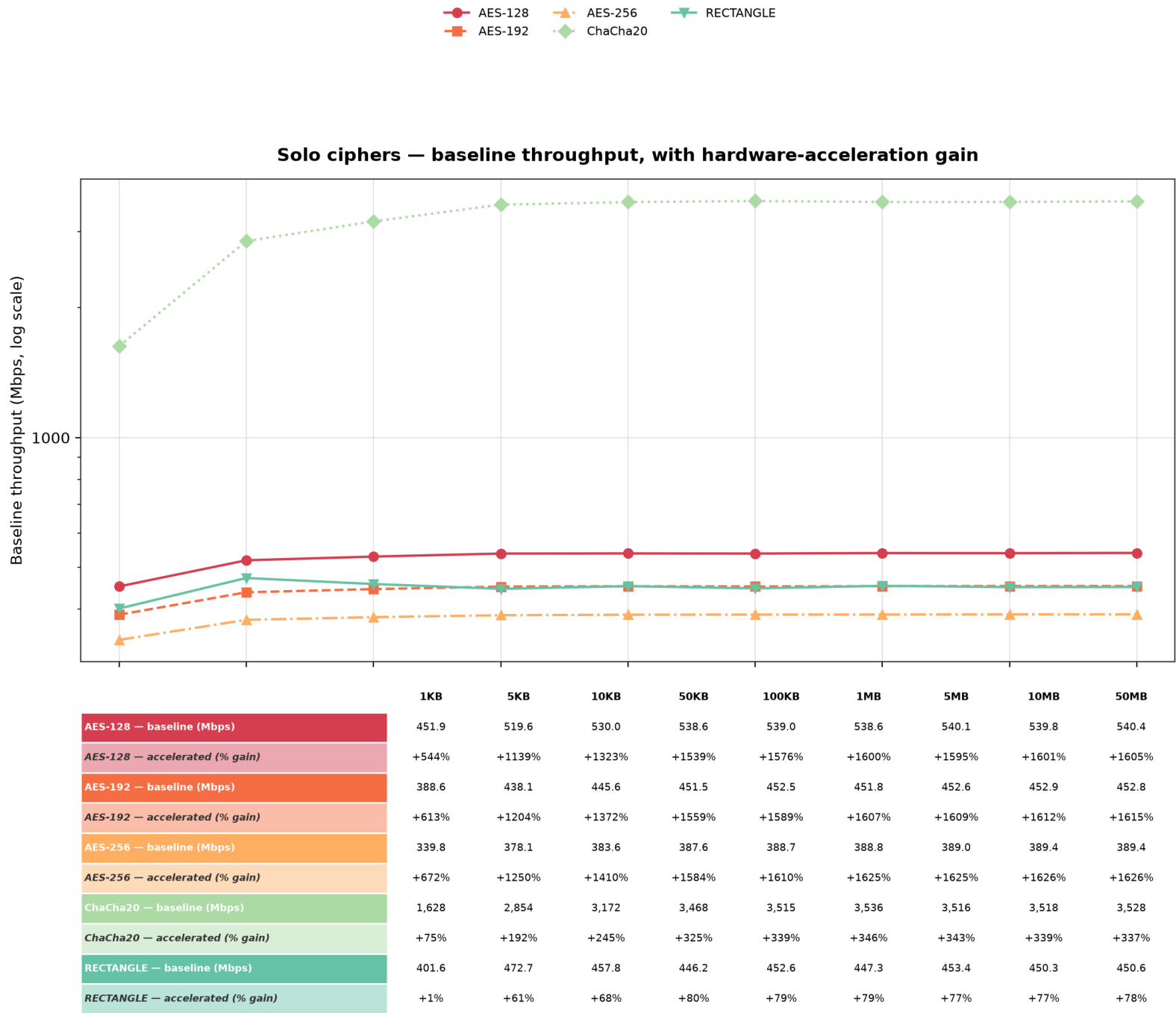


FIGURE C.1. Baseline throughput (Mbps, log scale) and hardware-acceleration gain (%) for solo ciphers, by payload size.

Figure C.2 presents the corresponding results for the lightweight-cipher pairings RECTANGLE+HIGHT, ChaCha20+SPECK, AES-128+HIGHT, AES-128+SPECK, and AES-256+AES-128.

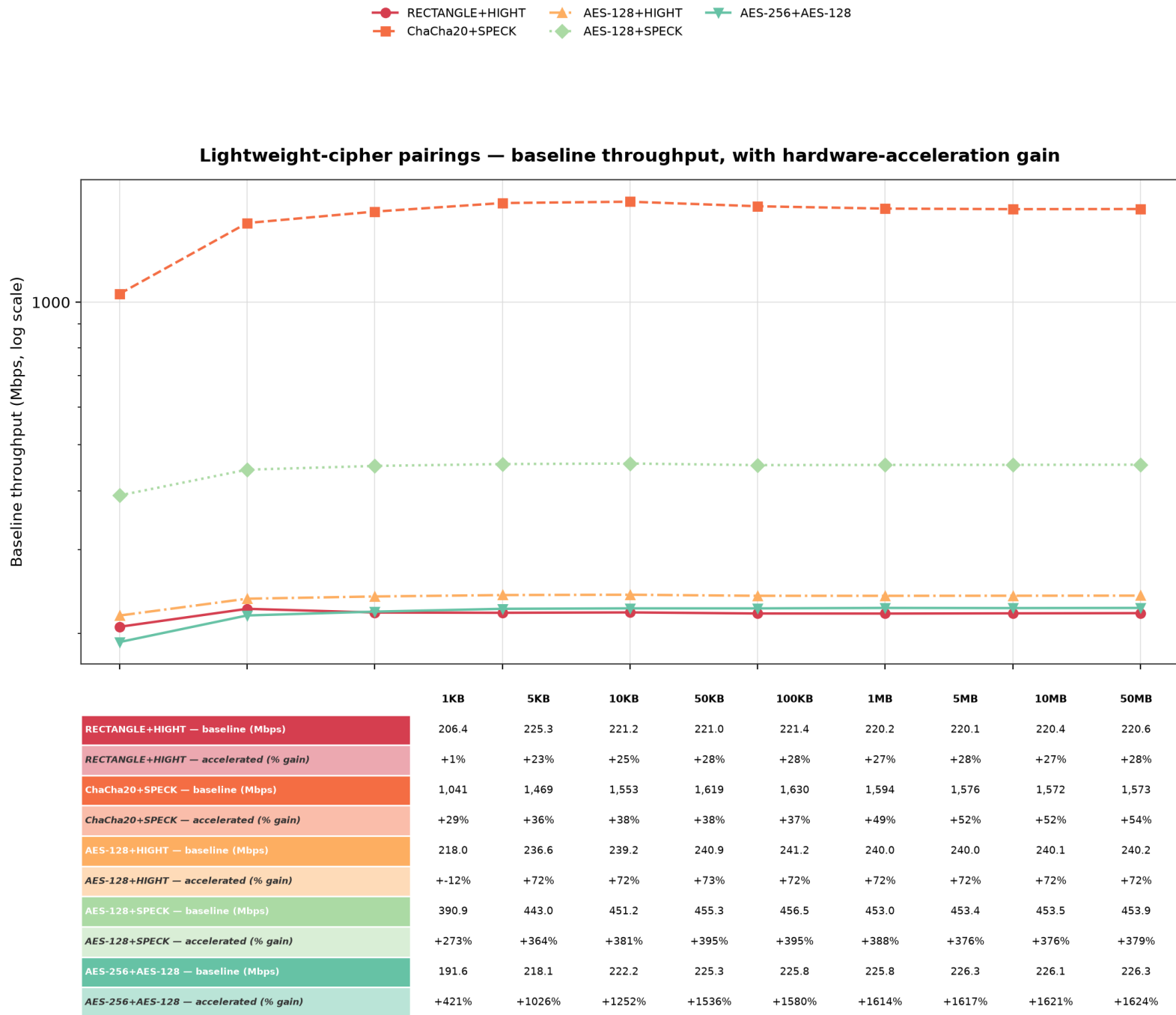


FIGURE C.2. Baseline throughput (Mbps, log scale) and hardware-acceleration gain (%) for lightweight-cipher pairings, by payload size.

Figure C.3 presents the baseline throughput and per-state acceleration gain (AES-NI only, ChaCha SIMD only, and both enabled) for the two-cipher cascade AES-256+ChaCha20.

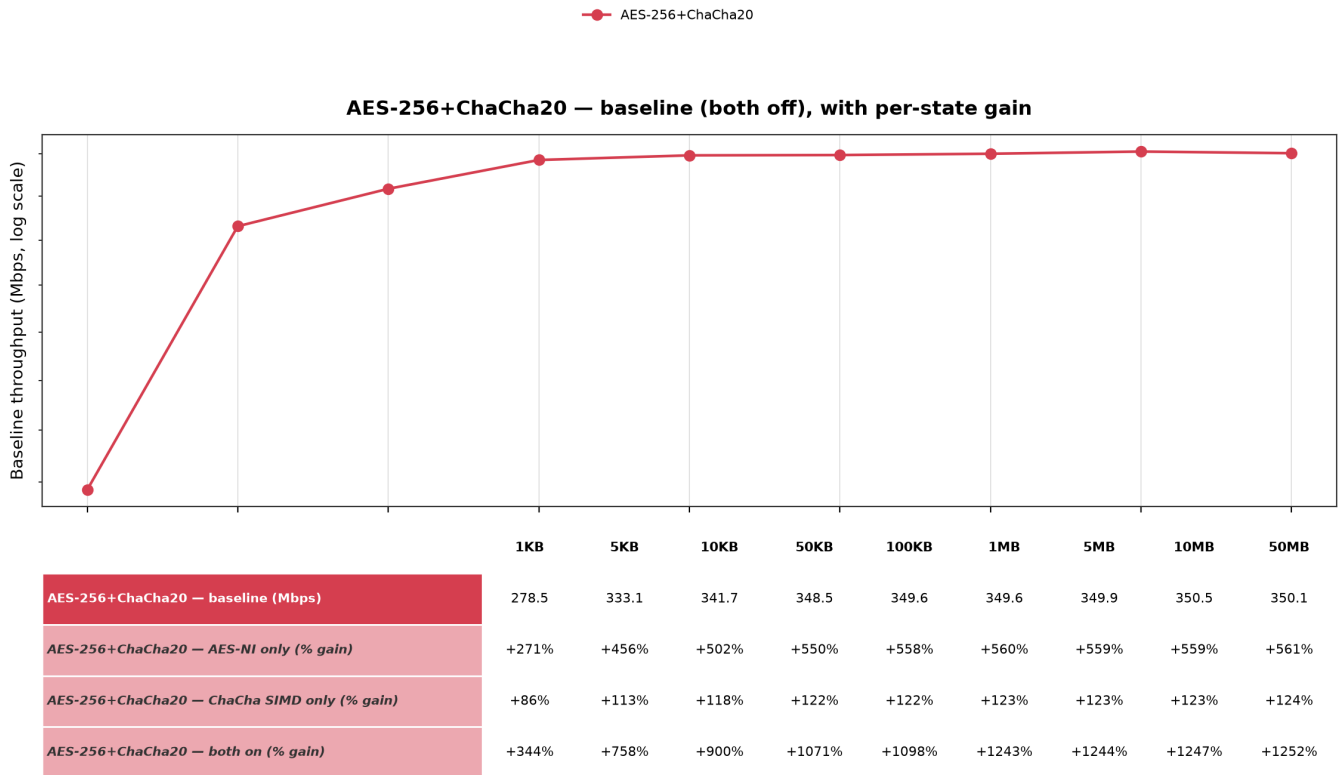


FIGURE C.3. Baseline throughput (Mbps, log scale) and per-state hardware-acceleration gain (%) for AES-256+ChaCha20, by payload size.

Figure C.4 presents the equivalent results for the three-cipher cascade AES-256+ChaCha20+AES-128.

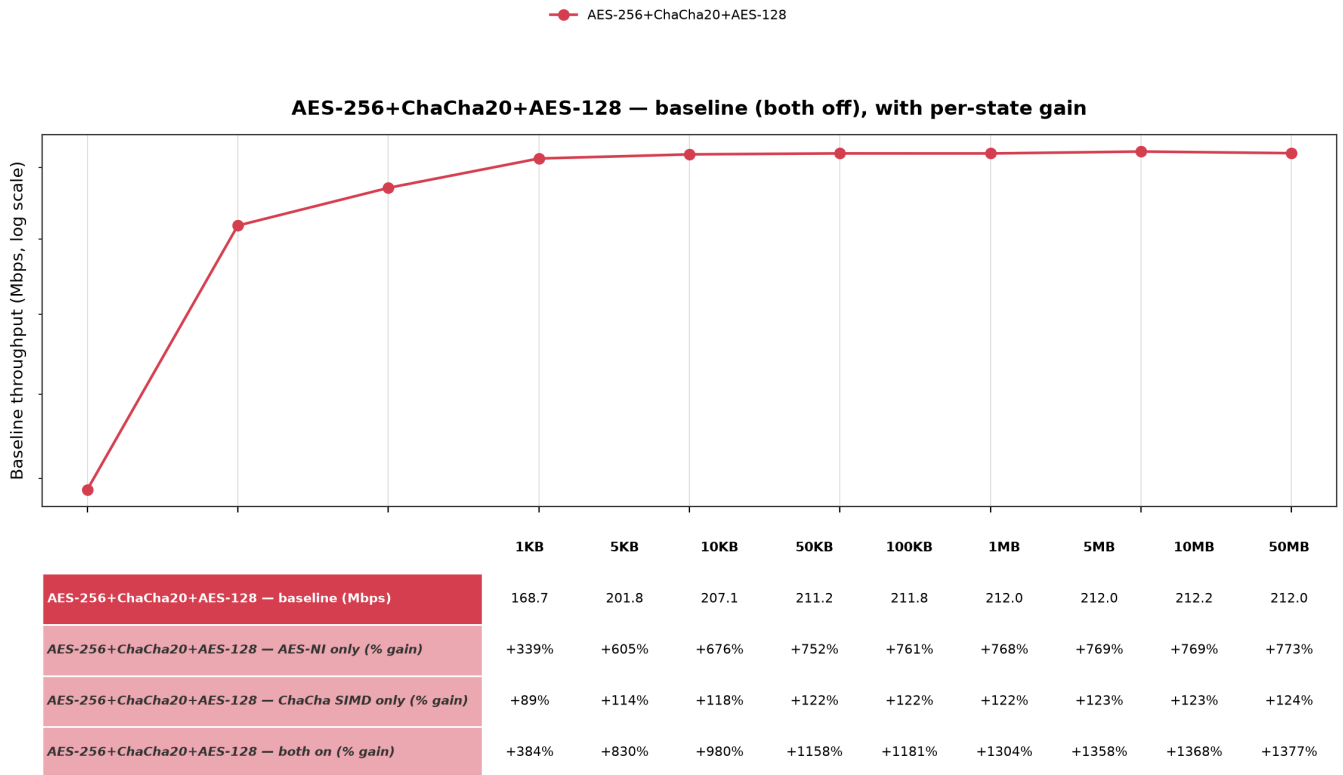


FIGURE C.4. Baseline throughput (Mbps, log scale) and per-state hardware-acceleration gain (%) for AES-256+ChaCha20+AES-128, by payload size.

APPENDIX D

Setup and ECC Key-Exchange Cost

This appendix presents the share of total processing time attributable to the ECC key-exchange handshake when performed before encryption. The reported percentage is calculated as the ECC handshake time divided by the combined ECC handshake and encryption time. Results are shown across payload sizes ranging from 1 KB to 50 MB. The vertical axis uses a logarithmic scale.

D.1. Single Ciphers

Figure D.1 presents the ECC handshake share of the total time for AES-128, AES-256, ChaCha20, HIGHT, RECTANGLE, and SPECK.

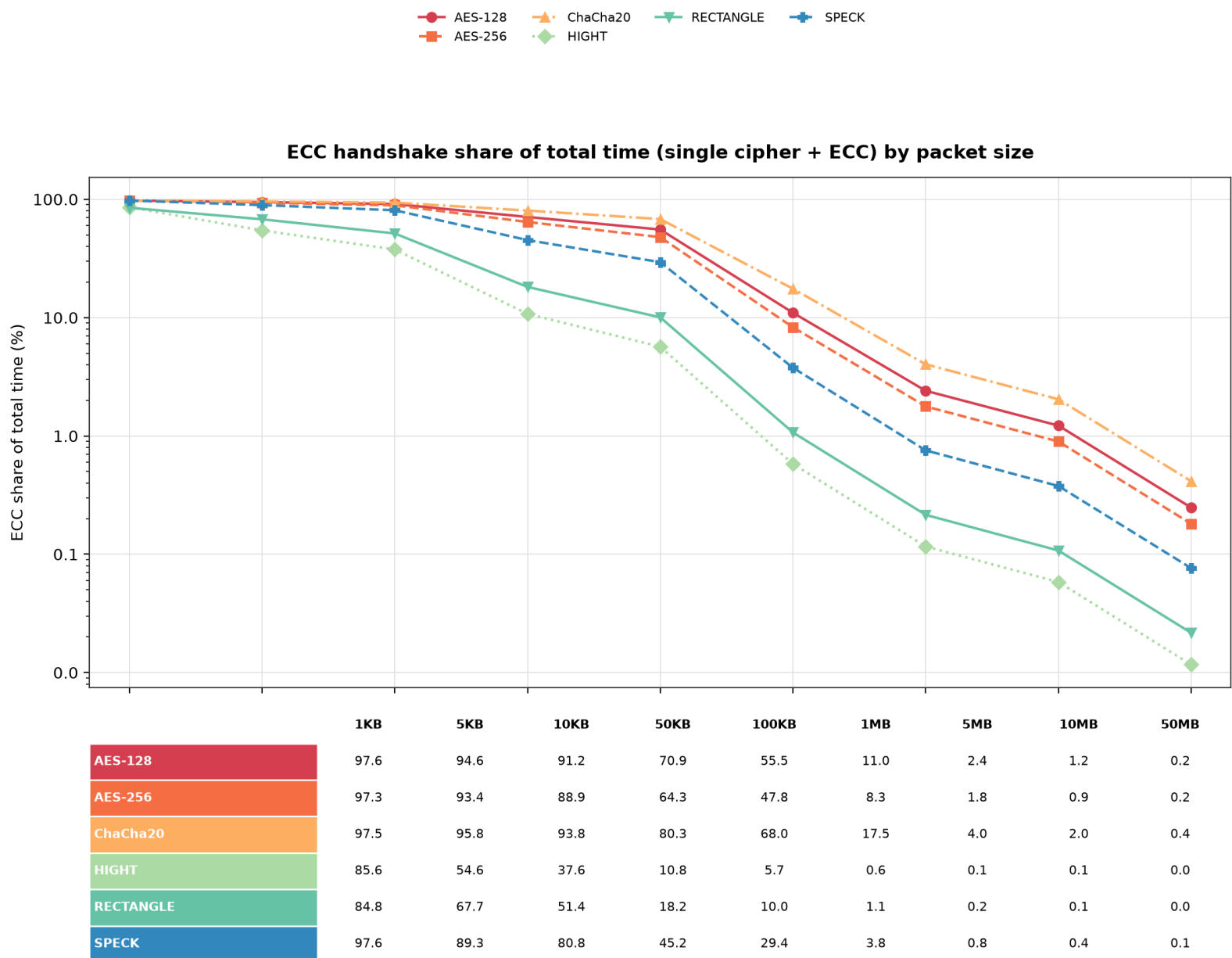


FIGURE D.1. ECC handshake share of total time (% , log scale) for single ciphers by payload size.

D.2. Two-Cipher Cascades

Figure D.2 presents the ECC handshake share of the total time for the selected two-cipher cascades AES-128+SPECK, AES-256+AES-128, AES-256+ChaCha20, ChaCha20+SPECK, and SPECK+HIGHT.

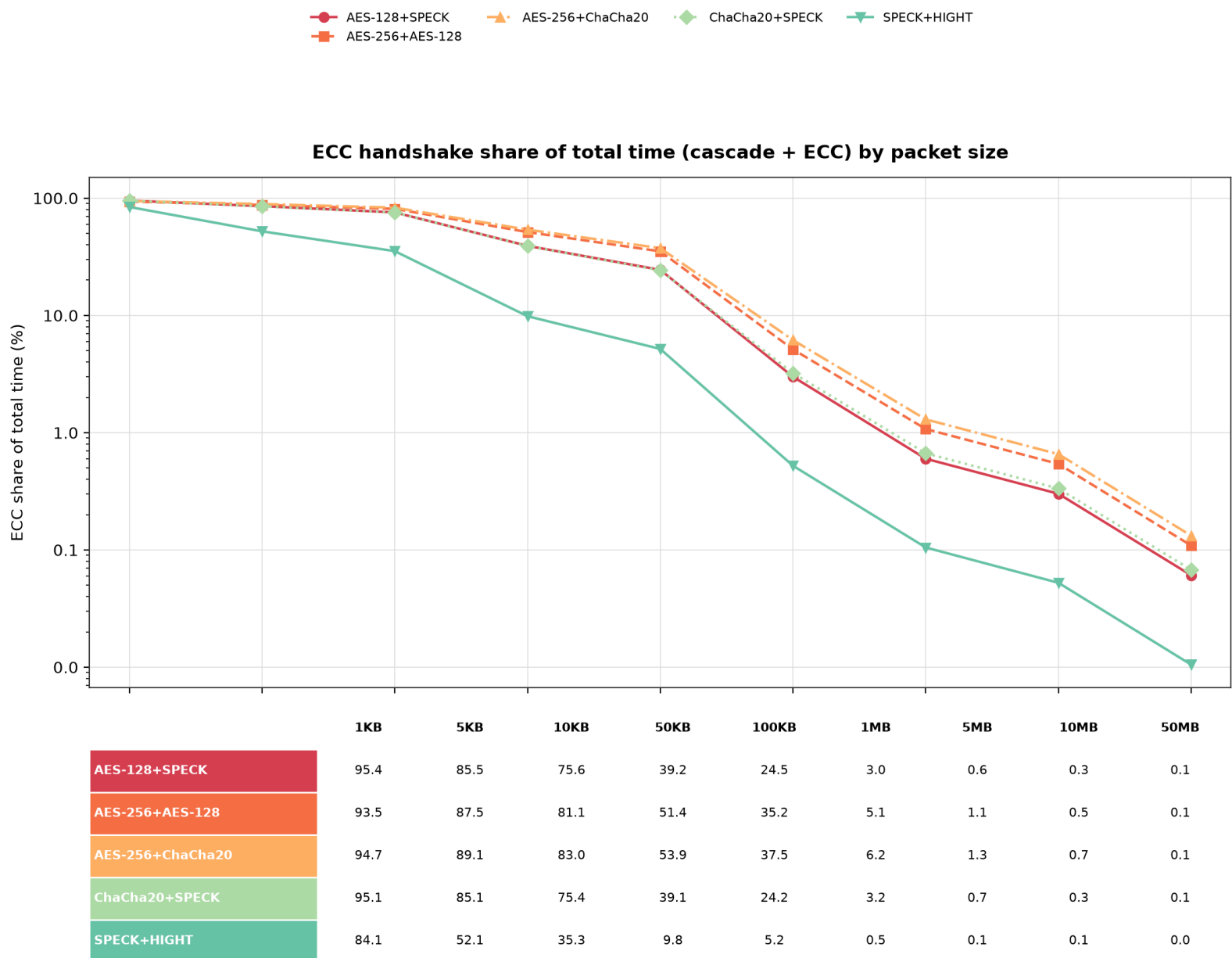


FIGURE D.2. ECC handshake share of total time (% , log scale) for two-cipher cascades by payload size.

D.3. Three-Cipher Cascade

Figure D.3 presents the ECC handshake share of the total time for the three-cipher cascade AES-256+ChaCha20+AES-128.

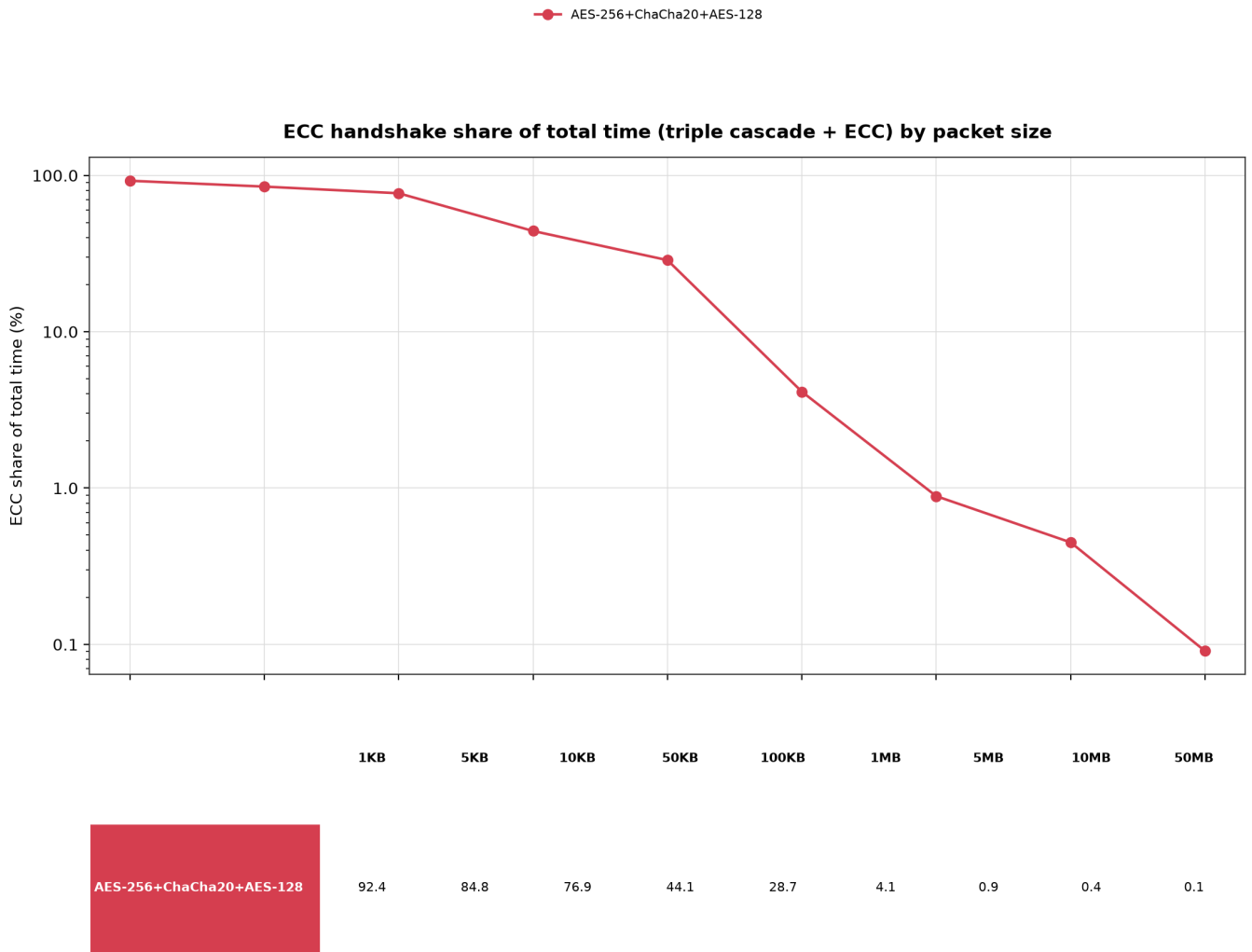


FIGURE D.3. ECC handshake share of total time (% , log scale) for the three-cipher cascade AES-256+ChaCha20+AES-128, by payload size.

D.4. Hardware-Acceleration Configurations

Figure D.4 presents the ECC handshake share of the total time for selected individual ciphers and cascades under different hardware-acceleration configurations. The configurations include AES-NI enabled or disabled, ChaCha20 SIMD enabled or disabled, and combinations in which both mechanisms are disabled.

AES-128
 AES-128+SPECK
 AES-256
 AES-256+AES-128
 AES-256+ChaCha20 (AES-NI on)

AES-256+ChaCha20 (ChaCha-SIMD on)
 AES-256+ChaCha20 (both off)
 AES-256+ChaCha20+AES-128 (AES-NI on)
 AES-256+ChaCha20+AES-128 (ChaCha-SIMD on)

AES-256+ChaCha20+AES-128 (both off)
 ChaCha20
 ChaCha20+SPECK
 RECTANGLE

ECC handshake share of total time — selected entities (no/partial hardware acceleration)

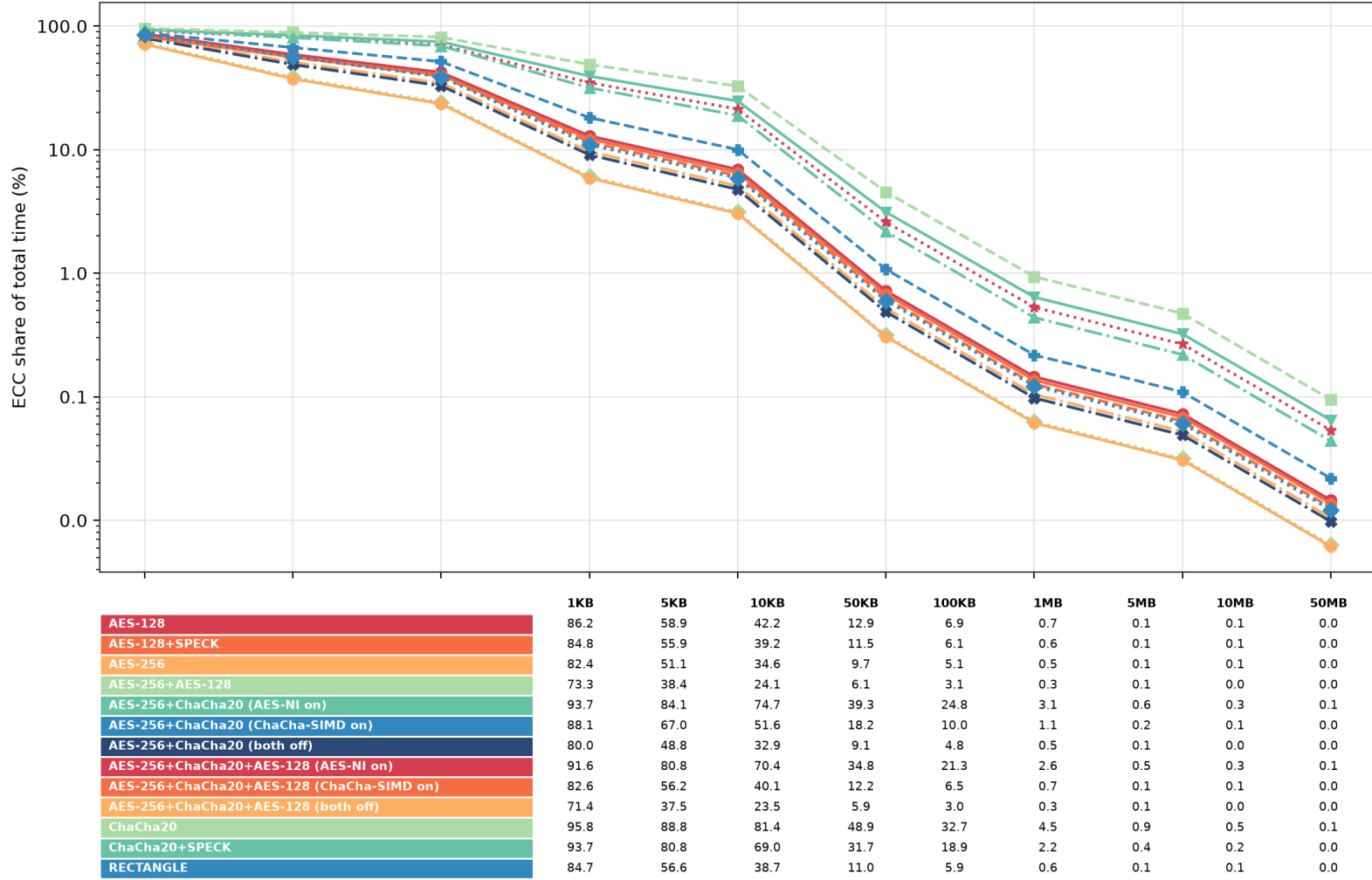


FIGURE D.4. ECC handshake share of total time (% , log scale) for selected ciphers and cascades under different hardware-acceleration configurations, by payload size.

APPENDIX E

UML Diagrams

This appendix presents the Use Case, Activity, and Sequence diagrams of the PRISEC IV prototype, discussed in Section 5.2.

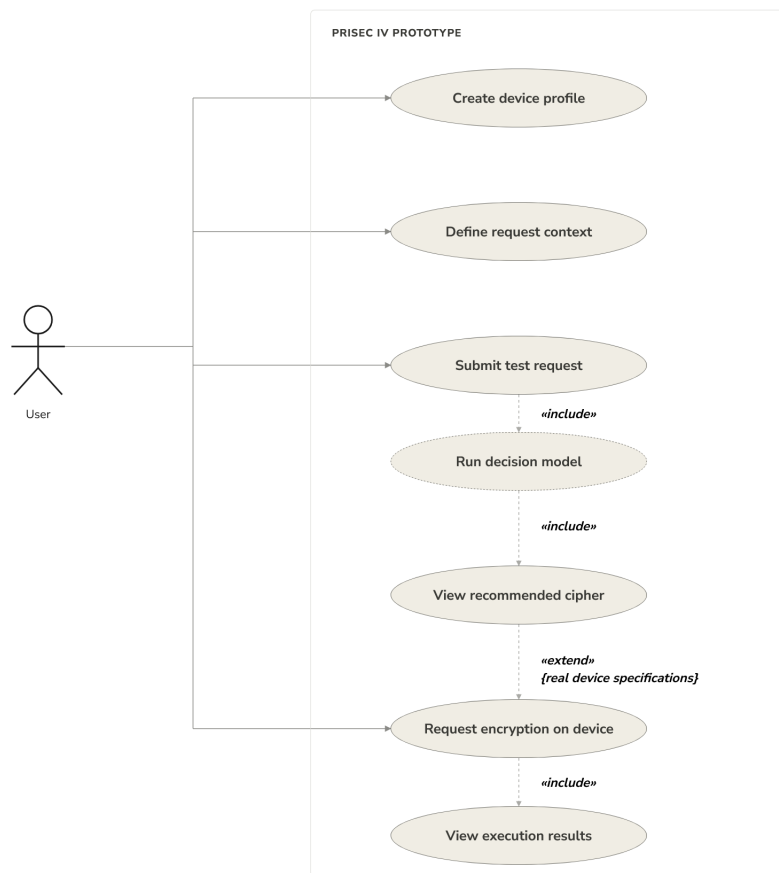


FIGURE E.1. Use case diagram of the PRISEC IV prototype.

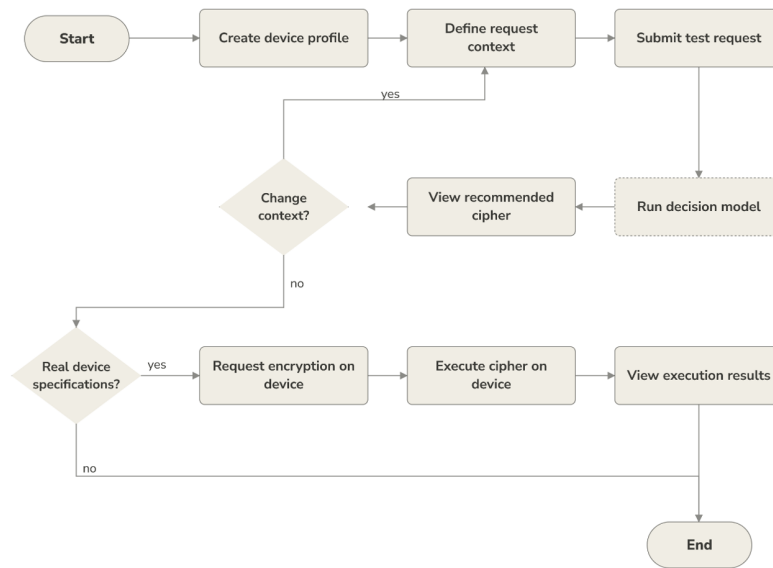


FIGURE E.2. Activity diagram of the PRISEC IV prototype.

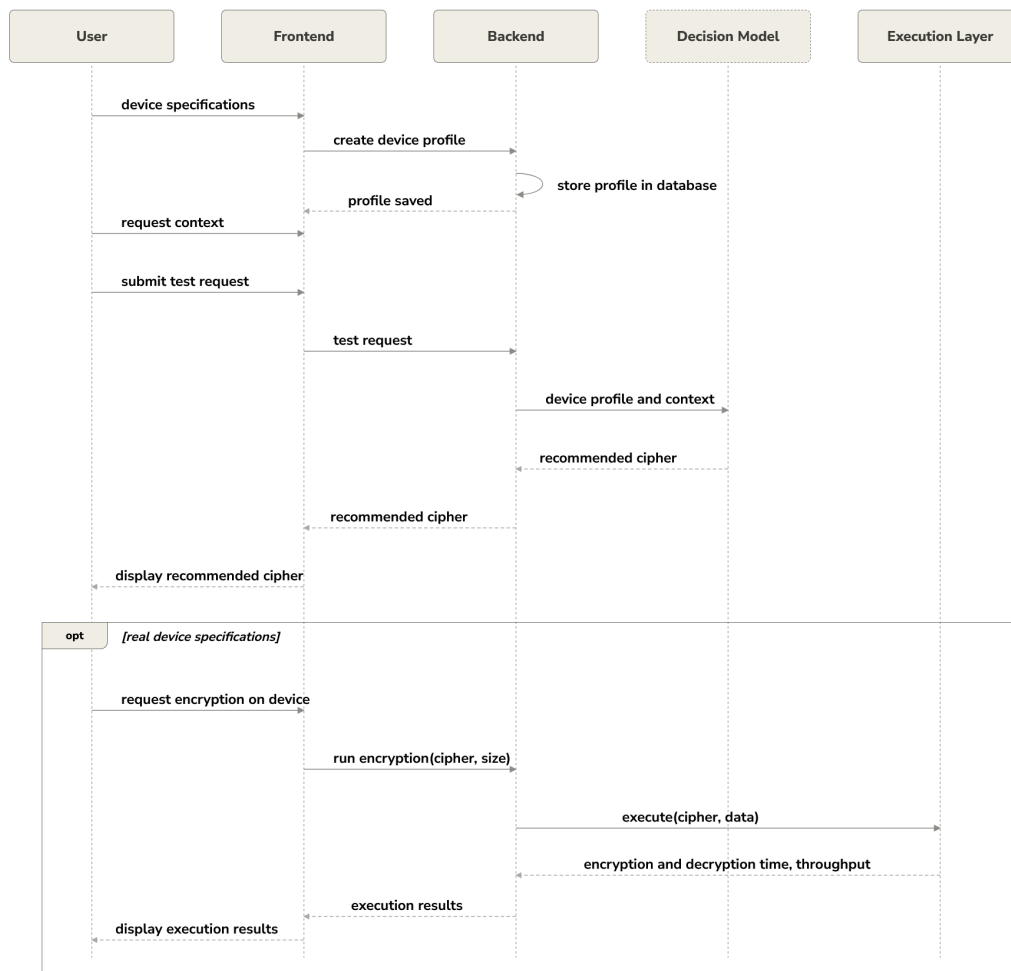


FIGURE E.3. Sequence diagram of the PRISEC IV prototype.

APPENDIX F

Device Fit Weighting Scheme

Table F.1 presents the weights assigned to memory fit, word fit, and energy fit when computing device fit for a battery-powered device, as a function of the request’s duty cycle. Memory fit and word fit are always weighted equally to each other, while energy fit’s weight grows as duty cycle becomes more continuous, drawing that additional weight from memory fit and word fit in equal measure.

TABLE F.1. Device fit weights by duty cycle.

Duty Cycle	w_{mem}	w_{word}	w_{energy}
Sporadic	0.45	0.45	0.10
Periodic	0.40	0.40	0.20
Continuous	0.35	0.35	0.30

APPENDIX G

Security Fit Value Scales and Cipher Ranking

G.1. Value Scales

Table G.1 presents the score assigned to each security level value, and Table G.2 presents the shared score scale used for both data confidentiality and data lifetime, as used in the requirement calculation in Section 5.4.2.

TABLE G.1. Security level value scale.

Security Level	Score
Guest	0.25
Basic	0.5
Advanced	0.75
Admin	1.0

TABLE G.2. Data confidentiality and data lifetime value scale.

Confidentiality / Data Lifetime	Score
Low / Short-term	0.2
Medium / Medium-term	0.6
High / Long-term	1.0

G.2. Security Strength Ranking

Table G.3 presents the full `security_strength` ranking used to evaluate every candidate cipher and cascade in the catalogue, comprising 23 distinct ranks across 27 ciphers and cascades.

TABLE G.3. Security strength ranking, by cipher and cascade.

Rank	Cipher / Cascade	Score
1	ECC + AES-256 → ChaCha20 → AES-128	1.00
2	ECC + AES-256 → ChaCha20	0.97
3	ECC + AES-256 → AES-128	0.96
4	AES-256 → ChaCha20 → AES-128	0.95
5	ECC + AES-256	0.92
5	AES-256 → ChaCha20	0.92
5	ECC + ChaCha20 → SPECK	0.92

TABLE G.3. Security strength ranking, by cipher and cascade
(continued).

Rank	Cipher / Cascade	Score
6	AES-256 $\rightarrow$ AES-128	0.91
7	ECC + ChaCha20	0.89
8	AES-256	0.87
8	ChaCha20 $\rightarrow$ SPECK	0.87
9	ChaCha20	0.84
10	ECC + AES-128 $\rightarrow$ SPECK	0.73
11	AES-192	0.71
12	ECC + AES-128	0.68
12	AES-128 $\rightarrow$ SPECK	0.68
13	AES-128 $\rightarrow$ HIGHT	0.66
14	AES-128	0.63
15	ECC + SPECK $\rightarrow$ HIGHT	0.51
16	ECC + SPECK	0.48
17	SPECK $\rightarrow$ HIGHT	0.46
18	SPECK	0.43
19	ECC + RECTANGLE	0.40
20	RECTANGLE $\rightarrow$ HIGHT	0.38
21	RECTANGLE	0.35
22	ECC + HIGHT	0.31
23	HIGHT	0.26

APPENDIX H

Application Fit Value Scales and Weights

H.1. Value Scales

Table H.1 presents the score assigned to each latency tolerance value, and Table H.2 presents the score assigned to each throughput required value, as used in the throughput and latency fit calculations in Section 5.4.3.

TABLE H.1. Latency tolerance value scale.

Latency Tolerance	Score
High	0.4
Medium	0.7
Low	1.0

TABLE H.2. Throughput required value scale.

Throughput Required	Score
Low	0.25
Medium	0.5
High	0.75
Very High	1.0

H.2. Duty Cycle Base Weights

Table H.3 presents the base weights assigned to setup fit, throughput fit, and latency fit as a function of the request’s duty cycle, before setup’s weight is scaled down by the amortization factor as described in Section 5.4.3.

TABLE H.3. Application fit base weights by duty cycle.

Duty Cycle	w_{setup}	$w_{\text{throughput}}$	w_{latency}
Sporadic	0.33	0.33	0.33
Periodic	0.20	0.40	0.40
Continuous	0.00	0.50	0.50

APPENDIX I

Deeper Analysis Table

This appendix presents the full Deeper Analysis table discussed in Section 5.6.2, showing every candidate cipher and cascade's complete score breakdown: its `device_fit` score and `memory_fit/word_fit/energy_fit` sub-scores, its `security_fit` score and the `security_strength/requirement` pair behind it, and its `application_fit` score and `throughput_fit/latency_fit/setup_fit/amortization_factor` together.

Cipher	Final	Device Fit			energy	Security Fit			Application Fit					
		score	mem	word		score	strength	requir.	score	throughput	latency	setup	amort.	w(setup)
★ ECC+ChaCha20	0.8058	1.000	1.000	1.000	—	0.667	0.890	1.000	0.750	0.500	1.000	0.971	0.029	0.000
ChaCha20	0.7933	1.000	1.000	1.000	—	0.630	0.840	1.000	0.750	0.500	1.000	1.000	0.000	0.000
ECC+AES-256+ChaCha20+AES-128	0.7930	1.000	0.999	1.000	—	1.000	1.000	1.000	0.379	0.139	0.620	0.994	0.006	0.000
ECC+AES-256+ChaCha20	0.7345	1.000	1.000	1.000	—	0.728	0.970	1.000	0.476	0.218	0.734	0.991	0.009	0.000
ECC+AES-256	0.7317	1.000	1.000	1.000	—	0.690	0.920	1.000	0.505	0.317	0.694	0.987	0.013	0.000
AES-256+ChaCha20	0.7220	1.000	1.000	1.000	—	0.690	0.920	1.000	0.476	0.218	0.734	1.000	0.000	0.000
AES-256	0.7192	1.000	1.000	1.000	—	0.652	0.870	1.000	0.505	0.317	0.694	1.000	0.000	0.000
ECC+AES-256+AES-128	0.7108	1.000	1.000	1.000	—	0.720	0.960	1.000	0.413	0.178	0.648	0.992	0.008	0.000
ECC+AES-128	0.7050	1.000	1.000	1.000	—	0.510	0.680	1.000	0.605	0.500	0.710	0.983	0.017	0.000
AES-256+AES-128	0.6983	1.000	1.000	1.000	—	0.682	0.910	1.000	0.413	0.178	0.648	1.000	0.000	0.000
AES-256+ChaCha20+AES-128	0.6972	1.000	0.999	1.000	—	0.712	0.950	1.000	0.379	0.139	0.620	1.000	0.000	0.000
AES-128	0.6925	1.000	1.000	1.000	—	0.473	0.630	1.000	0.605	0.500	0.710	1.000	0.000	0.000
AES-192	0.6890	1.000	1.000	1.000	—	0.532	0.710	1.000	0.535	0.367	0.702	1.000	0.000	0.000
ECC+ChaCha20+SPECK	0.6773	1.000	1.000	1.000	—	0.690	0.920	1.000	0.342	0.104	0.580	0.995	0.005	0.000
ChaCha20+SPECK	0.6648	1.000	1.000	1.000	—	0.652	0.870	1.000	0.342	0.104	0.580	1.000	0.000	0.000
ECC+AES-128+SPECK	0.6245	1.000	1.000	1.000	—	0.547	0.730	1.000	0.326	0.092	0.560	0.996	0.004	0.000
AES-128+SPECK	0.6120	1.000	1.000	1.000	—	0.510	0.680	1.000	0.326	0.092	0.560	1.000	0.000	0.000
ECC+SPECK	0.5751	1.000	1.000	1.000	—	0.360	0.480	1.000	0.365	0.125	0.606	0.995	0.005	0.000
SPECK	0.5626	1.000	1.000	1.000	—	0.323	0.430	1.000	0.365	0.125	0.606	1.000	0.000	0.000
AES-128+HIGHT	0.5397	1.000	1.000	1.000	—	0.495	0.660	1.000	0.124	0.007	0.242	1.000	0.000	0.000
ECC+RECTANGLE	0.5194	1.000	1.000	1.000	—	0.300	0.400	1.000	0.258	0.026	0.491	0.998	0.002	0.000
RECTANGLE	0.5069	1.000	1.000	1.000	—	0.262	0.350	1.000	0.258	0.026	0.491	1.000	0.000	0.000
ECC+SPECK+HIGHT	0.4933	1.000	1.000	1.000	—	0.383	0.510	1.000	0.098	0.005	0.191	0.999	0.001	0.000
SPECK+HIGHT	0.4808	1.000	1.000	1.000	—	0.345	0.460	1.000	0.098	0.005	0.191	1.000	0.000	0.000
ECC+HIGHT	0.4563	1.000	1.000	1.000	—	0.232	0.310	1.000	0.136	0.007	0.265	0.999	0.001	0.000
HIGHT	0.4438	1.000	1.000	1.000	—	0.195	0.260	1.000	0.136	0.007	0.265	1.000	0.000	0.000
RECTANGLE+HIGHT	0.4282	1.000	0.999	1.000	—	0.285	0.380	1.000	0.000	0.000	0.000	1.000	0.000	0.000

FIGURE I.1. Full Device fit, security fit, and application fit score breakdown for every candidate cipher and cascade.